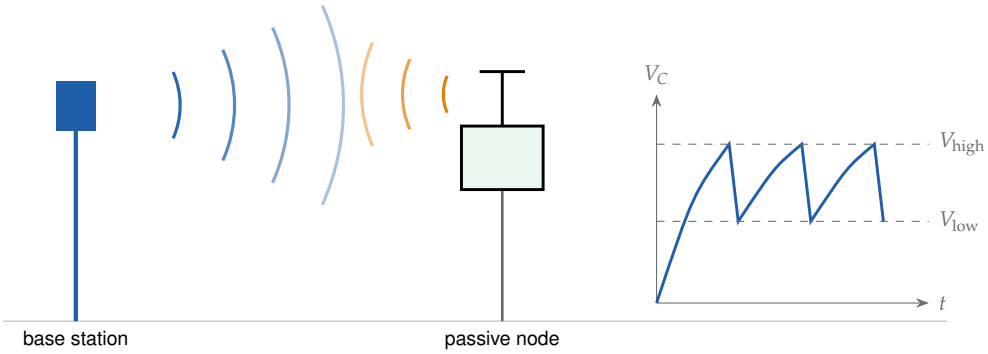
A TEACHING TEXT

Amber and 6G Ambient IoT

From backscatter physics and energy harvesting
to protocols and a discrete-event simulator



A study guide to the SenSys 2026 demo abstract
“Demo Abstract: Amber: 6G Ambient IoT Simulator”
M. Manafova, T. Voigt and L. Mottola

and to the open-source Amber simulator, version 0.1.0

This is an independent teaching text. It was not written by the Amber authors.
Statements about the code were checked against the source; the behaviours that matter were also tested by running it.

Preface

The problem this book solves. A device that has no battery, harvests a few microwatts from radio waves, and answers a base station by *reflecting* the base station's own signal is easy to describe in one sentence and surprisingly hard to reason about. Its ability to talk depends on its stored energy; its stored energy depends on the radio channel; the radio channel depends on distance, antennas and on *what the device itself is doing* at that moment; and the protocol that sits on top must cope with hundreds of such devices that wake up and fall asleep at unpredictable times. No single textbook chapter covers all of this. The *Amber* simulator was built to let researchers explore exactly this tangle, and its authors describe it in a two-page SenSys demo abstract.

What this book does. We use Amber as a vehicle for teaching the *whole* subject. For every part of the simulator we first explain the *theory* it abstracts—the physics of antennas, path loss, backscatter, rectifiers and capacitors, the mathematics of random access and interference cancellation, the machinery of discrete-event simulation—and only then show *how Amber models it*, where the model is exact, where it is a deliberate simplification, and where the code does something different from what the textbook says. You will not find page after page of source listings: the aim is that after reading you could re-derive every equation inside Amber and write a simulator like it yourself.

How the book is organised.

Part I (Chapters 1–2).
The idea of Ambient IoT, 3GPP's view of it, and a one-chapter tour of Amber so that you always know where a topic fits.

Part II (Chapters 3–8).
The physics: decibels and noise, antennas, path loss, backscatter communication, RF energy harvesting, and energy storage in a capacitor with threshold-driven intermittent operation.

Part III (Chapters 9–11).
Sharing the medium: ALOHA and its framed variants, capture, interference and successive interference cancellation, and how these combine into registration and polling protocols.

Part IV (Chapters 12–13).
Simulation methodology (discrete-event simulation with SimPy) and the architecture of Amber.

Part V (Chapters 14–18).
A module-by-module deep dive: topology and coverage, the node controller, the backscatter module, the base-station engine, and packet analysis.

Part VI (Chapters 19–20).
Experiments: the four shipped demos, our re-creation of the paper's node-density experiment, and how to build your own protocol on top of Amber.

Part VII (Chapter 21).
A critical assessment: assumptions, limits and a catalogue of subtle behaviours that

anyone using the code should know.

Appendices.
Derivations, a formula and parameter sheet, an annotated reading of the paper, exercise solutions and a glossary.

Conventions used throughout. The coloured boxes have fixed meanings.

Key idea
A concept you must carry forward. If you only read the boxes, you still get the skeleton of the subject.

Intuition
A physical or mathematical way of *seeing* why a result is true, usually before or after a derivation.

Worked example
A calculation with real Amber numbers, each checked against the simulator or by an independent hand calculation.

Code reality check
Where the code deviates from the textbook theory, or from what its own documentation says. These are the most valuable boxes if you intend to *use* Amber for research. Each was reproduced by a small experiment (see Chapter 21).

Where this lives in the code
A short table linking a concept to the module, class or function that implements it.

Identifiers set in `typewriter` are names in the Amber source. Section and chapter exercises are collected at the end of each chapter; worked solutions are in Appendix D.

A note on honesty. Amber is a young research prototype (version 0.1.0). The accompanying paper is a demo abstract: it deliberately describes the simulator at a high level and reports one experiment. We therefore draw a clear line between (i) what the paper states, (ii) what the code implements, and (iii) what we ran ourselves to find out. Numbers reported as *measured* come from runs of the unmodified simulator; their scripts are in the `textbook/code` directory, so you can reproduce every figure and table.

Prerequisites. Undergraduate-level probability (random variables, the Poisson distribution), basic circuit analysis (Ohm's law, a capacitor's $i = C\,\mathrm{d}v/\mathrm{d}t$), elementary Python, and a first course in communications or signals. We introduce everything else.

Reading Roadmap

Different readers need different routes. The table suggests three.

If you are…	Read…
New to Ambient IoT and want the concepts	Chapters 1, 2, 3, 6, 7, 8, then 9–11. Skim Part V.
A wireless/networking researcher who will *use* Amber	Chapters 2, 6, 8, 10, 12, 13, then all of Part V and Chapter 21. Do Chapter 19 and 20 hands-on.
Extending Amber (new protocol, energy model, channel)	Chapters 13, 15–17, 20, 21, and Appendix B. The *Code reality check* boxes in Chapters 8, 10, 14 and 17 matter most.

The one-paragraph summary of the whole book

An Ambient IoT node lives a double life. In the *energy* domain it is a capacitor that a weak radio-frequency (or solar) source fills slowly and a duty-cycled load drains quickly, switching on at V_{high} and off at V_{low}. In the *communication* domain it is a passive reflector that can answer a base station only when it is awake, only in the slot the base station advertised, and only with the power budget the two-way path loss leaves it. The base station cannot see the node's energy state, so the protocol must be robust to nodes that are absent, collide, or arrive late. Amber simulates the coupling between these two domains millisecond by millisecond.

Notation and Symbols

Quantities are written in the SI system unless stated. Logarithmic powers use dBm (decibels relative to one milliwatt), antenna gains use dBi (relative to an isotropic radiator), and pure ratios use dB. The right column shows the corresponding name in the Amber source where there is one.

Symbol	Meaning	Amber name
f_c, λ, c	carrier frequency, wavelength $\lambda = c/f_c$, speed of light	`freq_hz`, `C`
d, d_{2D}, d_{3D}	distance; horizontal and slant (3-D) distance	`distance_m`
h_{BS}, h_{UT}	antenna heights of base station and node	`Sector.height`, `Node.height`
G, $G_{\max}$	antenna gain (linear); peak gain	`antenna_gain_dbi`
φ, $\varphi_{3\text{dB}}$, A_m	angular error, half-power beamwidth, pattern floor	`beamwidth_deg`
$L(d)$, $\mathrm{PL}(d)$	path loss (linear ratio ≤ 1; dB)	`fspl_lin`, `urban_macro_loss_db_38_901`
P_{BS}	transmit power of a base-station sector	`Sector.power`
P_{dl}	downlink power arriving at a node	`best_pw_dbm`
η	power split: fraction reflected (backscattered)	`Node.efficiency`
P_h, P_{bs}	harvested and backscattered power at a node	`harvesting_power_dbm`, `backscatter_power_dbm`
P_{ul}	backscatter power received at the base station	`per_sector_powers`
k_B, T_0, B, F	Boltzmann constant, 290 K, noise bandwidth, noise factor	`thermal_noise_watts`
N, γ, γ_{req}	noise power, SINR, required SINR	`required_sinr_db`
ε	SIC residual: $1-$ cancellation factor	`cancellation_factor`
C, V, $E = \frac{1}{2}CV^2$	capacitance, capacitor voltage, stored energy	`CapacitorParams.C`
R_{s}, R_{l}	series and leakage resistance	`R_series`, `R_leakage`
V_{src}	equivalent source voltage of the harvester	`voltage_source`
V_{high}, V_{low}, $V_{\max}$	activation, shut-down and clamp voltages	`thresholds_v`, `voltage_max`
$\tau_c = R_{\text{s}}C$, $\tau_\ell = R_{\text{l}}C$	charging and leakage time constants	–
Δt	simulation time step (1 ms)	`dt`
I_L	load current of the node	`CurrentsA`
S, n, K	slots per frame, contending nodes, number of subcarrier shifts	`rx_slots`, `subcarrier_shift`
g	offered load of a random-access system (attempts per slot)	–

Contents

Preface i

Reading roadmap iii

Notation and symbols iv

I The Ambient IoT Idea 1

1 Why Ambient IoT? 2
- 1.1 The battery is the bottleneck . . . 2
- 1.2 Three ancestors . . . 3
- 1.3 3GPP's view of Ambient IoT . . . 3
- 1.4 Four properties that make these devices hard to reason about . . . 4
- 1.5 The simulation gap . . . 5
- 1.6 Chapter summary . . . 6

2 Amber at a Glance 7
- 2.1 Three entity types . . . 7
- 2.2 The life of a node . . . 7
- 2.3 The four design principles . . . 8
- 2.4 Time scales . . . 9
- 2.5 One poll, end to end . . . 10
- 2.6 What is simulated and what is abstracted . . . 11
- 2.7 Map of the book to the code . . . 11
- 2.8 Chapter summary . . . 11

II The Physics of Radio and Energy 13

3 Decibels, Noise and the Link Budget 14
- 3.1 Why a logarithmic scale? . . . 14
  - 3.1.1 The conversion Amber uses . . . 15
  - 3.1.2 Rules of thumb you should memorise . . . 15
- 3.2 The link budget . . . 15
- 3.3 Thermal noise . . . 16
- 3.4 SNR, SINR and sensitivity . . . 17
- 3.5 The dynamic-range problem of backscatter . . . 17
- 3.6 Chapter summary . . . 18

4 Antennas, Sectors and Geometry 19

4.1 Gain: concentrating a fixed amount of power . . . 19
4.2 The 3GPP sector pattern . . . 19
4.3 Sectorisation . . . 21
4.4 Geometry: bearings and angular errors . . . 21
4.5 Node antennas and reciprocity . . . 22
4.6 Chapter summary . . . 23

5 Path Loss: From Friis to 3GPP Urban Macro **24**
5.1 Free-space propagation: Friis . . . 24
5.2 Beyond free space . . . 25
5.3 3GPP TR 38.901 Urban Macro (UMa) . . . 25
5.4 Medium attenuation . . . 27
5.5 What Amber assumes about the channel . . . 27
5.6 Chapter summary . . . 27

6 Backscatter Communication **29**
6.1 Active radio versus backscatter radio . . . 29
6.2 Modulating a reflection . . . 30
6.3 The two-way link budget . . . 30
6.4 Splitting power between harvesting and reflecting . . . 32
6.5 Self-interference and frequency-shifted backscatter . . . 32
6.6 Receiving with no receiver: the passive downlink . . . 34
6.7 Topologies and duplexing . . . 34
6.8 Chapter summary . . . 35

7 Energy Sources and RF Harvesting **37**
7.1 Where can the energy come from? . . . 37
7.2 The RF harvesting chain . . . 37
7.3 Amber's harvesting model . . . 38
7.3.1 From power to an equivalent source . . . 38
7.3.2 Three power modes . . . 39
7.4 The external source . . . 39
7.5 When is harvesting updated? . . . 40
7.6 Chapter summary . . . 40

8 Energy Storage: The Capacitor and Intermittent Operation **42**
8.1 Why a capacitor? . . . 42
8.2 The circuit . . . 42
8.3 Solving it . . . 43
8.4 Discretisation: how Amber steps the equation . . . 44
8.4.1 The four step types . . . 45
8.4.2 Discharge . . . 45
8.5 Thresholds, hysteresis and the energy window . . . 45
8.5.1 The duty-cycle theorem . . . 46
8.5.2 What the node forgets . . . 47
8.6 Chapter summary . . . 48

III Sharing the Medium 50

9 Random Access: From ALOHA to Framed Slots 51
9.1 The problem: many talkers, one channel . . . 51
9.2 Pure ALOHA . . . 51
9.3 Slotted ALOHA . . . 52
9.4 Framed slotted ALOHA . . . 52
9.5 Estimating the number of contenders . . . 54
9.6 An energy-aware twist . . . 54
9.7 Several orthogonal channels: subcarrier shifts . . . 55
9.8 From the model to the simulator . . . 55
9.9 Chapter summary . . . 55

10 Capture, SINR and Successive Interference Cancellation 57
10.1 From “collision” to SINR . . . 57
10.2 Successive interference cancellation . . . 58
10.2.1 The principle . . . 58
10.2.2 The SINR sequence . . . 58
10.3 How Amber resolves collisions . . . 59
10.4 Does SIC matter? A controlled experiment . . . 61
10.5 What the threshold model abstracts away . . . 62
10.6 Chapter summary . . . 62

11 Protocols: Discovery, Registration and Polling 64
11.1 Command-driven networks . . . 64
11.2 Amber’s built-in protocol . . . 64
11.3 Broadcast versus unicast polling . . . 65
11.3.1 Airtime is rarely the bottleneck . . . 66
11.4 Limited visibility . . . 66
11.5 The discovery phase in detail . . . 67
11.5.1 What the paper reports . . . 67
11.5.2 A mean-field model . . . 67
11.5.3 Re-creating the experiment with Amber . . . 67
11.6 Design principles for energy-aware protocols . . . 68
11.7 Chapter summary . . . 69

IV Simulation Methodology and Architecture 70

12 Discrete-Event Simulation and SimPy 71
12.1 Models, state and time . . . 71
12.2 SimPy in ten minutes . . . 72
12.2.1 Order of events at the same instant . . . 72
12.3 How Amber uses SimPy . . . 73
12.4 Precomputation and scalability . . . 73
12.5 Randomness and replication . . . 74
12.6 Verification, validation and what Amber has had . . . 74
12.7 Chapter summary . . . 75

13 The Architecture of Amber 76

13.1 Objects at run time . . . 76
13.2 Interface contracts . . . 76
13.3 Data dictionaries . . . 78
13.4 The wiring recipe . . . 78
13.5 One simulated millisecond . . . 79
13.6 Chapter summary . . . 79

V Inside the Simulator **81**

14 Topology and the Coverage Engine **82**
14.1 The topology objects . . . 82
14.1.1 `Node` . . . 82
14.1.2 `Sector` and `BaseStation` . . . 83
14.2 The CoverageMap . . . 83
14.2.1 The coverage grid (for pictures) . . . 83
14.2.2 Downlink to the nodes . . . 83
14.2.3 Harvest and backscatter power . . . 84
14.2.4 Uplink to the base stations . . . 84
14.2.5 What this means for multiple base stations . . . 84
14.3 Numerical conventions . . . 85
14.4 Chapter summary . . . 85

15 The Node Controller **87**
15.1 Configuration . . . 87
15.2 The main loop . . . 88
15.3 The working-mode state machine . . . 89
15.4 One real energy cycle . . . 90
15.5 Bookkeeping and logging . . . 91
15.6 Chapter summary . . . 91

16 The Backscatter Module: Node-Side Protocol **93**
16.1 Division of labour . . . 93
16.2 Receiving a command . . . 93
16.3 The default handler . . . 94
16.4 When may the node transmit? . . . 94
16.5 Transmitting . . . 95
16.6 Customising the protocol . . . 96
16.7 Logging . . . 96
16.8 Chapter summary . . . 96

17 The Base-Station Engine **98**
17.1 Role and constructor . . . 98
17.2 The schedule language . . . 98
17.3 TX and RX windows . . . 99
17.4 Telling nodes when to answer: the RX-slot look-ahead . . . 99
17.4.1 Consequence: half of the contenders sit the frame out . . . 100
17.5 Policy mode . . . 101
17.6 Acknowledgements and registration state . . . 101

17.7 Counters and logs . . . 101
17.8 When the base station stops . . . 101
17.9 Chapter summary . . . 102

18 Packet Analysis and Performance Metrics **104**
18.1 Metrics . . . 104
18.1.1 How the demos compute them . . . 104
18.1.2 Name clash: "DER" . . . 105
18.2 The `packet_analysis` module . . . 105
18.3 Real-time versus post-hoc classification . . . 105
18.4 Practical guidance . . . 106
18.5 Chapter summary . . . 107

VI Experiments and Critical Assessment **108**

19 Reading the Experiments **109**
19.1 The four demos at a glance . . . 109
19.2 A budget table that predicts the results . . . 109
19.3 Measured results . . . 110
19.4 Case study: adaptive framed-slotted ALOHA . . . 110
19.5 The paper's experiment . . . 112
19.6 Things to try . . . 112
19.7 Chapter summary . . . 112

20 Extending Amber: Writing Your Own Protocol **114**
20.1 Anatomy of a user protocol: adaptive ALOHA . . . 114
20.1.1 Node half . . . 114
20.1.2 Base-station half . . . 114
20.1.3 Wiring . . . 115
20.2 Case study: does energy-aware polling help? . . . 115
20.3 A checklist for new experiments . . . 116
20.4 A gallery of mistakes . . . 117
20.5 Where the model invites extension . . . 117
20.6 Chapter summary . . . 118

21 Assumptions, Limits and Verified Caveats **119**
21.1 Catalogue . . . 119
21.1.1 Channel and radio . . . 119
21.1.2 Energy and node behaviour . . . 120
21.1.3 MAC, SIC and software . . . 121
21.2 Which caveats matter for which question? . . . 122
21.3 Ten validity questions for any simulation result . . . 122
21.4 Where to go next . . . 123
21.5 Chapter summary . . . 123

A Derivations **124**
A.1 Solution of the capacitor equation . . . 124
A.2 Maximum power into the capacitor . . . 124
A.3 Duty-cycle theorem . . . 124

A.4 Framed slotted ALOHA: slot statistics . . . 124
A.5 SIC: the ratio rule . . . 125
A.6 Sidebands of a switched reflection . . . 125
A.7 Friis in two lines . . . 125
A.8 3GPP UMa continuity at the breakpoint . . . 125

B Formula and Parameter Reference **126**
B.1 Formula sheet . . . 126
B.2 Default values at a glance . . . 127
B.3 The demos' electrical and timing parameters . . . 128

C The Paper, Annotated **129**
C.1 Abstract and introduction . . . 129
C.2 Section II: Amber overview . . . 130
C.3 Section II (continued): the physical layer and devices . . . 131
C.4 Section III: the experiment . . . 132
C.5 What the abstract does not say . . . 132

D Solutions to the Exercises **133**

E Glossary **140**

F Reproducing the Results **143**
F.1 Setting up . . . 143
F.2 Scripts . . . 143
F.3 Building the book . . . 143
F.4 What is deterministic . . . 144
F.5 Repository map . . . 144

Bibliography **145**

List of Figures

2.1 Energy and information flow in Amber. The base station radiates a carrier; a node splits the received power between harvesting and reflection. Energy flows downward through the harvester into the capacitor; the controller turns capacitor voltage into duty-cycled activity and permits the protocol module to answer. . . 8
2.2 Capacitor voltage of the three nodes of the broadcast demo (measured; random seed fixed for the figure). Node 0 and Node 1 harvest only from the base station's carrier at 20 m and 30 m. Node 2 is 100 m away, too far to power itself from RF alone, and runs on its *solar* trace instead. Dashed lines: $V_{\text{high}} = 1.7\,\text{V}$ and $V_{\text{low}} = 1.3\,\text{V}$. . . 9
2.3 Module map. Solid arrows: Python imports. Dashed arrows: run-time interaction through SimPy interrupts and shared attributes (Chapter 13). . . 11

4.1 The pattern of Eq. (4.2) for $G_{\max} = 15$ dBi and three beamwidths. Dots mark the half-power points at $\pm\varphi_{3\text{dB}}/2$ ($G_{\max} - 3$ dB). Beyond about 1.58 $\varphi_{3\text{dB}}$ the pattern is flat at $G_{\max} - A_m$. 20
4.2 A coverage map produced by Amber's `CoverageMap` for an illustrative deployment (three base stations of three 65° sectors, 30 dBm, 17 dBi, 924 MHz, 15 random nodes), in the style of the paper's Figure 1. Colour is the *best* received power among all sectors (Chapter 14). This figure is our own example, not the paper's data. 21
4.3 Compass versus mathematical angles. The sector's boresight points at compass azimuth 30° (math 60°); a target at math bearing 12° is $\Delta\varphi = 48°$ off boresight and sees the pattern attenuation $12(48/65)^2 = 6.5$ dB for a 65° sector. 22

5.1 Path loss at 924 MHz with $h_{BS} = 25$ m, $h_{UT} = 1.5$ m, computed by Amber's own functions. Free space (dashed) and UMa LOS agree closely up to the breakpoint $d'_{BP} \approx 148$ m, after which LOS steepens to a 40 dB/decade slope. NLOS is ≈ 11–22 dB worse. The shaded region at left marks the code's non-physical behaviour below 10 m (see the reality check below). 26

6.1 A backscatter front-end. A digital switch connects the antenna either to a matched path that feeds the rectifier (energy harvesting) or to a reflecting termination. The data bits control the switch and thereby impose themselves on the reflected carrier. There is no oscillator, mixer or power amplifier. 30
6.2 Downlink power at the node and uplink backscatter power at the base station against distance, for the demo's north-facing sector (46 dBm, 15 dBi, $\eta = 0.7$), computed by Amber. The uplink starts ∼ 45 dB lower and falls twice as fast. With the demo's −100 dBm sector sensitivity the link would close out to about 300 m; the node would be energy-limited long before. 31
6.3 Frequency-domain picture of the dual-tone downlink and frequency-shifted backscatter. A clean carrier on a dedicated subcarrier powers and illuminates the nodes; one data-bearing subcarrier carries the downlink. Each node reflects its reply at an offset $f_{s,k}$ from the carrier (shown on one side only; real sidebands appear on both). Different offsets are *orthogonal*; nodes with the same offset overlap in frequency and time and must be separated by capture or SIC (Chapter 10). 33

8.1 Equivalent circuit of Amber's energy store. Typical demo values: $R_s = 5\,\text{k}\Omega$, $R_l = 100\,\text{k}\Omega$, $C = 300\,\mu\text{F}$ (200 µF in the SIC demo); the load current I_L takes values 0.14 mA–5 mA depending on the node's state. 43
8.2 Charging from empty for three source voltages. Lines: Amber's discrete update; dots: the exact solution Eq. (8.4) (they coincide). The top curve is clipped by $V_{\max} = 2$ V. The bottom curve ($V_{\text{src}} = 0.66$ V) saturates far below V_{low} and never activates a node. 44
8.3 Hysteretic two-state behaviour of an energy-harvesting node. The 0.4 V gap between thresholds sets the size of the usable energy window. 46
8.4 Power delivered into C (solid) versus the power credited by the active-mode step (dashed), both relative to $P_h = V_{\text{src}}^2/R_s$. 47
8.5 Zero-load charging from 1 V: the charge-only step coincides with the exact RC solution; the charge-and-discharge step charges faster. 48

9.1 Left: throughput of pure and slotted ALOHA. Right: the success probability of a single node in a frame of S slots when n nodes contend (Eq. (9.7)). 52
9.2 Slot efficiency of framed slotted ALOHA against the load ratio n/S. All curves peak near $n/S = 1$ at about 0.37–0.39; the $S \to \infty$ limit is re^{-r}. 53

10.1 SINR of the stronger of two colliding packets against the power gap, for thermal noise -88 dBm. At -60 dBm the curve is interference-limited; at -80 dBm it saturates because noise dominates. The dashed line is Amber's default requirement of 3 dB. 58
10.2 SIC on three colliding packets at -60, -65 and -70 dBm with a -88 dBm noise floor and $\gamma_{\text{req}} = 3$ dB. Bar heights are *linear* power. Without cancellation only the strongest would be decoded (capture); with SIC all three are. 59

11.1 Message sequence of Amber's default protocol for one node. The two phases correspond to those of the paper. In each phase the *reply* is a backscatter packet inside an RX window the base station advertised in the command. 65
11.2 Percentage of nodes registered after one, two and four discovery rounds against node count; mean over 10 runs with a 10–90% band; left without SIC, right with SIC. Dashed: the mean-field model of Eq. (11.1) with $p_a = \frac{1}{2}$, $S = 10$, $K = 3$. This is *our* re-creation of the paper's experiment (the paper's frame length and number of rounds are unknown), so only the qualitative trend is comparable. 68

13.1 Runtime object graph. Solid arrows are calls or attribute reads; the arrows labelled `interrupt` are SimPy messages. The coverage map is shared by all nodes; the `Capacitor`, `Controller` and `BackscatterModule` are instantiated once per node; a `BSBehavior` exists per base station. 77

15.1 The working-mode FSM. Numbers are the demo durations and currents. A node that is not yet registered skips sensing and processing (upper loop) and waits for a command (*send_id*) to give it a slot. A freshly registered node that has no sensor value yet returns from *wait_slot* to *sensing* first. 89
15.2 A 120 ms window of Node 0 (measured). Top: capacitor voltage; middle: the node's FSM state per millisecond (gold: waiting for a slot; grey: switched off and charging); bottom: the base station's frame (TX 5 ms, two RX windows of 10 ms). Black triangles mark the instants at which the node backscatters. Each packet is delivered at the *start* of the node's transmit state, inside an RX window. 90

17.1 RX-slot look-ahead: a command at $t = 0$ advertises eight windows for a four-window frame 100

19.1 Adaptive framed-slotted ALOHA over 15 s (from the repository's committed output). Top: frame size per frame; it collapses from 20 to about 8 within 1.5 s as the BS learns how few nodes are contending. Bottom: decoded and collided packets per 0.5 s bin. 111

Part I

The Ambient IoT Idea

1

Why Ambient IoT?

In this chapter you will learn

- why batteries, not radios, are the real bottleneck of massive IoT;
- the lineage—RFID, ambient backscatter, cellular IoT—that leads to *3GPP Ambient IoT*;
- how 3GPP classifies Ambient IoT devices (A, B, C) and connects them to a network;
- the four properties that make these devices hard to reason about, and in particular the coupling of *energy state* to *communication ability*;
- what gap Amber fills, and what questions a simulator of this kind can and cannot answer.

1.1 The battery is the bottleneck

The promise of the Internet of Things is to attach a communicating sensor to everything worth measuring: soil moisture in a field, a pallet in a warehouse, a strain gauge in a bridge, a temperature probe in a cold chain. Forecasts speak of tens of billions of devices. Two properties dominate whether such a number is reachable: the *cost* of each device and the *labour* of keeping it alive.

Consider the energy budget of the humble coin cell. A typical lithium coin cell stores about 225 mAh at 3 V, that is

$$E_{\text{cell}} \;=\; 0.225\,\text{Ah} \times 3\,\text{V} \times 3600\,\text{s/h} \;=\; 2\,430\,\text{J}.$$

How long does a coin cell last?

A sensor whose *average* power draw is 100 µW drains the cell in

$$t = \frac{2430\,\text{J}}{100 \times 10^{-6}\,\text{W}} = 2.43 \times 10^{7}\,\text{s} \approx 281\ \text{days}.$$

At 10 µW the same cell lasts about 7.7 years (ignoring self-discharge, which in practice shortens this). A cellular radio that transmits at 23 dBm ≈ 200 mW, even for a fraction of a second per day, quickly dominates the budget.

Now scale up. A deployment of 10^6 devices with a five-year battery life needs, on average, $10^6/(5 \times 365) \approx 548$ battery replacements *per day*, each requiring a human to find and open a device that may be inside a wall, on a streetlight or in a sealed container. Batteries also contain materials one would rather not scatter by the billion. The conclusion is simple: for the very largest and cheapest deployments, the only sustainable design is a device that needs *no battery to be replaced*.

Key idea

Massive IoT is not limited by how cheaply we can make a radio. It is limited by how cheaply we can *power* one for a decade without touching it. Ambient IoT attacks the problem at its root: reduce the device's power to the microwatt regime and feed it from the environment.

1.2 Three ancestors

Ambient IoT did not appear from nowhere. It sits where three technologies meet.

RFID. A passive UHF RFID tag contains an antenna and a tiny chip, but no battery. A *reader* radiates a continuous wave (the carrier). The tag's antenna captures some of that power, a rectifier turns it into a DC supply for the chip, and the chip answers by *switching its antenna load* so that the reflected carrier is modulated with the tag's data. The idea goes back to Stockman's 1948 paper on "communication by means of reflected power" [22]. Two features of RFID will recur throughout this book: *the reader talks first*, and *the tag answers by reflecting*. RFID readers inventory many tags using *framed slotted ALOHA* (Chapter 9).

Ambient backscatter. In 2013 Liu et al. showed that devices could communicate by backscattering *ambient* signals such as TV broadcasts, with no dedicated reader at all [14]. The survey by Huynh et al. [12], cited by the Amber paper, organises the field into *monostatic*, *bistatic* and *ambient* topologies depending on where the carrier comes from (Chapter 6).

Cellular IoT. NB-IoT and LTE-M bring standardised, wide-area, secure connectivity to low-power devices, but they are *active radios*: they generate their own carrier, which costs tens to hundreds of milliwatts while transmitting. They therefore need a battery.

	Passive UHF RFID	3GPP Ambient IoT	NB-IoT / LTE-M
Energy source	RF from reader	RF, light, … (+ small storage)	Battery
Uplink radio	Backscatter	Backscatter (or low-power active)	Active transmitter
Typical active power	∼µW–tens of µW	∼1 µW–few hundred µW	$\gg$ mW
Who initiates	Reader	Base station / reader	Device or network
Network	Dedicated readers	Cellular infrastructure	Cellular infrastructure

Orders of magnitude only; the Ambient-IoT column follows the 3GPP design targets quoted below.

Ambient IoT can be read as "RFID's energy and backscatter physics, with cellular IoT's infrastructure and standardisation". That combination is what 3GPP set out to specify.

1.3 3GPP's view of Ambient IoT

3GPP studied *Ambient IoT* (originally "ambient power-enabled IoT") in Release 18 and Release 19 and is extending it in Release 20, with the vision of carrying it into 6G. The Amber paper cites the overview by López et al. [15]. Public 3GPP material (the study report TR 38.848, the Release-19 solutions study TR 38.769, and the device radio specification TS 38.191) defines the following

device categories by how much energy they can store and whether they can generate a radio signal on their own.

Type	Energy storage	Independent signal generation	Uplink
Device A	none	no	backscatter of an external carrier
Device B	yes (small)	no	backscatter; stored energy may be used to amplify the reflection
Device C	yes (small)	yes (active RF components)	active transmission

In the Release-19 normative work the focus was on the lowest-power class, described in public summaries as a device with *about* 1 μW *peak power consumption*, some energy storage and an RF *envelope-detector* receiver, whose uplink is backscatter on a carrier *provided externally* (typically by the base station). Release 20 extends the scope towards higher-power classes. The storage in B and C is *orders of magnitude smaller* than in an NB-IoT device; this smallness is exactly why *capacitor* models matter later in this book.

Topologies. The study also considered how devices attach to the network:

- **T1.** base station ↔ Ambient IoT device (direct);
- **T2.** base station ↔ *intermediate node* ↔ device;
- **T3.** base station ↔ *assisting node* ↔ device ↔ base station, in which a separate node assists the downlink (for example by supplying a carrier) while the device answers the base station directly—a *bistatic*-style arrangement (we describe the roles loosely; consult the report for the exact definition);
- **T4.** user equipment ↔ device.

Amber implements topology T1 only: base stations talk directly to nodes, and the same base station that radiates the carrier also receives the reflection.

Where do Amber's nodes sit in this taxonomy?

Amber's `Node` carries a `node_type` field ("passive" or "active"), but the field is currently *cosmetic*: it only chooses a colour when a coverage map is drawn. What really defines an Amber node is its behaviour: it has a capacitor (so it has storage), it uplinks by reflecting the base station's carrier (no signal generation), and it does not amplify the reflection. That makes it closest to a storage-equipped backscatter device (3GPP's Device B / "type 1"). Note, however, that the *example currents* in the shipped demos (a few mA while transmitting; see Chapter 15) are far above the microwatt targets of Release 19: they model a sensor-class node, not an ultra-low-power tag. All such numbers are configurable.

1.4 Four properties that make these devices hard to reason about

It is worth stating, once and clearly, what is *different* about an Ambient IoT device from a conventional wireless node. Each property below reappears as a design decision inside Amber.

P1 — Energy intermittency. The harvested power is tiny (microwatts to a milliwatt) and may vary. The device cannot run continuously; it accumulates charge, wakes up, does a little

work and falls asleep again. Whether it is *present* at a given moment is a stochastic process in its own right. (Chapters 7–8.)

P2 — Reflective uplink. With no oscillator, mixer or power amplifier the device can only *modulate the reflection* of someone else's signal. Its transmit "power" is a fraction of what it received, and the signal it creates is subject to the path loss of *both* hops (Chapters 5 and 6).

P3 — Reader-driven medium access. A device that cannot afford a receiver chain cannot carrier-sense, and without a carrier it cannot transmit at all. The base station therefore sequences everything: commands, reply slots, acknowledgements (Chapters 9–11).

P4 — Massive scale and limited visibility. There may be hundreds of devices per cell, and the base station *cannot observe their energy state*. It sees only whether a reply arrived.

The Amber abstract compresses the consequence of these four properties into one sentence that we will return to repeatedly:

The central coupling

In an Ambient IoT network, *a node's energy state directly determines its ability to communicate*.

Suppose a base station broadcasts a request and expects one hundred replies. If sixty devices are momentarily below their activation threshold, the protocol sees sixty "failures" that are *not* radio failures at all. A simulator that models radio but not energy would blame the channel; a simulator that models energy but not protocols would not know that anything failed. Seeing both at once is what Amber is for.

1.5 The simulation gap

Why not just build a testbed? The Amber paper gives the answer in its introduction: *the scale of Ambient IoT deployments, combined with unpredictable energy availability and limited visibility into device state, makes field testing and validation impractical.* Simulation is the natural tool, but existing ones leave a gap, which the authors characterise as follows.

- **Signal-level simulators** such as Sionna [11] and ray-tracing platforms [23] model the radio channel in great detail but, for lack of energy-harvesting, capacitor and state-dependent power-consumption models, do not support Ambient IoT nodes.
- **Network simulators** that study MAC and PHY performance typically assume *always-available power*.
- **Energy-harvesting studies** usually omit protocol-level communication.

Amber integrates the three missing pieces in one open-source, event-driven simulator: *RF energy harvesting, capacitor storage dynamics and backscatter communication*. The authors state it is, to their knowledge, the first to do so. It lets a researcher ask system-level questions of the form:

> *If I put N nodes in this cell, with this antenna, this capacitor and this protocol, what fraction of them can I register, how often can each report, and what is lost to collisions versus to empty capacitors?*

What a system-level simulator is *not*. It is not a bit-accurate PHY simulator: Amber does not simulate waveforms, modulation, coding or fading (Chapters 6 and 21). It is not hardware-validated either: the paper is a demo abstract and reports no comparison with measurements. Treat Amber's numbers as *comparisons between designs under shared, explicit assumptions*, not as

predictions of absolute field performance. Learning to do this honestly is one of the goals of this book.

1.6 Chapter summary

- Batteries limit massive IoT; Ambient IoT removes them by combining RF/solar harvesting, small storage and backscatter uplinks.
- 3GPP defines Device A (no storage, backscatter), B (storage, backscatter), C (storage, active); Release-19 targeted a $\sim 1\,\mu W$ class.
- Four properties—intermittency, reflective uplink, reader-driven access, scale with limited visibility—couple energy to communication.
- Amber closes the gap between signal-level and energy-harvesting studies with a system-level, discrete-event model.

Exercises

Exercise 1

A 225 mAh, 3 V cell powers a sensor with an average load of 20 µW. Estimate the lifetime in years. By what factor must the average load fall to reach a 15-year lifetime?

Exercise 2

A city deploys 2×10^6 battery-powered sensors with a four-year battery life. How many replacements per day does the municipality face on average? Why might this figure *underestimate* the labour involved?

Exercise 3

A tag stores energy in a 100 µF capacitor, backscatters a carrier supplied by a base station, and cannot generate any RF signal itself. Which 3GPP device category fits best? What would change if it contained a local oscillator and a small power amplifier?

Exercise 4

Explain in three sentences why a device with no energy storage cannot send anything when the reader's carrier is absent, and list two protocol-design consequences of "the reader talks first".

2

Amber at a Glance

In this chapter you will learn

- the three entity types Amber simulates and how they interact;
- the life cycle of a single node, told with real simulator output;
- the four design principles stated in the paper and how each is realised;
- the time scales involved, and a one-page walk-through of a single poll;
- a map of the rest of the book, so every later detail has a home.

2.1 Three entity types

The paper states that Amber "models three entity types: nodes, base stations, and the communication links between them." Let us fix what each one is.

Nodes. Stationary, passive, battery-free devices. A node is characterised by its *position*, its *antenna gain*, its *receiver sensitivity* (the weakest command it can notice) and a *power split ratio* that says how the received RF power is divided between energy harvesting and backscatter reflection. Its behaviour is driven by two voltage thresholds, V_{high} and V_{low} (Chapter 8).

Base stations. Each base station has several *sectors*. A sector is described by its azimuth, beamwidth, height, antenna gain, transmit power, receiver sensitivity and a 3GPP-compliant antenna pattern (Chapter 4). The base station provides the *carrier* (power and backscatter illumination), sends *commands* to nodes, and *receives* the reflected replies.

Links. "Communication links are computed bidirectionally: downlink for wireless power transfer and carrier provision, uplink for backscattered data." Because nodes and base stations do not move, both directions are computed *once, before the simulation starts*, and then simply looked up.

2.2 The life of a node

Figure 2.2 shows the capacitor voltage of three nodes during a 20-second run of the shipped broadcast demo. This is *real output* of the simulator; reading it carefully teaches most of what follows.

1. **Charging.** All capacitors start empty. Each rises along an RC curve towards the equivalent source voltage determined by the harvested power. The closer node reaches $V_{\text{high}} = 1.7\,\text{V}$ sooner (about 1.8 s for Node 0, about 3.4 s for Node 1).

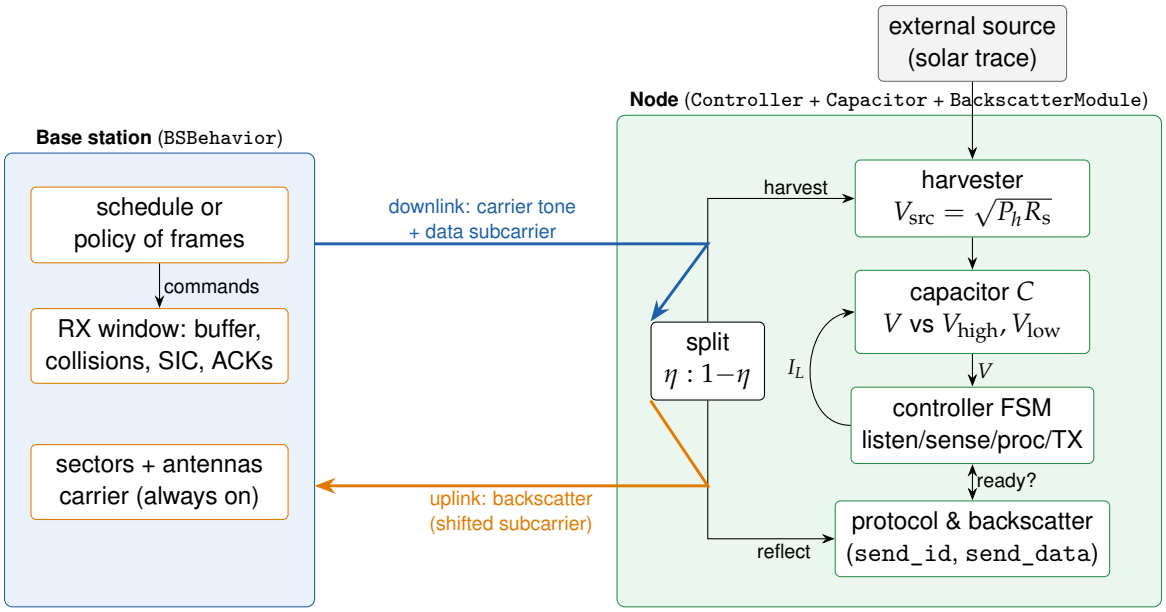


Figure 2.1: Energy and information flow in Amber. The base station radiates a carrier; a node splits the received power between harvesting and reflection. Energy flows downward through the harvester into the capacitor; the controller turns capacitor voltage into duty-cycled activity and permits the protocol module to answer.

2. **Activation.** Once $V \geq V_{\text{high}}$ (and a random start-up delay of up to two seconds has elapsed), the node becomes *active* and starts its duty cycle.
3. **Duty-cycling.** Each time the node backscatters a packet it pays a large current for a few milliseconds—the sharp downward steps. Between packets it charges again—the slanted ramps.
4. **Shut-down.** When the voltage falls to $V_{\text{low}} = 1.3\,\text{V}$ the node switches off and charges back up. The *saw-tooth* is the signature of threshold-driven intermittent operation.

The three traces differ in a way the radio alone would not predict. Node 2 is the farthest yet charges the *fastest*, because the simulator lets it draw on an external (solar) source; it also sends the most packets. In the demo's 20-second run Node 0, Node 1 and Node 2 sent about 46, 18 and 58 packets respectively (counts vary slightly between runs). **Communication range is not the same as energy range.**

2.3 The four design principles

The paper lists four principles. We restate each and say where it is realised.

Physically grounded modelling. Energy dynamics are *voltage- and current-based capacitor models* accounting for series resistance, leakage and simultaneous charging/discharging; RF propagation uses *free-space path loss or the 3GPP TR 38.901 Urban Macro (UMa) model* with sectored antenna patterns; link budgets use the *Friis transmission equation* with configurable gains, carrier frequency and medium attenuation. (Chapters 5, 8.)

Modularity. Capacitor, controller, backscatter model and base station are independent classes with defined interfaces, so each can be swapped without touching the framework. (Chapter 13.)

Energy awareness. "All simulated events are conditioned on the node's instantaneous energy state." The simulator tracks capacitor voltage at millisecond resolution, and models current

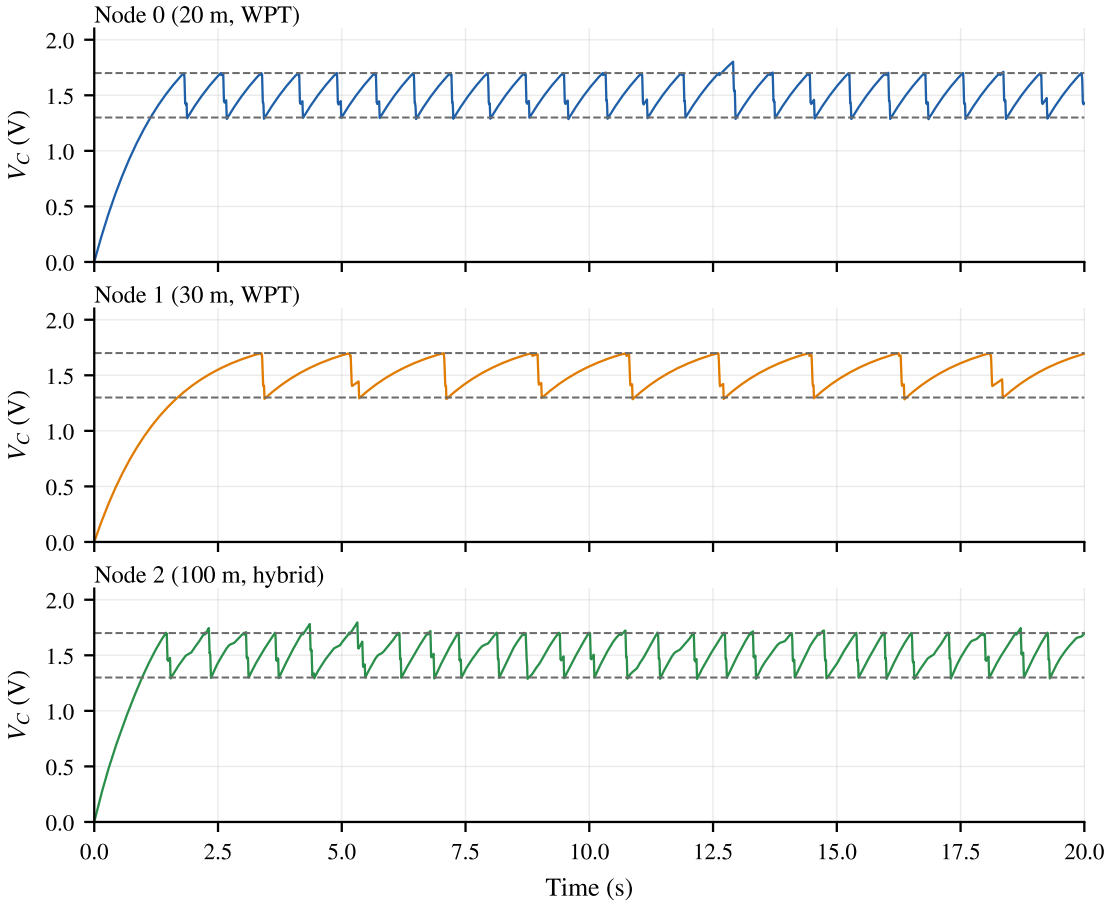


Figure 2.2: Capacitor voltage of the three nodes of the broadcast demo (measured; random seed fixed for the figure). Node 0 and Node 1 harvest only from the base station's carrier at 20 m and 30 m. Node 2 is 100 m away, too far to power itself from RF alone, and runs on its *solar* trace instead. Dashed lines: $V_{\text{high}} = 1.7\,\text{V}$ and $V_{\text{low}} = 1.3\,\text{V}$.

consumption separately for the states *listening*, *sensing*, *processing*, *transmitting*. (Chapter 15.)

Scalability. Built on SimPy with a 1 ms time step and interrupt-driven communication between node and base-station processes. Because nodes and base stations are stationary, coverage maps and link budgets are *pre-computed once*, enabling "hundreds of nodes". (Chapter 12.)

2.4 Time scales

A system that couples electronics with protocols spans several orders of magnitude in time. Keeping them straight prevents many modelling errors.

Process	Time scale	Where it comes from
Simulation tick	1 ms	`dt`, controller loop
Command / reply slot	5 ms–15 ms	BS schedule (demos)
Node transmit state	15 ms	`DurationsMs.transmitting`
Capacitor charge time constant	$\tau_c = R_s C = 1.5\,\text{s}$	demo values
Time to first activation	∼1.5 s–3.5 s	Chapter 8
Capacitor leakage constant	$\tau_\ell = R_l C = 30\,\text{s}$	demo values
Solar trace length	3 s, looped	`EnvEnergySource`
Typical simulated horizon	15 s–20 s	demos

2.5 One poll, end to end

To prepare for Part V, here is the complete story of one *unicast poll* in the shipped `unicast` demo, told in ten steps. Do not worry about unfamiliar names yet: each is explained in its own chapter.

1. At the start of its *TX slot* the base station calls `_deliver_command("send_data",target=0)`. It also looks ahead in its schedule to find the next RX slot(s) and attaches them to the command.
2. The target node's `BackscatterModule` checks that the *downlink power* reaching the node exceeds its sensitivity. If not, the command is ignored. Otherwise it records the command and remembers the advertised slot.
3. Meanwhile the node's `Controller`, which wakes up every millisecond, has been walking through its state machine. If the capacitor is above V_{low} the node is *active*.
4. If the node is active and in its *wait_slot* state, the controller asks `controller_tx_ready`: "is a command pending, is t inside the advertised slot, and is the slot not stale?"
5. If all checks pass, the module calls `do_transmit`: it finds the best receiving sector of every base station, writes the payload and the pre-computed uplink RSSI into the base station object, and *interrupts* the base station process.
6. The base station is inside an RX window; the interrupt makes it append the packet to a *buffer*.
7. At the end of the window the base station groups buffered packets by subcarrier and time, applies the SINR/SIC rules, drops anything below its sector sensitivity, and logs the survivors as `RxPackets`.
8. The node meanwhile enters its *transmitting* state: for 15 ms the controller commands a 5 mA load, and the capacitor voltage steps down by about 0.25 V.
9. The controller returns to *listening*, then *sensing* and *processing*, and again waits for a slot. If the voltage has fallen to V_{low}, the node powers down instead and the capacitor recharges.
10. The simulation continues, recording counters from which DER and throughput are computed (Chapter 18).

Notice that **energy gates *transmission*** (steps 3–4: a node that is off or in the wrong state cannot answer), while the **radio gates *reception*** (step 2: a weak command is ignored). This asymmetry will matter in Chapter 21.

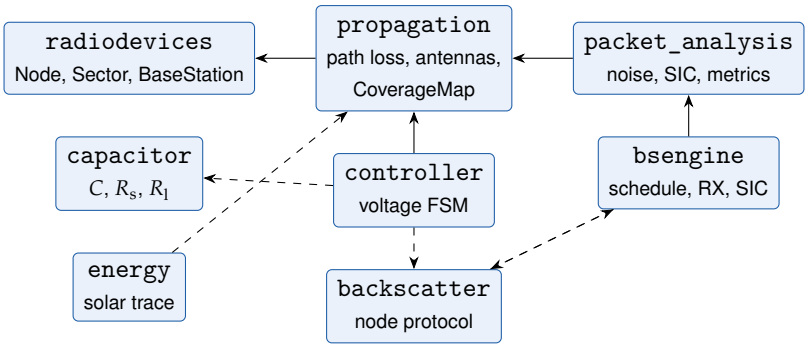


Figure 2.3: Module map. Solid arrows: Python imports. Dashed arrows: run-time interaction through SimPy interrupts and shared attributes (Chapter 13).

2.6 What is simulated and what is abstracted

Layer	Modelled explicitly	Abstracted away
Propagation	Deterministic path loss (FSPL or 3GPP UMa), antenna patterns, two-way backscatter budget	Fading, shadowing, multipath, mobility
Physical layer	RSSI thresholds, SINR for collisions, thermal noise	Waveforms, modulation, coding, bit errors, synchronisation
Energy	Capacitor ODE (charge, leak, load), thresholds, per-state currents	Rectifier non-linearity (RF–DC efficiency), converter losses
MAC / protocol	Commands, slots, registration, polling, collisions, SIC	Preambles, CRCs, retransmission timers (user-defined)
Time	Millisecond clock, per-state durations	Anything below 1 ms

2.7 Map of the book to the code

Figure 2.3 shows the eight modules of the `amber` package and how they depend on each other. The table below tells you where each theme of this book is implemented and discussed.

Module	Theory it implements	Chapters
`radiodevices`	node, sector and base-station parameters	14
`propagation`	dB arithmetic, antenna patterns, Friis/UMa path loss, two-way backscatter budget, harvesting modes	3–6, 7, 14
`capacitor`	RC charging, leakage, constant-current discharge	8
`energy`	external (solar) energy trace	7
`controller`	threshold hysteresis and the duty-cycle state machine	8, 15
`backscatter`	node-side protocol: registration, polling, slot validation	11, 16
`bsengine`	schedules, RX windows, collisions, SIC	9–10, 17
`packet_analysis`	thermal noise, SINR, SIC, DER and throughput	10, 18

2.8 Chapter summary

- Amber simulates nodes, base stations and the links between them; links are pre-computed because everything is stationary.

- A node's life is an RC charge, a threshold-triggered activation, a duty cycle of listen–sense–process–wait–transmit, and a shut-down at V_{low}.
- Energy gates a node's *transmissions*; radio power gates its *receptions*.
- The four principles (physical grounding, modularity, energy awareness, scalability) map onto Chapters 5–8, 13, 15 and 12.

Exercises

Exercise 1

Name the three entity types, and for each list two parameters that describe it (according to the paper).

Exercise 2

In Figure 2.2, Node 2 is farthest from the base station but activates first and sends the most packets. Give two possible explanations that do *not* involve the quality of its radio link, and say which of the two the demo configuration actually implements.

Exercise 3

In the walk-through of Section 2.5, suppose that when the command arrives the node is in its *processing* state and the advertised RX slot lasts 10 ms. Under what condition can the node still reply in this slot?

Part II

The Physics of Radio and Energy

3

Decibels, Noise and the Link Budget

In this chapter you will learn

- why radio engineers work in decibels, and how to convert fluently between dB, dBm, dBi and linear units;
- how to write down a *link budget* as a column of additions and subtractions;
- where thermal noise comes from, how to compute the noise floor of a receiver, and the difference between SNR, SINR and *sensitivity*;
- why the signal a backscatter receiver wants to detect is almost 100 dB weaker than the carrier that produced it.

Every number you will meet in Amber's propagation code is a power, a gain or a loss. Before we study how they arise (Chapters 4–6), we need a common language for talking about them.

3.1 Why a logarithmic scale?

The powers in a link-budget differ by absurd factors. A base station sector may radiate 46 dBm $\approx$ 40 W; a backscattered reply may arrive at the same base station with -69 dBm ≈ 0.14 nW. The ratio is about 3×10^{11}. Worse, every stage between transmitter and receiver *multiplies* the power by a gain or loss factor: antenna gain, path loss, power split, receiver gain. Working with products of numbers like 10^{-7} invites mistakes. Logarithms turn products into sums and make huge ratios fit into small numbers.

Decibel

For two powers P_1 and P_2 the *ratio in decibels* is

$$\left(\frac{P_2}{P_1}\right)_{\text{dB}} = 10 \log_{10} \frac{P_2}{P_1}.$$

For an amplitude (voltage, field) ratio, $20 \log_{10}$ is used instead, because power is proportional to amplitude squared. A decibel value is always a *ratio*; an absolute power needs a reference.

The references we need:

- **dBm**: reference 1 mW. $P_{\text{dBm}} = 10 \log_{10}(P/1\,\text{mW})$.
- **dBW**: reference 1 W; $P_{\text{dBm}} = P_{\text{dBW}} + 30$.
- **dBi**: antenna gain relative to an *isotropic* radiator (Chapter 4).

3.1.1 The conversion Amber uses

Amber's three conversion helpers in `propagation.py` are one-liners:

$$P_{\mathrm{W}} = 10^{(P_{\mathrm{dBm}}-30)/10}, \qquad \texttt{dbm_to_watts} \tag{3.1}$$

$$P_{\mathrm{dBm}} = 10\log_{10}\big(\max(P_{\mathrm{W}}, 10^{-15})\big) + 30, \qquad \texttt{watts_to_dbm} \tag{3.2}$$

$$G = 10^{G_{\mathrm{dBi}}/10}. \qquad \texttt{lin_from_dbi} \tag{3.3}$$

The "-30" converts watts to milliwatts: $1\,\mathrm{W} = 1\,000\,\mathrm{mW} = 30\,\mathrm{dB}$ above one milliwatt. Notice the floor $10^{-15}\,\mathrm{W} = -120\,\mathrm{dBm}$ in `watts_to_dbm`: it prevents $\log_{10} 0$, but it also means that Amber cannot represent any power below $-120\,\mathrm{dBm}$ (a *power of zero* is reported as $-120\,\mathrm{dBm}$).

dBm	Watts	Where it appears	dBm	Watts	Where it appears
46	39.8 W	demo sector power	−30	1 µW	weak energy harvest
43	20 W	`Sector.power` default	−45.7	27 nW	Node 0 uplink RSSI (demo)
30	1 W	paper's BS power	−60	1 nW	
0	1 mW	Node 0 downlink (demo)	−88	1.6 pW	thermal noise, 100 MHz
−10	100 µW	Node 2 downlink (demo)	−100	0.1 pW	sector sensitivity
−20	10 µW		−120	1 fW	Amber's representation floor

3.1.2 Rules of thumb you should memorise

+3 dB	$\approx \times 2$ in power	+10 dB	$= \times 10$
−3 dB	$\approx \times \frac{1}{2}$	+20 dB	$= \times 100$
+6 dB	$\approx \times 4$	+30 dB	$= \times 1000$
0 dBm	$= 1\,\mathrm{mW}$	30 dBm	$= 1\,\mathrm{W}$

Never add dBm values

Gains and losses *add* in decibels, but *powers* add only in the linear domain. Two interferers at $-70\,\mathrm{dBm}$ and $-73\,\mathrm{dBm}$ do not produce $-143\,\mathrm{dBm}$ (that would be a *product*, not a sum); they produce

$$10\log_{10}\big(10^{-7} + 10^{-7.3}\big)\ \mathrm{dBm} \;=\; -68.2\,\mathrm{dBm}.$$

Amber follows this rule: `compute_point_to_bs` converts each node's power to watts, sums, and converts back to obtain `total_pw_dbm`, and `apply_sic` converts every power to watts before summing interference.

3.2 The link budget

A *link budget* is the accounting that carries power from a transmitter to a receiver. In decibels it is a simple column of additions:

$$P_{\mathrm{rx}}\,[\mathrm{dBm}] \;=\; P_{\mathrm{tx}}\,[\mathrm{dBm}] \;+\; G_{\mathrm{tx}}\,[\mathrm{dBi}] \;-\; \mathrm{PL}\,[\mathrm{dB}] \;+\; G_{\mathrm{rx}}\,[\mathrm{dBi}] \;-\; L_{\mathrm{other}}\,[\mathrm{dB}]. \tag{3.4}$$

In Amber L_{other} is the optional *medium attenuation* term (Chapter 5); everything else maps one-to-one onto the arguments of `compute_rx_power_watts`. Link-budget thinking is the single most useful habit for understanding any Amber output.

The demo's downlink and uplink, line by line

Consider **Node 0** of the shipped demos: it stands 20 m due north of the base station, whose north-facing sector has $P_{tx} = 46$ dBm, $G_{max} = 15$ dBi; the node antenna is 0 dBi; carrier 924 MHz; 3GPP UMa LOS path loss gives PL = 60.08 dB (Chapter 5); the node reflects a fraction $\eta = 0.7$ (-1.55 dB).

Downlink (carrier to node)		
Sector transmit power	+46.00	dBm
Sector gain toward node (on boresight)	+15.00	dBi
Path loss	−60.08	dB
Node antenna gain	+0.00	dBi
Power arriving at node P_{dl}	**+0.92**	**dBm** (1.24 mW)
Backscatter uplink		
Reflected fraction $10 \log_{10} 0.7$	−1.55	dB
Node antenna gain (re-radiating)	+0.00	dBi
Path loss (same distance, same model)	−60.08	dB
Sector gain toward node (receiving)	+15.00	dBi
Power arriving at base station P_{ul}	−45.71	**dBm**

The value −45.71 is exactly what the simulator logs in `broadcast/output/rx_packets.csv` (column `rssi_dbm`) for Node 0. Repeating the arithmetic for Node 1 (30 m, PL = 62.10 dB) gives −1.10 and −49.74 dBm, and for Node 2 (100 m, PL = 71.57 dB) gives −10.57 and −68.69 dBm, again matching the simulator. Notice that the uplink budget contains the path loss *twice*: this is the signature of backscatter.

3.3 Thermal noise

Every conductor above absolute zero carries random thermal motion of charge carriers; a receiver sees it as noise. The available thermal-noise power in a bandwidth B is

$$N_{th} = k_B T B, \qquad k_B = 1.38 \times 10^{-23}\,\mathrm{J/K}. \tag{3.5}$$

At the conventional reference temperature $T_0 = 290$ K, $k_B T_0 = 4.0 \times 10^{-21}$ W/Hz, i.e.

$$10 \log_{10} \frac{4.0 \times 10^{-21}\,\mathrm{W/Hz}}{10^{-3}\,\mathrm{W}} = -174\,\mathrm{dBm/Hz}.$$

A real receiver adds noise of its own, summarised by the *noise figure* F (a ratio ≥ 1; NF = $10 \log_{10} F$): the receiver output has F times more noise than an ideal noiseless receiver would have. Hence

$$N = k_B T_0 B F, \qquad N_{dBm} = -174 + 10 \log_{10} B_{Hz} + \mathrm{NF}. \tag{3.6}$$

This is exactly Amber's `thermal_noise_watts(bandwidth_hz,noise_figure_db)`, with $T_0 = 290$ K hard-coded and the paper's defaults $B = 100$ MHz, NF = 6 dB:

$$N_{dBm} = -174 + 80 + 6 = -88\,\mathrm{dBm} \quad (1.59 \times 10^{-12}\,\mathrm{W}).$$

3.4 SNR, SINR and sensitivity

Whether a receiver can decode a packet depends on how large the wanted signal P_s is compared with everything else.

SNR, SINR

$$\text{SNR} = \frac{P_s}{N}, \qquad \text{SINR } \gamma = \frac{P_s}{\sum_{j \neq s} P_j + N},$$

where the sum runs over *interfering* signals (other nodes' backscatter in Amber). Decoding is assumed to succeed when $\gamma \geq \gamma_{\text{req}}$, a threshold determined by modulation and coding.

A related but distinct concept is *sensitivity*: the weakest received power at which a receiver can still operate. In a classical receiver $S_{\min} = N + \gamma_{\text{req}}$ (plus implementation losses), so sensitivity and noise floor are tied together by the bandwidth. **Amber stores sensitivity as an independent threshold**:

- `Node.sensitivity_dbm` (default -45 dBm): the weakest downlink command a node notices;
- `Sector.sensitivity_dbm` (default -100 dBm): the weakest backscatter packet a base-station sector can log.

Sensitivity and noise are not tied together in Amber

With the default $B = 100$ MHz, the noise floor is -88 dBm, *above* the default sector sensitivity of -100 dBm. A classical receiver could never decode a packet at -100 dBm in 100 MHz of noise (its SNR would be -12 dB). For the two numbers to be consistent with $\gamma_{\text{req}} = 3$ dB the noise bandwidth would have to be about 3.2 MHz (Exercise 3.6). In the simulator the two thresholds act at different places:

- the *sensitivity* gate drops any packet whose RSSI is below the sector's value;
- the *noise* enters only the SINR computation for *colliding* packets (Chapter 10). A *lone* packet that passes the sensitivity gate is decoded no matter what its SNR is.

We verified the last point by injecting a single packet at -105 dBm (SNR $= -17$ dB) into a base station with -110 dBm sensitivity; it was logged as decoded. When you use Amber, set `bandwidth_hz` and the sensitivities consistently with each other.

3.5 The dynamic-range problem of backscatter

Look again at the Node 0 budget: the base station *transmits* $+46$ dBm per sector and *wants to receive* -45.7 dBm; for the farthest demo node the wanted signal is -68.7 dBm. Between the transmitted carrier and the wanted backscatter lie 92–115 dB. A circulator or two separate antennas typically gives only of order 20–30 dB of isolation between transmit and receive paths, so even the paper's 30 dBm carrier leaks into the receiver at roughly 0 to $+10$ dBm—about 50 dB *stronger* than the strongest backscatter reply in our demos. This *self-interference* is the central practical difficulty of backscatter readers, and it is the reason the PRDCH structure and the *subcarrier shifting* of Chapter 6 exist: they move the reply into a frequency region where the carrier is not.

Where this lives in the code

Concept	Code (`amber/`)
dBm ↔ W, dBi → linear	`propagation.dbm_to_watts`, `watts_to_dbm`, `lin_from_dbi`
Link budget, Eq. (3.4)	`propagation.compute_rx_power_watts`
Noise power, Eq. (3.6)	`packet_analysis.thermal_noise_watts`
SINR	`packet_analysis.apply_sic`
Sensitivity gates	`backscatter.receive_command` (node), `bsengine._process_rx_buffer` (sector)

3.6 Chapter summary

- Decibels turn the multiplicative chain of gains and losses into a sum; powers in dBm must be converted to linear before being added.
- A link budget $P_{rx} = P_{tx} + G_{tx} - \mathrm{PL} + G_{rx}$ explains every RSSI in Amber's output; we reproduced the demo's -45.71 and -68.69 dBm by hand.
- Noise is $N = k_B T_0 B F$, giving -88 dBm for 100 MHz and NF $= 6$ dB.
- Sensitivity is a configured threshold in Amber, *decoupled* from the noise floor; noise matters only in collisions.
- Backscatter receivers face ~ 100 dB between carrier and wanted signal.

Exercises

Exercise 1

Convert: (a) 33 dBm to watts; (b) 2 µW to dBm; (c) -97 dBm to watts.

Exercise 2

Two co-channel interferers arrive at -70 and -73 dBm. What is their combined power in dBm? What would it be if there were four equal interferers at -73 dBm?

Exercise 3

Compute the thermal noise floor in dBm for $B = 1$ MHz and NF $= 8$ dB, and for $B = 100$ MHz and NF $= 6$ dB. By how many dB do they differ?

Exercise 4

A sector transmits 30 dBm with 17 dBi toward a node with a 1 dBi antenna, over a path loss of 80 dB. (a) What is the downlink power at the node in dBm and in µW? (b) Does it exceed Amber's default node sensitivity of -45 dBm? (c) If the node reflects $\eta = 0.5$ and the return path has the same loss and the same sector gain toward the node, what uplink power arrives at the base station?

Exercise 5

Show that a receiver with noise figure 6 dB has a -100 dBm sensitivity at $\gamma_{req} = 3$ dB only if its noise bandwidth is about 3.2 MHz.

4

Antennas, Sectors and Geometry

In this chapter you will learn

- what antenna *gain* really means and why it is measured in dBi;
- the 3GPP TR 38.901 horizontal sector pattern, including its half-power and floor behaviour;
- the geometric bookkeeping Amber performs: compass versus mathematical angles, bearings and angular error;
- how a three-sector base station divides the plane and how a node's "best" sector is chosen.

4.1 Gain: concentrating a fixed amount of power

An *isotropic* radiator spreads power equally in all directions; its power density at distance d is $P_t/(4\pi d^2)$. No physical antenna is isotropic, and the laws of energy conservation say that an antenna cannot create power, only *redistribute* it. A *directional* antenna concentrates power into some directions at the expense of others. Its *gain* $G(\theta, \varphi)$ in a direction is the ratio of the power density it produces there to that of an isotropic radiator fed with the same power. We express it in dBi.

Gain as a flashlight

Imagine pouring a fixed amount of light into a room. A bare bulb lights everything dimly; a torch lights a narrow cone brightly and the rest not at all. The brightness in the cone divided by the bare-bulb brightness is the gain. The *narrower* the beam, the *higher* the gain—a fact we can quantify.

For a beam whose half-power widths are θ_{az} and θ_{el} (in degrees) a standard approximation of the peak *directivity* is

$$D \approx \frac{41\,253}{\theta_{az}\,\theta_{el}}, \tag{4.1}$$

because 41 253 deg^2 is the solid angle of the whole sphere in square degrees. A cellular panel with a 65° azimuth beamwidth and a vertical beamwidth of roughly 10° has $D \approx 63 \approx 18$ dBi; losses reduce this to the 15 dBi–17 dBi used in Amber's demos and in the paper (17 dBi). A node's small antenna is nearly omnidirectional: 0 dBi by default, 1 dBi in the paper's experiment.

4.2 The 3GPP sector pattern

Amber models a base-station sector with the *horizontal* pattern of 3GPP TR 38.901 [4]. With φ the angle between the target and the antenna's boresight, $\varphi_{3\mathrm{dB}}$ the half-power beamwidth (`beamwid`

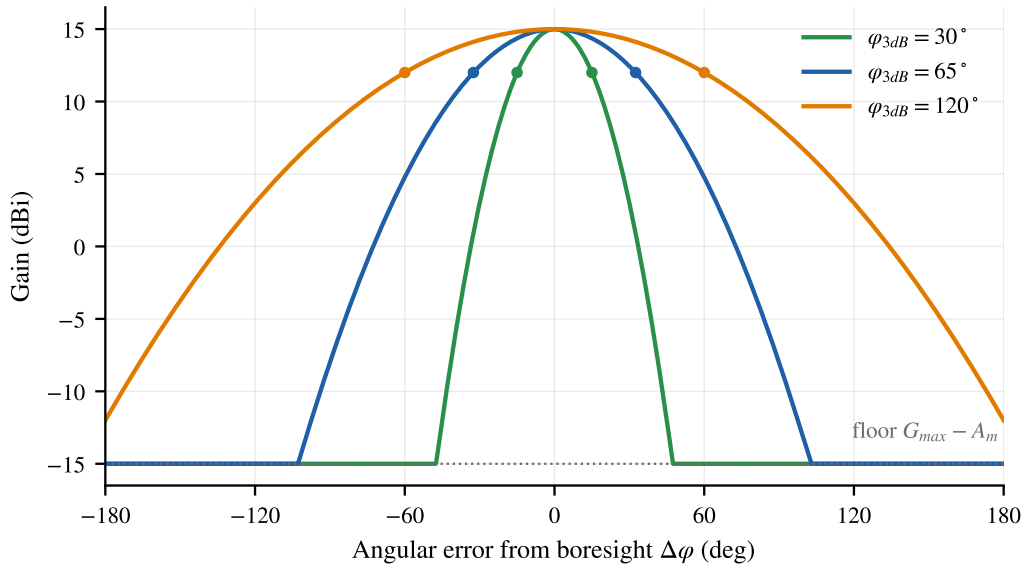


Figure 4.1: The pattern of Eq. (4.2) for $G_{\max} = 15$ dBi and three beamwidths. Dots mark the half-power points at $\pm\varphi_{3\mathrm{dB}}/2$ ($G_{\max} - 3$ dB). Beyond about $1.58\,\varphi_{3\mathrm{dB}}$ the pattern is flat at $G_{\max} - A_m$.

`th_deg`) and A_m the maximum attenuation (front-to-back ratio), the attenuation below the peak is

$$A_H(\varphi) \;=\; \min\left[\,12\left(\frac{\varphi}{\varphi_{3\mathrm{dB}}}\right)^2,\; A_m\right]\ \mathrm{dB}, \qquad G(\varphi) = G_{\max} - A_H(\varphi), \tag{4.2}$$

with $A_m = 30$ dB as hard-wired in `antenna_from_sector`.

Three properties follow immediately and are worth internalising.

1. **The half-power points.** At $\varphi = \varphi_{3\mathrm{dB}}/2$ the attenuation is $12 \cdot \frac{1}{4} = 3$ dB. So $\varphi_{3\mathrm{dB}}$ is the *full* width of the beam at half power—consistent with the name.
2. **The floor.** The parabola reaches A_m at $\varphi = \varphi_{3\mathrm{dB}}\sqrt{A_m/12} = 1.58\,\varphi_{3\mathrm{dB}}$; for 65° that is 102.8°. Beyond that the sector's gain is $G_{\max} - 30$, i.e. −15 dBi for a 15 dBi panel. The antenna never has a perfect null.
3. **Narrow beams roll off fast.** A 30° sector drops 12 dB at ±30°, whereas a 65° sector drops only about 2.6 dB there.

We can check item 2 against the simulator. In the demo, **Node 0** lies due north of the base station; the sectors at azimuths $0°, 120°, 240°$ see it at angular errors 0°, 120° and 120°. Sector 1 and 2 are therefore at the floor, 30 dB below sector 0, and indeed the simulator reports downlink powers of $+0.9$, -29.1 and -29.1 dBm for sectors `BS0_S0`, `BS0_S1`, `BS0_S2`.

Only the azimuth cut is modelled

The full 3GPP element pattern adds a *vertical* term $A_V(\theta)$ and combines the two. Amber implements the horizontal cut only (`SectorAntenna3GPP.gain_linear`). The base station's height enters path loss through the 3-D distance, but *there is no downtilt and no vertical attenuation*. For a node close to the mast (a 20 m ground distance under a 25 m mast means the node is $\approx 50°$ below the horizon) a real panel would give less than the modelled gain.

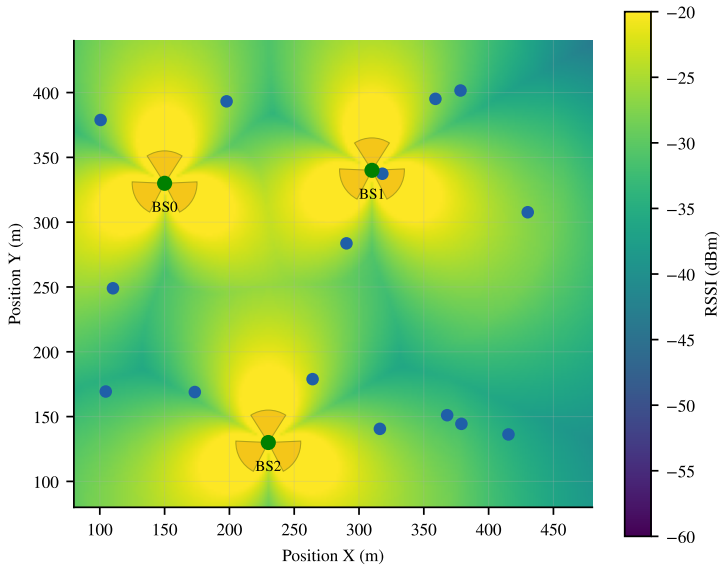


Figure 4.2: A coverage map produced by Amber's `CoverageMap` for an illustrative deployment (three base stations of three 65° sectors, 30 dBm, 17 dBi, 924 MHz, 15 random nodes), in the style of the paper's Figure 1. Colour is the *best* received power among all sectors (Chapter 14). This figure is our own example, not the paper's data.

4.3 Sectorisation

A cell site rarely uses a single omnidirectional antenna. The usual arrangement is *three sectors of* 120° *spacing*, each with a directional panel. This multiplies capacity (each sector reuses the spectrum) and, for a given transmit power, lifts the gain in each served direction.

Because the sector pattern falls off by about 10 dB at the crossover (60° off boresight when sectors are spaced 120° apart), a 65° panel leaves a *ripple* in the coverage:

$$G(60^\circ) = 15 - 12\left(\tfrac{60}{65}\right)^2 = 4.8\,\text{dBi},$$

more than 10 dB below the peak. This ripple is visible as the darker bands between sectors in coverage maps such as Figure 4.2.

The best sector. Amber assigns each point (grid point or node) to the sector delivering the highest power: the "owner" of that point (`owner`, `best_pw_dbm`). There is no hysteresis or handover logic, and ties are resolved in favour of the sector evaluated first (the comparison is a strict ">").

4.4 Geometry: bearings and angular errors

To evaluate Eq. (4.2) we need φ, the angle between a sector's boresight and the direction to a node. Two angle conventions meet here, and confusing them is a classic source of bugs.

- **Compass** (navigation) azimuth: 0° is North, increasing *clockwise*. This is what `Sector.azimuth_deg` and `Node.azimuth_deg` use.
- **Mathematical** angle: 0° is East, increasing *counter-clockwise*. This is what `numpy.arctan2` returns.

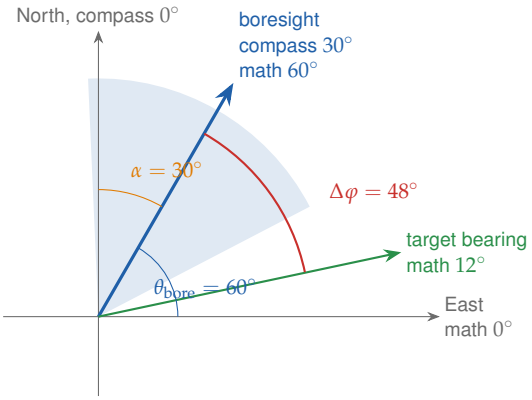


Figure 4.3: Compass versus mathematical angles. The sector's boresight points at compass azimuth 30° (math 60°); a target at math bearing 12° is $\Delta\varphi = 48°$ off boresight and sees the pattern attenuation $12(48/65)^2 = 6.5\,\text{dB}$ for a 65° sector.

The conversion is $\theta_{\text{math}} = (90° - \alpha_{\text{compass}}) \bmod 360°$ (`compass_to_math`). The bearing from a point (x_0, y_0) to (x, y) is $\text{atan2}(y - y_0,\, x - x_0) \bmod 360°$ (`bearing_deg`), and the smallest angular difference between two angles, correct across the 0°/360° seam, is

$$\Delta\varphi = \left|\, (\theta_{\text{target}} - \theta_{\text{bore}} + 180°) \bmod 360° - 180° \,\right| \tag{4.3}$$

(`angular_error_deg`). The shift by 180° followed by the modulus maps the difference into $[-180°, 180°)$, whose absolute value is the shortest way round.

Why a node due north sees the 120° sector at the floor

The sector with compass azimuth 120° has $\theta_{\text{bore}} = (90 - 120) \bmod 360 = 330°$. A node due north has math bearing 90°. By Eq. (4.3): $|(90 - 330 + 180) \bmod 360 - 180| = |300 - 180| = 120°$, and $12(120/65)^2 = 40.9 > A_m$, so the attenuation clips at 30 dB.

4.5 Node antennas and reciprocity

A node is described by `antenna_type`: "omni" (default; gain `antenna_gain_dbi`, independent of direction) or "3GPP" (the same pattern as a sector, with the node's own `azimuth_deg` and `beamwidth_deg`). In both the downlink and the uplink the simulator evaluates the node's gain toward the base station *with the same function*. This reflects the *reciprocity* of passive antennas: the pattern is the same in transmit and receive. For backscatter, where one antenna both receives the carrier and re-radiates the reflection, reciprocity is physically exact.

Where this lives in the code

Concept	Code (`amber/propagation.py`)
Pattern Eq. (4.2)	`SectorAntenna3GPP.gain_linear`
Omni antenna	`OmniAntenna.gain_linear`
Build antenna from device	`antenna_from_sector`, `antenna_from_node`
Compass → math	`compass_to_math`
Bearing, error Eq. (4.3)	`bearing_deg`, `angular_error_deg`
Best sector	the `mask=p_w>best_pw_w` update in `compute_bs_to_point`

4.6 Chapter summary

- Gain redistributes power; narrower beams have higher gain, $D \approx 41\,253/(\theta_{az}\theta_{el})$.
- Amber uses 3GPP's horizontal pattern: $G = G_{\max} - \min[12(\varphi/\varphi_{3\text{dB}})^2, 30]$.
- Three 65° sectors at 120° spacing leave a ≈ 10 dB ripple at the crossovers.
- Compass and mathematical angles differ; Eq. (4.3) gives the correct shortest angular error.
- No vertical pattern or downtilt is modelled.

Exercises

Exercise 1

A sector has $G_{\max} = 17$ dBi and $\varphi_{3\text{dB}} = 65°$. What is its gain at an angular error of 40°? At 90°? At 150°?

Exercise 2

Find the angular error at which the pattern reaches its floor for (a) $\varphi_{3\text{dB}} = 30°$ and (b) $\varphi_{3\text{dB}} = 120°$.

Exercise 3

Convert the compass azimuths 0°, 45°, 135°, 270° to mathematical angles. A sector has compass azimuth 10°; a node is at math bearing 350°. What is the angular error?

Exercise 4

Three sectors with $\varphi_{3\text{dB}} = 65°$ at compass azimuths $0°, 120°, 240°$. For a node at compass bearing 60° from the mast, compute each sector's gain (take $G_{\max} = 15$ dBi) and say which sector Amber would call the owner.

Exercise 5

Estimate the peak gain of a panel with a $90° \times 15°$ beam using Eq. (4.1).

5

Path Loss: From Friis to 3GPP Urban Macro

In this chapter you will learn

- how the Friis transmission equation follows from two simple ideas—power density and effective aperture;
- why real environments need path-loss models beyond free space, and what the *break-point distance* means;
- the 3GPP TR 38.901 Urban Macro (UMa) line-of-sight and non-line-of-sight formulas that Amber implements, with all constants explained;
- how Amber applies medium attenuation, and two subtle behaviours of the code near the base station.

5.1 Free-space propagation: Friis

Two ideas, both geometric, give the free-space law.

Idea 1: power density. An isotropic transmitter of power P_t spreads its power over a sphere of area $4\pi d^2$ at distance d. With antenna gain G_t toward the receiver the power density is

$$S = \frac{P_t G_t}{4\pi d^2} \qquad [\mathrm{W/m^2}].$$

Idea 2: effective aperture. A receiving antenna of gain G_r intercepts power from a passing wave as if it had an *effective area*

$$A_e = \frac{G_r \lambda^2}{4\pi}, \qquad \lambda = \frac{c}{f_c}.$$

This shows an interesting fact: at a given gain, a lower frequency means a larger effective area, so a 924 MHz antenna ($\lambda = 0.325$ m) captures ≈10 times more power than a 3 GHz one of the same gain. Multiply the two:

$$P_r = S\,A_e = P_t\,G_t\,G_r \left(\frac{\lambda}{4\pi d}\right)^2. \tag{5.1}$$

This is the *Friis transmission equation* [10]. The last factor,

$$L_{\mathrm{FS}}(d) = \left(\frac{\lambda}{4\pi d}\right)^2, \qquad \mathrm{PL}_{\mathrm{FS}}(d) = 20\log_{10}\frac{4\pi d}{\lambda}\ \mathrm{dB}, \tag{5.2}$$

is the *free-space path loss*. It is exactly Amber's `fspl_lin`. In convenient units, $\mathrm{PL_{FS}} = 32.45 + 20\log_{10} f_{\mathrm{MHz}} + 20\log_{10} d_{\mathrm{km}}$. At 924 MHz the loss at 1 m is 31.76 dB, and every doubling of distance costs $20\log_{10} 2 = 6.02$ dB more.

A common misconception

Free-space "loss" is not absorption. No energy disappears; it simply *spreads out*. The λ^2 in the formula comes from the receive aperture, not from the medium: path loss at a fixed distance grows with frequency because the receiving antenna gets *smaller*.

Free space at 20 m

$\lambda = 3 \times 10^8 / 924 \times 10^6 = 0.324\,7$ m. $\mathrm{PL_{FS}}(20) = 20\log_{10}(4\pi \cdot 20/0.3247) = 20\log_{10}(774) = 57.78$ dB.

5.2 Beyond free space

Free space is the best case. In a real street the wave also reflects from the ground and buildings, is blocked by obstacles, and diffracts around edges. Empirically one finds that received power falls as d^{-n} with a *path-loss exponent* n ranging from about 2 (open) to 4–6 (cluttered). The simplest physical mechanism that makes $n = 4$ is the *two-ray ground reflection*: the direct ray and its reflection from the ground interfere destructively beyond a *breakpoint distance* $d_{BP} \approx 4h_t h_r/\lambda$ (for antenna heights h_t, h_r); inside it the loss is nearly free-space ($n \approx 2$) and outside it is steeper ($n \approx 4$). This "two-slope" behaviour is built into the 3GPP urban macro model.

5.3 3GPP TR 38.901 Urban Macro (UMa)

3GPP's channel-model report TR 38.901 [4] specifies a statistical path-loss formula for every deployment scenario for carrier frequencies between 0.5 and 100 GHz. Amber implements the **UMa** scenario in `urban_macro_loss_db_38_901`. Let f_c be in GHz, h_{BS} the base-station height (default 25 m) and h_{UT} the node height (default 1.5 m). The code defines

$$d_{3D} = \sqrt{d_{2D}^2 + (h_{BS} - h_{UT})^2} \quad \text{(clamped to } \geq 10\,\mathrm{m}), \tag{5.3}$$

$$d'_{BP} = \frac{4\,(h_{BS} - 1)\,(h_{UT} - 1)\,f_c}{c} \quad [f_c \text{ in Hz in the code}], \tag{5.4}$$

where the "-1" is the effective environment height $h_E = 1$ m. For the demo's heights $d'_{BP} = 4 \cdot 24 \cdot 0.5 \cdot 924 \times 10^6 / 3 \times 10^8 = 147.8$ m.

Line of sight (LOS). The loss has two slopes:

$$\mathrm{PL_1} = 28.0 + 22\log_{10} d_{3D} + 20\log_{10} f_c, \qquad 10\,\mathrm{m} \leq d_{2D} \leq d'_{BP}, \tag{5.5}$$

$$\mathrm{PL_2} = 28.0 + 40\log_{10} d_{3D} + 20\log_{10} f_c - 9\log_{10}\!\left(d'^2_{BP} + (h_{BS} - h_{UT})^2\right), \quad d_{2D} > d'_{BP}. \tag{5.6}$$

Inside the breakpoint the exponent is 2.2 (close to free space); outside it is 4.0. The last term of $\mathrm{PL_2}$ makes the curve continuous at d'_{BP}.

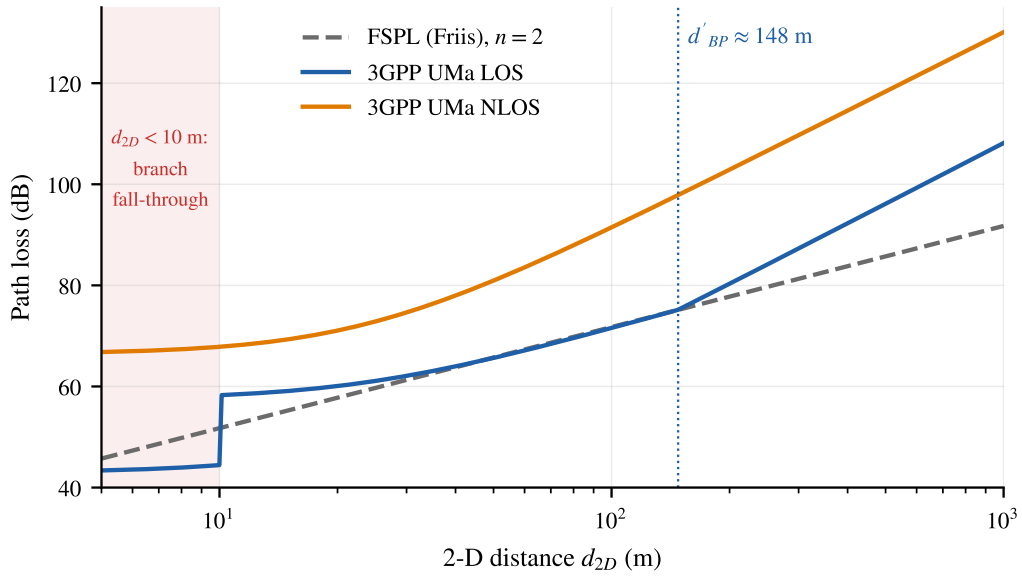


Figure 5.1: Path loss at 924 MHz with $h_{BS} = 25$ m, $h_{UT} = 1.5$ m, computed by Amber's own functions. Free space (dashed) and UMa LOS agree closely up to the breakpoint $d'_{BP} \approx 148$ m, after which LOS steepens to a 40 dB/decade slope. NLOS is $\approx$ 11–22 dB worse. The shaded region at left marks the code's non-physical behaviour below 10 m (see the reality check below).

Non line of sight (NLOS). The 3GPP formula is

$$\mathrm{PL}'_{\mathrm{NLOS}} = 13.54 + 39.08 \log_{10} d_{3D} + 20 \log_{10} f_c - 0.6\,(h_{UT} - 1.5), \tag{5.7}$$

and the model uses $\mathrm{PL}_{\mathrm{NLOS}} = \max\left(\mathrm{PL}_{\mathrm{LOS}}, \mathrm{PL}'_{\mathrm{NLOS}}\right)$ so that NLOS is never better than LOS. The code selects LOS or NLOS with the single flag `CoverageMap.los`; there is no probabilistic LOS/NLOS switching with distance.

d_{2D} (m)	d_{3D} (m)	FSPL(d_{2D}) (dB)	UMa LOS (dB)	UMa NLOS (dB)
20	30.9	57.8	60.1	71.1
30	38.1	61.3	62.1	74.6
50	55.2	65.7	65.6	80.9
100	102.7	71.8	71.6	91.5
148	149.7	75.2	75.2	97.9
300	300.9	81.3	87.3	109.7
500	500.6	85.7	96.1	118.4
1000	1000.3	91.8	108.2	130.1

Generated by calling Amber's `fspl_lin` and `urban_macro_loss_db_38_901`.

Path loss at 100 m by hand

$d_{3D} = \sqrt{100^2 + 23.5^2} = 102.72$ m, below the breakpoint, so Eq. (5.5): $28 + 22 \log_{10} 102.72 + 20 \log_{10} 0.924 = 28 + 44.26 - 0.69 = 71.57$ dB, matching the table and the downlink budget of the demo's Node 2 (Chapter 3).

5.4 Medium attenuation

`compute_rx_power_watts` accepts an optional `extra_np_per_m`, a medium attenuation α in nepers per metre, and multiplies the received power by $e^{-\alpha d}$. Two remarks. First, in electromagnetics a "neper per metre" usually quantifies attenuation of the field *amplitude* (8.686 dB/m per Np/m); because Amber applies the factor to *power*, its α corresponds to $4.343\,\alpha$ dB/m. Second, the attenuation is applied once per hop, so a backscatter link suffers it *twice*. It is zero by default.

5.5 What Amber assumes about the channel

Everything in this chapter is a *deterministic, static* function of distance and angles. There is no small-scale fading (Rayleigh/Rician), no log-normal shadow fading (3GPP specifies σ_{SF} of several dB for UMa), no multipath delay spread, no frequency selectivity and no mobility. This is a conscious system-level simplification: it makes link budgets computable *once* (Chapter 12) and results repeatable, at the price of not reproducing the variability of real channels.

Two behaviours of the path-loss code near the mast

(1) A 14 dB step at 10 m. The LOS formula selects PL_1 only for $10 \le d_{2D} \le d'_{BP}$ (the range in which 3GPP defines it). For $d_{2D} < 10$ m the code falls through to the *post-breakpoint* branch PL_2. The result at 5 m is 43.4 dB, whereas PL_1 would give 57.7 dB; the curve jumps by 13.8 dB between 9.99 m and 10 m (Figure 5.1). We measured this directly. Keep nodes at least 10 m (horizontally) from a mast. Note that the shipped custom-protocol demo places nodes from 5 m outward, so a few of its nodes enjoy this extra ~ 14 dB.
(2) Free space uses the horizontal distance. In `pathloss="fspl"` mode the function receives d_{2D}, not the slant range d_{3D}: the 25 m mast height is ignored. At $d_{2D} = 5$ m this understates the loss by 14 dB; at 100 m by only 0.2 dB. The UMa model does use d_{3D} internally.

Where this lives in the code

Concept	Code (`amber/propagation.py`)
Friis, Eq. (5.1), (5.2)	`fspl_lin`, `compute_rx_power_watts`
UMa LOS/NLOS	`urban_macro_loss_db_38_901`
Model selection	`pathloss` string: `"fspl"` or `"macro"`/`"3gpp"`/`"urban"`
Medium attenuation	factor `np.exp(-extra_np_per_m*distance_m)`

5.6 Chapter summary

- Friis: $P_r = P_t G_t G_r (\lambda/4\pi d)^2$—spreading plus a λ^2 aperture; $\mathrm{PL}_{FS}(1\,\mathrm{m}, 924\,\mathrm{MHz}) = 31.76$ dB.
- UMa LOS has exponent 2.2 before and 4.0 after the breakpoint $d'_{BP} = 4(h_{BS}-1)(h_{UT}-1)f_c/c$; NLOS is $\max(\mathrm{LOS}, \mathrm{NLOS}')$.
- Amber's channel is deterministic and static: no fading or shadowing.
- Beware $d_{2D} < 10$ m (a 14 dB discontinuity) and the horizontal-distance convention in FSPL mode.

Exercises

Exercise 1

Derive Eq. (5.1) from the two ideas of Section 5.1, and then compute the free-space loss at 924 MHz and 3.5 GHz for 50 m with $G_r = G_t = 1$. Why is the second larger?

Exercise 2

Compute d'_{BP} for $h_{BS} = 30$ m, $h_{UT} = 2$ m at 2 GHz. Is a node at $d_{2D} = 200$ m before or after the breakpoint?

Exercise 3

Using Eq. (5.6), compute the UMa LOS loss at $d_{2D} = 300$ m for the demo parameters (924 MHz, 25/1.5 m). Compare with the table.

Exercise 4

A medium attenuation of $\alpha = 0.01$ Np/m is configured. What is the extra loss in dB per hop at 100 m as Amber applies it? And over the round trip of a backscatter link?

Exercise 5

By what factor (in dB) does the *uplink* power at the base station change when a node moves from 50 m to 100 m in the LOS regime below the breakpoint, compared with the change of the *downlink* power at the node?

6

Backscatter Communication

In this chapter you will learn

- why a backscatter radio consumes microwatts where a conventional radio consumes milliwatts;
- how switching an antenna's load *modulates* a reflection, using the reflection coefficient;
- how to derive the *two-way* (radar-like) link budget and why its exponent doubles—exactly the formula behind Amber's uplink;
- the power-split trade-off between harvesting and reflecting, parameterised by η;
- why strong carrier self-interference forces *frequency-shifted* backscatter, how a square-wave switch creates sidebands, and what Amber's *subcarrier shifts* abstract;
- how a device with no receiver chain can still receive commands, and what the paper's dual-tone PRDCH idea contributes.

6.1 Active radio versus backscatter radio

A conventional radio transmitter must *create* a carrier. A crystal oscillator sets the reference, a phase-locked loop multiplies it to the carrier frequency, a mixer imposes the data, and a power amplifier raises the signal to the level the link budget requires. Each block draws power; the amplifier alone, at a typical efficiency of tens of percent, makes transmission cost on the order of tens to hundreds of milliwatts for a wide-area link.

A *backscatter* transmitter does none of that. It does not create RF energy at all. It relies on an *external source*—in our case the base station's carrier—and changes how its own antenna *reflects* the wave that arrives. The only active element is a switch (a transistor) and the logic that drives it, which can run on microwatts. The price is that

- the transmit power is only a fraction of what the device received, so the uplink is weak (Section 6.3);
- a strong external carrier must be present while the device talks;
- the reader's receiver must pick a faint reflection out of the carrier's own huge leakage (Section 6.5).

Key idea

Backscatter trades *device energy* for *infrastructure complexity*: it moves the expensive parts of a radio (oscillator, mixer, amplifier) from the many cheap devices to the few base stations.

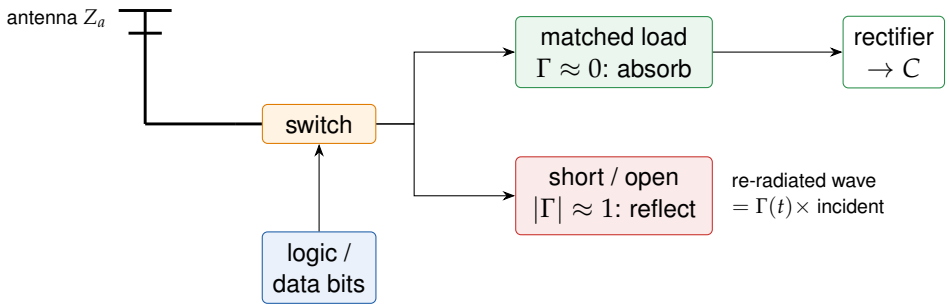


Figure 6.1: A backscatter front-end. A digital switch connects the antenna either to a matched path that feeds the rectifier (energy harvesting) or to a reflecting termination. The data bits control the switch and thereby impose themselves on the reflected carrier. There is no oscillator, mixer or power amplifier.

6.2 Modulating a reflection

Consider an antenna with complex impedance Z_a terminated by a load Z_L. An incident wave delivers power to the load unless the load reflects it. The (power-wave) *reflection coefficient* at the antenna terminals is

$$\Gamma \;=\; \frac{Z_L - Z_a^*}{Z_L + Z_a}. \tag{6.1}$$

Two extremes capture the idea [8, 18].

- **Absorbing (matched) state:** $Z_L = Z_a^*$ gives $\Gamma = 0$. All captured power enters the load and can be *rectified for harvesting*.
- **Reflecting state:** $Z_L = 0$ (short) or $Z_L = \infty$ (open) gives $|\Gamma| = 1$. The captured power is re-radiated.

A transistor that switches the load between two states Γ_0 and Γ_1 therefore *modulates the amplitude (and/or phase) of the reflected wave*, which is the incident carrier multiplied by $\Gamma(t)$. Switching between $\Gamma_0 = 0$ and $|\Gamma_1| = 1$ gives amplitude-shift keying (on–off keying); switching between $+1$ and -1 gives binary phase-shift keying (compare the OOK/BPSK modulations that 3GPP's Release-19 summaries report for the uplink).

6.3 The two-way link budget

The key difference to a conventional link: the signal travels over *two* hops, the forward hop (reader to tag) and the reverse hop (tag to reader). Apply the Friis equation twice.

Forward hop. The power captured by the tag is $P_{\text{tag}} = P_t\, G_t\, G_a\, L(d_1)$, where G_a is the tag antenna gain and $L(d_1) = (\lambda/4\pi d_1)^2$ in free space.

Re-radiation. The tag re-radiates a fraction η of that power (the *backscatter efficiency*): $P_{\text{bs}} = \eta\, P_{\text{tag}}$.

Reverse hop. The reader receives $P_r = P_{\text{bs}}\, G_a\, G_r\, L(d_2)$. Combining,

$$P_r \;=\; P_t\, G_t\, G_r\, G_a^2\, \eta\, L(d_1)\, L(d_2) \;\overset{\text{free space}}{=}\; \frac{P_t\, G_t\, G_r\, G_a^2\, \eta\, \lambda^4}{(4\pi)^4\, d_1^2\, d_2^2}. \tag{6.2}$$

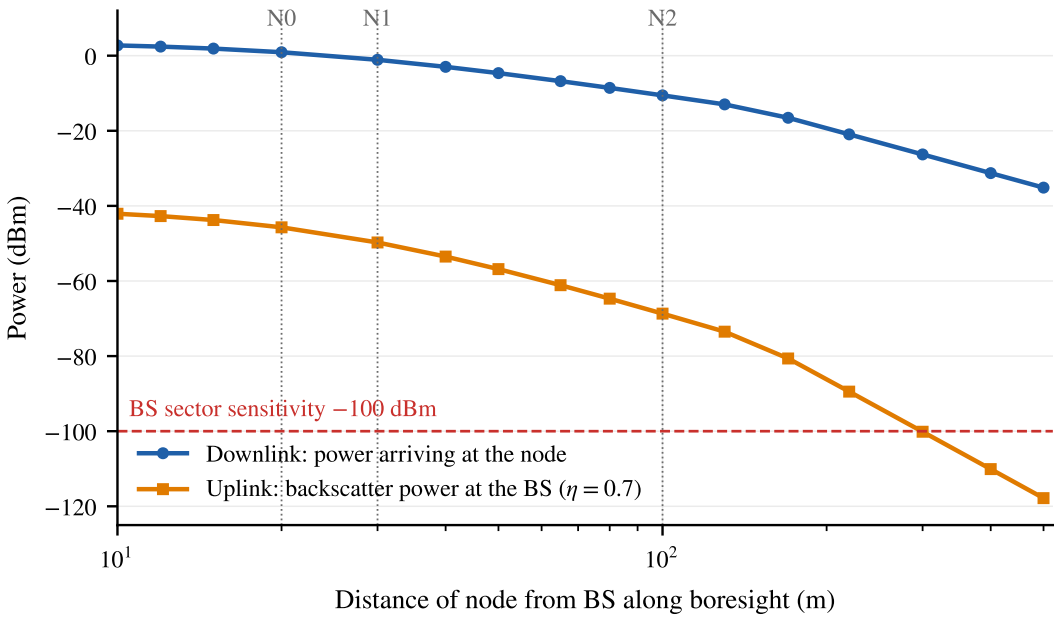


Figure 6.2: Downlink power at the node and uplink backscatter power at the base station against distance, for the demo's north-facing sector (46 dBm, 15 dBi, $\eta = 0.7$), computed by Amber. The uplink starts ~ 45 dB lower and falls twice as fast. With the demo's -100 dBm sector sensitivity the link would close out to about 300 m; the node would be energy-limited long before.

This is the *dyadic backscatter channel*, a close cousin of the radar equation. When the reader's transmit and receive antennas coincide (a *monostatic* reader), $d_1 = d_2 = d$ and

$$P_r \propto d^{-4}.$$

The path-loss exponent doubles. In decibels,

$$P_r\,[\text{dBm}] = P_t + G_t + G_r + 2G_a + 10\log_{10}\eta - \text{PL}(d_1) - \text{PL}(d_2). \tag{6.3}$$

Amber's uplink is Eq. ( )

The simulator does not have a special "backscatter channel." It composes two ordinary hops. `calculate_node_power` stores the downlink power multiplied by η as the node's `backscatter_power_dbm`; `compute_point_to_bs` then treats that as the *transmit power* of a new link, multiplies it by the node's antenna gain, applies the same path loss again, and multiplies by the sector's receive gain. In decibels this is exactly Eq. (6.3) with $d_1 = d_2$ (same sector, same node, same model). The hand calculation of Chapter 3 reproduced the logged -45.71 dBm to two decimals.

Which hop limits the system? A tag works only if *both* conditions hold:

(forward / energy)	P_{tag} large enough to power the device,
(reverse / information)	$P_r \geq S_{\text{rx}}$.

Which one fails first depends on design. For a passive tag with no storage, the forward link usually limits: the rectifier needs of order -20 dBm or more to turn on. For a device with a battery or large storage the reverse link limits. In Amber's demos the downlink is generous

(milliwatts at ≤ 30 m), and the uplink margin at 100 m is over 30 dB, so the energy *accumulation time* (Chapter 8) rather than the link margin decides how often a node can talk.

The paper's scenario: can an RF-only node even start?

The paper's experiment uses 30 dBm sectors with 17 dBi, 1 dBi nodes and 924 MHz. At 20 m the UMa LOS loss is 60.1 dB, so the node receives $30 + 17 + 1 - 60.1 = -12.1$ dBm $= 62\,\mu$W; at 100 m only -23.6 dBm $= 4.4\,\mu$W. Under Amber's harvesting model with the demo's $R_s = 5\,\text{k}\Omega$ these correspond to equivalent source voltages $\sqrt{PR_s} \approx 0.56$ V and 0.15 V (Chapter 7), far below the $V_{high} = 1.7$ V used in the demos. Even the best case beyond 10 m gives only ≈ 0.65 V. *Such nodes can never wake up on RF alone*—which is why the paper operates them in **hybrid power mode** with an external (solar) source. The thresholds and R_s of the paper's run are not stated in the abstract; we use the demo values.

6.4 Splitting power between harvesting and reflecting

An antenna captures a fixed amount of power. What it *reflects* it cannot also *rectify*; to first order,

$$P_{\text{harvest}} + P_{\text{reflect}} = P_{\text{captured}}.$$

The paper describes nodes by "a power split ratio determining how received RF power is divided between energy harvesting and backscatter reflection." In Amber that ratio is $\eta =$ `Node.efficiency` $\in [0,1]$ (default 1), and the bookkeeping is a *state-dependent* one:

$$P_{\text{backscatter}} = \eta\, P_{\text{dl}} \qquad \text{(always computed)}, \tag{6.4}$$

$$P_{\text{harvest}} = \begin{cases} P_{\text{dl}} & \text{node not transmitting,} \\ (1-\eta)\, P_{\text{dl}} & \text{node in the } \textit{transmitting} \text{ state.} \end{cases} \tag{6.5}$$

While the node is idle its antenna absorbs everything; while it transmits a fraction η is reflected and so lost to the harvester. Because transmissions are brief this costs little energy, but the model captures the principle that *communicating and harvesting compete for the same photons*. For Node 0 ($P_{\text{dl}} = 1.24$ mW, $\eta = 0.7$) the equivalent source voltage drops from 2.49 V to 1.36 V for the duration of a transmission.

What η must absorb

In Amber η is a *single scalar* that must stand for everything that reduces the reflected *information-bearing* power: the incomplete modulation depth, the conversion of carrier power into the wanted sideband (next section), the antenna's mismatch and structural scattering. A square-wave-shifted OOK tag, for instance, puts only $(1/\pi)^2 \approx 0.10$ of the incident power into each first sideband. The default $\eta = 1$ is therefore the most optimistic choice; the shipped demos use 0.7. Choose it deliberately.

6.5 Self-interference and frequency-shifted backscatter

Recall the dynamic-range problem of Section 3.5: the reader's own carrier, even after ~ 25 dB of antenna isolation, is tens of dB stronger than the reflected data. A receiver cannot filter it out if the wanted signal sits at the *same* frequency. The solution is to move the reply away in frequency.

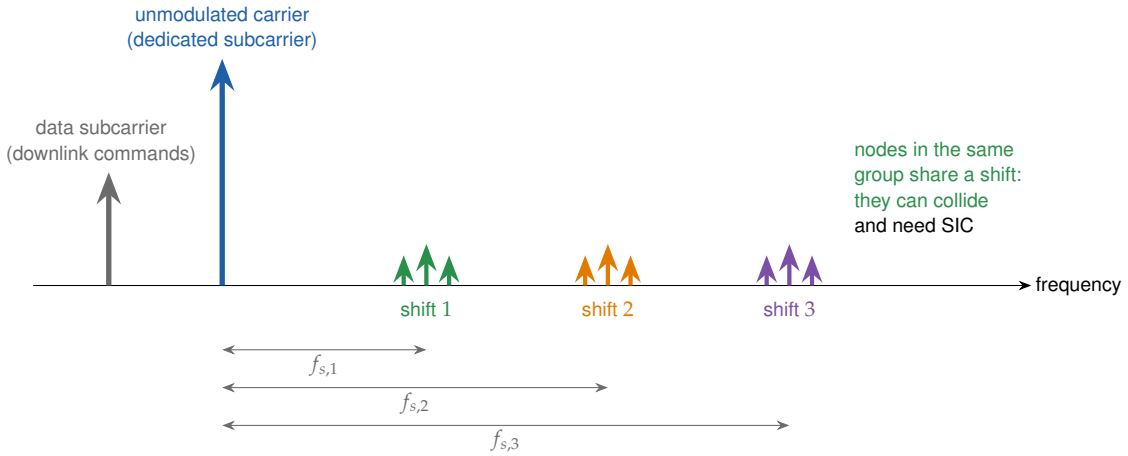


Figure 6.3: Frequency-domain picture of the dual-tone downlink and frequency-shifted backscatter. A clean carrier on a dedicated subcarrier powers and illuminates the nodes; one data-bearing subcarrier carries the downlink. Each node reflects its reply at an offset $f_{s,k}$ from the carrier (shown on one side only; real sidebands appear on both). Different offsets are *orthogonal*; nodes with the same offset overlap in frequency and time and must be separated by capture or SIC (Chapter 10).

Let the incident carrier be $x(t) = A\cos(2\pi f_c t)$ and let the tag toggle its reflection coefficient between Γ_0 and Γ_1 with a unit square wave $s(t) \in \{0,1\}$ of frequency f_s, so that $\Gamma(t) = \Gamma_0 + \Delta\Gamma\, s(t)$, $\Delta\Gamma = \Gamma_1 - \Gamma_0$. The Fourier series of the square wave is $s(t) = \frac{1}{2} + \frac{2}{\pi}\sum_{k\ \text{odd}} \frac{1}{k}\sin(2\pi k f_s t)$. The reflected signal $y(t) = \Gamma(t)x(t)$ is therefore

$$y(t) = \left(\Gamma_0 + \tfrac{\Delta\Gamma}{2}\right) A\cos(2\pi f_c t) \tag{6.6}$$

$$+ \frac{\Delta\Gamma}{\pi} A\Big[\sin\big(2\pi(f_c+f_s)t\big) - \sin\big(2\pi(f_c-f_s)t\big)\Big] + \text{odd harmonics}. \tag{6.7}$$

Term (6.6) is a copy of the carrier at f_c (the *static* reflection, carrying no data). The information-bearing energy appears at $f_c \pm f_s$ in (6.7). Each first-order sideband has power $(\Delta\Gamma/\pi)^2$ times the incident carrier power: for $\Delta\Gamma = 1$ (on–off) that is -9.9 dB; for a phase flip between $+1$ and -1 ($\Delta\Gamma = 2$) it is -3.9 dB. The reader now tunes its receiver to $f_c + f_s$, where the carrier leakage is absent.

Frequency shifting as multiple access. If different nodes choose *different* f_s, their replies land in different frequency slots and do not collide even if they transmit simultaneously. The paper's three *subcarrier shifts* exploit exactly this: "Distinct subcarrier offsets provide frequency-domain separation, and successive interference cancellation (SIC) resolves collisions among nodes sharing the same offset." Subcarrier shifting thus has *two* roles: keeping replies away from carrier interference, and acting as a coarse FDMA-style resource.

How Amber models subcarrier shifts

Amber does not synthesise any spectrum. `Node.subcarrier_shift` is an *integer label*; the base station groups packets by that label and assumes that different labels are *perfectly orthogonal* (Chapter 10). Neither carrier leakage, adjacent-shift leakage, nor the sideband loss of Eq. (6.7) is modelled explicitly—the latter is yours to place inside η.

6.6 Receiving with no receiver: the passive downlink

So far we have covered uplink. How does a μW device *hear* commands? A conventional receiver needs a low-noise amplifier, a local oscillator and a mixer, and consumes milliwatts. A passive device instead uses an *envelope detector*: a diode followed by a low-pass filter that outputs the slowly varying *amplitude* of the incoming signal, much like a crystal radio. It works naturally with *on–off keying* (OOK): a "1" is carrier present, a "0" is carrier absent. The 3GPP Release-19 summaries describe exactly this: an RF envelope-detection receiver and OOK on the reader-to-device link. The costs are poor sensitivity (typically in the region of -40 dBm to -50 dBm for simple passive detectors, tens of dB worse than an active receiver) and low data rates—which is why Amber's default *node* sensitivity is -45 dBm while its *sector* sensitivity is -100 dBm.

OFDMA downlinks for passive nodes. Cellular networks use OFDM, whose many subcarriers are normally separated by an FFT, which is expensive. The paper cites Zhu et al. [25], who demonstrate OFDMA downlink demodulation at around 100 µW by replacing the digital FFT with *analogue filtering*. This is the enabling idea behind the Amber paper's *dual-tone principle*.

The dual-tone PRDCH. The paper says that Amber's physical layer is by default based on the *Physical Reader to Device Channel* (PRDCH): "an external unmodulated carrier occupies a dedicated OFDM subcarrier, enabling both energy harvesting and backscatter reflection [25], while a single data-bearing subcarrier carries downlink information from base station to node." Two tones, two jobs:

1. the *carrier tone* is a clean sinusoid, ideal both for harvesting (constant, rectifier-friendly power) and as the illumination to be reflected (nothing to cancel except the carrier itself);
2. the *data subcarrier* carries OOK-like commands that an analogue filter can separate from the carrier.

Figure 6.3 sketches this arrangement. In 3GPP's terms the PRDCH is the physical reader-to-device channel studied in TR 38.769.

The downlink is not waveform-simulated

In Amber a command is a Python call. It is delivered if the best downlink power reaching the node exceeds the node's sensitivity; there are no bit errors, no demodulation, no data-rate limit and no per-command energy cost to the node. The *carrier* is implicit and always on: the base-station behaviour class states that nodes "harvest energy all the time" and that "the schedule only handles communication".

6.7 Topologies and duplexing

Literature distinguishes three backscatter arrangements [12].

Monostatic. The reader transmits the carrier and receives the reflection (one device, or two co-located). The forward and return distances are equal, $P_r \propto d^{-4}$. This is Amber's arrangement.

Bistatic. A separate *carrier emitter* illuminates the tag while the receiver is elsewhere; $d_1 \neq d_2$. This resembles 3GPP's topology T3 (with an assisting node).

Ambient. The carrier is an unrelated, already-present signal (TV, Wi-Fi, cellular [14]); the tag has no control over it.

The paper describes the reference experiment as a polling protocol "operating in full-duplex mode". Because the carrier is transmitted *continuously* on its own subcarrier, the base station can receive backscatter while the carrier is on. Amber's *schedule* nevertheless alternates TX (command) and RX (reply) windows: the command is on the data subcarrier and the base station does not listen while it sends one (Chapter 17).

Where this lives in the code

Concept	Code (`amber/`)
Backscatter / harvest split η	`CoverageMap.calculate_node_power`; `Node.efficiency`
Dyadic uplink budget, Eq. (6.3)	`CoverageMap.compute_point_to_bs`
Subcarrier label	`Node.subcarrier_shift`; `BSBehavior._process_rx_buffer`
Command reception gate	`BackscatterModule.receive_command`
Always-on carrier	documented in `BSBehavior`; no explicit object

6.8 Chapter summary

- Backscatter saves energy by switching a reflection instead of generating a carrier; the switch alternates between absorbing ($\Gamma \approx 0$) and reflecting ($|\Gamma| \approx 1$).
- The dyadic link $P_r = P_t G_t G_r G_a^2 \eta\, L(d_1) L(d_2)$ has a doubled path-loss exponent; Amber implements it as two chained hops.
- η splits received power between reflection and harvesting; a transmitting node harvests only $(1-\eta)P_{\text{dl}}$.
- A square-wave switch creates sidebands at $f_c \pm f_s$ carrying $(\Delta\Gamma/\pi)^2$ of the power; shifting the reply away from the carrier defeats self-interference and enables frequency-domain multiple access.
- Passive devices receive with envelope detectors; the PRDCH dual-tone design pairs a clean carrier with one data subcarrier.

Exercises

Exercise 1

Derive Eq. (6.2) step by step. If both d_1 and d_2 double, by how many dB does P_r change in free space? By how many dB does the *downlink* power change?

Exercise 2

A reader transmits 36 dBm through a 9 dBi antenna at 915 MHz. A tag with a 2 dBi antenna and $\eta = 0.3$ is 4 m away; the same antenna receives. Compute the free-space power the tag captures and the power returned to the reader. (Take the reader receive gain equal to its transmit gain.)

Exercise 3

Using Eq. (6.7), find the fraction of incident power in one first-order sideband when the tag toggles between $\Gamma_0 = 0$ and $\Gamma_1 = -0.8$. What η would you configure in Amber to include this loss?

Exercise 4

In Amber, Node 1 (at 30 m) transmits and its `efficiency` is 0.7. By how many dB does the harvested power fall while it transmits? Repeat for $\eta = 0.2$ and $\eta = 1.0$.

Exercise 5

Explain, with reference to Figure 6.3, why nodes assigned the *same* subcarrier shift can collide but nodes with *different* shifts cannot, and what resource this resembles in a cellular system.

7

Energy Sources and RF Harvesting

In this chapter you will learn

- which ambient energy sources can power a node, and roughly how much each delivers;
- how a rectenna turns RF into DC and why its efficiency depends strongly on input power;
- Amber's harvesting model: how received power becomes an *equivalent source voltage* behind a series resistor, and what that implies about how much energy reaches the capacitor;
- the three *power modes*—wireless power transfer, external, hybrid—and the difference between combining them by *max* and by *sum*;
- how an external energy trace is replayed, and a surprise about the shipped trace.

7.1 Where can the energy come from?

"Ambient" in Ambient IoT refers to the environment as the supplier of energy and, sometimes, of the carrier. The main candidates differ by orders of magnitude.

Source	Typical power density	Remarks
Outdoor sunlight	$\sim 10\,\mathrm{mW/cm^2}$ and more	Very high, intermittent (day/night, shade)
Indoor light	$\sim$ 10–100 $\mu\mathrm{W/cm^2}$	Moderate; depends on lighting
Thermal gradients	tens of $\mu\mathrm{W/cm^2}$	Needs a temperature difference
Vibration	$\sim 100\,\mu\mathrm{W/cm^3}$	Machinery, vehicles
Ambient RF	$\ll 1\,\mu\mathrm{W/cm^2}$	Very low except near a transmitter
***Dedicated* RF (carrier)**	depends on distance	Controlled; what Amber's base stations provide

Order-of-magnitude values commonly quoted in the energy-harvesting literature; actual numbers vary widely with conditions.

Ambient RF alone is too weak to be dependable, so 3GPP Ambient IoT relies on the network itself to *beam* energy at the devices: the base station's carrier is both the power source and the illumination for backscatter. A node can additionally use a second source—typically light—so that it can wake up even where the RF field is weak. Amber lets you model exactly that combination.

7.2 The RF harvesting chain

A *rectenna* (rectifying antenna) consists of

1. the antenna, which captures the RF power P_{dl};
2. an impedance-matching network;
3. a rectifier—diodes or transistors configured as a voltage multiplier—that converts the oscillating RF voltage into DC;
4. a smoothing capacitor and a power-management stage that feed the storage element.

The *RF-to-DC conversion efficiency* $\eta_{\text{rf-dc}} = P_{\mathrm{dc}}/P_{\mathrm{rf}}$ depends strongly on the input power, because a diode barely conducts until the RF voltage exceeds its threshold. At very low input power (below roughly -20 dBm for commonly used devices) the efficiency collapses, and many rectifiers cannot output anything usable at all—the *cold-start* problem. At moderate power the efficiency peaks (tens of percent) and at high power the diode breaks down. A rectenna therefore has a *minimum input power* below which the harvested energy is effectively zero, however long you wait. This is why, in a real deployment, the *forward link* often limits range.

Amber has no rectifier curve

In Amber every watt of received RF power reaches the harvester: effectively $\eta_{\text{rf-dc}} = 1$ at all input levels, with no turn-on threshold, no saturation and no cold start. The code comment simply converts received power to DC power. If you need a realistic harvester, the hook is the single conversion in `CoverageMap.calculate_node_power` (Exercise 7.6).

7.3 Amber's harvesting model

7.3.1 From power to an equivalent source

The capacitor model of Chapter 8 is driven by a *voltage* source in series with a resistor R_{s}. The controller therefore turns the harvested power P_h (a number in dBm stored in `node.harvesting_power_dbm`) into an equivalent open-circuit voltage by

$$V_{\mathrm{src}} = \sqrt{P_h\, R_{\mathrm{s}}}, \qquad \text{i.e. } P_h = \frac{V_{\mathrm{src}}^2}{R_{\mathrm{s}}}. \tag{7.1}$$

This is the usual $P = V^2/R$ read backwards. Physically it says: "a source of internal resistance R_{s} and open-circuit voltage V_{src} would *dissipate* P_h if short-circuited into a resistance equal to its own". It is a modelling device, not a circuit that exists on a board; but its consequences are physical and worth understanding.

How much of P_h actually reaches the capacitor?

When the capacitor sits at voltage V, the current is $(V_{\mathrm{src}} - V)/R_{\mathrm{s}}$ and the power *delivered into the capacitor* is $V(V_{\mathrm{src}} - V)/R_{\mathrm{s}}$. With $x = V/V_{\mathrm{src}}$,

$$P_{\text{into C}}(x) = P_h\, x(1 - x),$$

which is a parabola with maximum $P_h/4$ at $x = \tfrac{1}{2}$ (the familiar *maximum-power-transfer* theorem). So the *effective* harvesting efficiency of the model is at most 25% and falls to zero as the capacitor approaches V_{src}. For Node 0 at $V = 1.5$ V, $x = 0.60$ and the capacitor receives $0.24\,P_h \approx 0.30$ mW of the 1.24 mW nominally harvested; the rest is dissipated in R_{s}. The charge-only update of the simulator reproduces this behaviour exactly (Chapter 8).

7.3.2 Three power modes

The harvest power of each node is recomputed from the pre-computed downlink power by `CoverageMap.calculate_node_power`, according to `node_energy_mode`. Let P_{dl} be the best downlink power (in watts) at the node, η its `efficiency`, P_{ext} the instantaneous power of the external source, and $\mathbb{1}_{\text{tx}}$ be 1 while the node is in its *transmitting* state and 0 otherwise.

Mode	Harvested power P_h	Backscatter power
`"wpt"`	$P_{\text{dl}}\left(1-\eta\,\mathbb{1}_{\text{tx}}\right)$	ηP_{dl}
`"external"`	P_{ext} (RF is *not* harvested)	ηP_{dl}
`"hybrid"`	$\max\left(P_{\text{wpt}}, P_{\text{ext}}\right)$ if `combine_mode="max"` (default), else $P_{\text{wpt}}+P_{\text{ext}}$	ηP_{dl}

where $P_{\text{wpt}} = P_{\text{dl}}(1-\eta\,\mathbb{1}_{\text{tx}})$. Three points deserve emphasis.

1. The backscatter power is ηP_{dl} in *every* mode: a node powered by solar light still needs the base station's carrier to reflect.
2. "Hybrid" does *not* mean "sum". With the default `combine_mode="max"` the node simply uses whichever source is stronger at that instant.
3. The base station carriers are not summed either: the node harvests the *best* sector's power only (`best_pw_w`), a conservative simplification when several carriers overlap.

Why the hybrid node of the demos lives on sunlight

In the broadcast demos Node 2, 100 m from the base station, receives $P_{\text{dl}} = -10.57\,\text{dBm} = 87.7\,\mu\text{W}$, giving $V_{\text{src}} = \sqrt{87.7\times10^{-6}\cdot 5000} = 0.662\,\text{V}$ from RF. The steady-state voltage (Chapter 8) is $V_{\text{src}}R_{\text{l}}/(R_{\text{l}}+R_{\text{s}}) = 0.63\,\text{V} \ll V_{\text{high}}$, so from RF alone Node 2 would *never* start. The solar trace has mean power 1.43 mW ($V_{\text{src}} \approx 2.66\,\text{V}$), 16× larger. Because `combine_mode="max"` picks the larger term, *the RF contribution is irrelevant*; Node 2's harvesting is solar-only, which is why it activates fastest and sends the most packets (Figure 2.2). With `combine_mode="sum"` the two would add in power: $\sqrt{(87.7+1433)\,\mu\text{W}\cdot 5000} = 2.76\,\text{V}$.

7.4 The external source

`EnvEnergySource` is a SimPy actor that *replays a recorded trace*. It reads an Excel file, takes one column (default `V_IM`) of voltages, and at every simulation millisecond sets

$$P_{\text{ext}}(t_k) = \frac{V_k^2}{R}, \qquad R = \texttt{resistance}\ (1\,\Omega \text{ default; the demos pass } R_s = 5\,\text{k}\Omega).$$

After the last row it loops. The object exposes the current value as `ext_power`; the coverage map reads it through the callback `node_ext_power_fn`, which in the demos is `lambdanode:energy_source.ext_power`. Because the callback ignores its argument, *all nodes see the same external power at the same instant*: there is no spatial variation (shading) of the solar source.

When `resistance` equals the controller's R_{s}, the round trip $V \to P = V^2/R \to \sqrt{PR_{\text{s}}}$ returns the recorded voltage unchanged. In the demos the trace's `V_IM` column is thus directly the equivalent source voltage of the solar harvester.

The "300-second" trace is three seconds long

The file `threehundred_seconds_stable.xlsx` contains 2999 rows with a time step of 1 ms (the last time stamp is 2.998 s). Since `EnvEnergySource` consumes one row per millisecond and loops, the trace is *three seconds long*, repeated. Its statistics are: V_{IM} between 1.78 and 3.01 V, mean 2.66 V, standard deviation 0.25 V, i.e. a nearly constant ≈ 1.4 mW source (1.56 dBm). For simulations of tens of seconds the periodicity of the trace could show up as spurious structure.

7.5 When is harvesting updated?

Harvest power is recomputed *every millisecond for every node* inside the controller loop (Chapter 15). For a pure WPT node this matters only when its state toggles into or out of *transmitting* (the $1 - \eta$ factor); for a node with an external source it also tracks the trace. The comment in the controller source explains the economy: if no coverage map or downlink results are supplied, "wpt is static, no need to recalculate every ms".

Where this lives in the code

Concept	Code
Harvest/backscatter bookkeeping	`CoverageMap.calculate_node_power` (`propagation.py`)
Hybrid combination	`combine_mode` (`"max"` default)
Power to source voltage, Eq. (7.1)	`Controller.run`: `(power_w*R_series)**0.5`
Solar trace	`EnvEnergySource` (`energy.py`)
Callback	`CoverageMap.node_ext_power_fn`

7.6 Chapter summary

- Sunlight dwarfs ambient RF by orders of magnitude; 3GPP therefore beams a dedicated carrier at the devices.
- Real rectennas have a minimum input power and a power-dependent efficiency; Amber omits both.
- Amber converts harvested power to a Thévenin-style source $V_{\text{src}} = \sqrt{P_h R_s}$; at most $P_h/4$ ever reaches the capacitor.
- Modes: wpt, external, hybrid (max or sum). Backscatter always uses the base-station carrier.
- The shipped trace is 3 s long, shared by all nodes, and effectively a 2.7 V source.

Exercises

Exercise 1

The solar trace has mean voltage 2.66 V and $R = 5\,\text{k}\Omega$. Compute the external power in mW and dBm. (Careful: use the mean of V^2 or of V? How big is the difference given $\sigma = 0.25$ V?)

Exercise 2

In hybrid mode, $P_{\text{wpt}} = 1\,\text{mW}$ and $P_{\text{ext}} = 1.4\,\text{mW}$. Compute the equivalent source voltage ($R_{\text{s}} = 5\,\text{k}\Omega$) for `combine_mode="max"` and for `"sum"`.

Exercise 3

A WPT node with $\eta = 0.7$ receives 1 mW. By how many dB does its harvested power drop while it is transmitting? What if $\eta = 0.2$?

Exercise 4

For which received downlink power P_{dl} can a WPT-only node eventually reach $V_{\text{high}} = 1.7\,\text{V}$ with $R_{\text{s}} = 5\,\text{k}\Omega$ and $R_{\text{l}} = 100\,\text{k}\Omega$? Express your answer in µW and dBm. (Hint: you need the steady-state formula of Chapter 8.)

Exercise 5

Suppose you want to include an RF-to-DC efficiency $\eta_{\text{rf-dc}}(P_{\text{in}})$ that is zero below $-20\,\text{dBm}$ and 0.3 above. Where in the pipeline $P_{\text{dl}} \rightarrow V_{\text{src}}$ would you insert it, and how would this change the paper's scenario of Section 6.3?

8

Energy Storage: The Capacitor and Intermittent Operation

In this chapter you will learn

- the circuit model of a harvester feeding a capacitor through a series resistor, with leakage and a load;
- how to solve it: time constants, steady state, charge time, sustainable current;
- how Amber discretises the model with a 1 ms step and why the discretisation is accurate;
- why a node needs *two* voltage thresholds (hysteresis), how big an "energy window" they create, and the *duty-cycle theorem* $D = P_{\text{in}} / P_{\text{active}}$;
- the *energy-accounting* subtlety in the simulator's charge-while-active step, with a measured size of its effect.

8.1 Why a capacitor?

A harvesting node needs somewhere to *accumulate* energy between its brief bursts of activity. A rechargeable battery has high energy density but wears out, leaks chemically and is costly; a *capacitor* (or supercapacitor) stores less but can be charged and discharged almost without limit, which suits devices intended to live for a decade. Its great limitation is that its voltage is not constant:

$$Q = CV, \qquad E = \tfrac{1}{2}CV^2, \qquad i = C\frac{\mathrm{d}V}{\mathrm{d}t}. \tag{8.1}$$

Draining a capacitor lowers its voltage, and electronics need a minimum voltage. The *usable* energy is therefore only that between two voltages, a point that drives the threshold logic of Section 8.5.

8.2 The circuit

Amber's capacitor sees the circuit of Figure 8.1: the harvester as a source V_{src} behind a series resistance R_{s} (Section 7.3), the capacitor C, a parallel *leakage* resistance R_{l} that models self-discharge, and a load that draws a current I_L chosen by the node's state.

Kirchhoff's current law at the capacitor node gives the governing equation

$$C\frac{\mathrm{d}V}{\mathrm{d}t} = \frac{V_{\text{src}} - V}{R_{\text{s}}} - \frac{V}{R_{\text{l}}} - I_L. \tag{8.2}$$

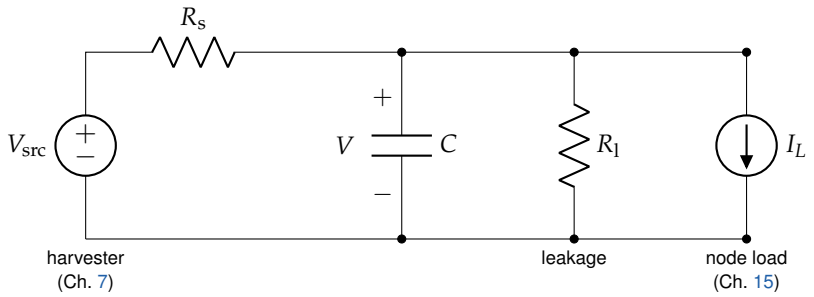


Figure 8.1: Equivalent circuit of Amber's energy store. Typical demo values: $R_s = 5\,\text{k}\Omega$, $R_l = 100\,\text{k}\Omega$, $C = 300\,\mu\text{F}$ (200 µF in the SIC demo); the load current I_L takes values 0.14 mA–5 mA depending on the node's state.

8.3 Solving it

For a constant V_{src} and I_L, Eq. (8.2) is linear first order. Define

$$\tau = (R_s \parallel R_l)\, C = \frac{R_s R_l}{R_s + R_l}\, C, \qquad V_\infty = \left(V_{src} - I_L R_s\right) \frac{R_l}{R_l + R_s}. \tag{8.3}$$

Then

$$V(t) = V_\infty + \left(V_0 - V_\infty\right) \mathrm{e}^{-t/\tau}. \tag{8.4}$$

Look at what the pieces mean.

Time constants. With the demo's $R_s = 5\,\text{k}\Omega$, $R_l = 100\,\text{k}\Omega$, $C = 300\,\mu\text{F}$: the charging constant is $\tau_c = R_s C = 1.5\,\text{s}$, the leakage constant $\tau_\ell = R_l C = 30\,\text{s}$, and the effective constant $\tau = 1.43\,\text{s}$ is dominated by charging.

Steady state. With no load ($I_L = 0$) the capacitor settles at $V_{src}\, R_l/(R_l + R_s)$: a *voltage divider* between the series and leakage resistances. With $R_l/R_s = 20$ the final voltage is 95.2% of V_{src}; leakage steals about 5%. If a load draws I_L, the steady voltage falls by $I_L R_s\, R_l/(R_l + R_s)$. A node is *energy-starved* if $V_\infty < V_{low}$ for its typical load: it can never sustain operation.

Sustainable current. At voltage V the harvester can supply a steady current $(V_{src} - V)/R_s$ and leakage takes V/R_l, so the *sustainable* load is

$$I_{max}(V) = \frac{V_{src} - V}{R_s} - \frac{V}{R_l}. \tag{8.5}$$

At $V = 1.5\,\text{V}$, Node 0 ($V_{src} = 2.49\,\text{V}$) can sustain 182 µA, which exceeds the 140 µA listening current: it could idle in the listening state indefinitely. Node 1 ($V_{src} = 1.97\,\text{V}$) can sustain only 79 µA, so even continuous listening would drain it; it *must* duty-cycle. This one-line calculation predicts a lot about who will be awake.

Time to reach a threshold. Solving Eq. (8.4) for $V(t) = V_{th}$ starting at $V_0 < V_{th} < V_\infty$:

$$t_{th} = \tau \ln \frac{V_\infty - V_0}{V_\infty - V_{th}}. \tag{8.6}$$

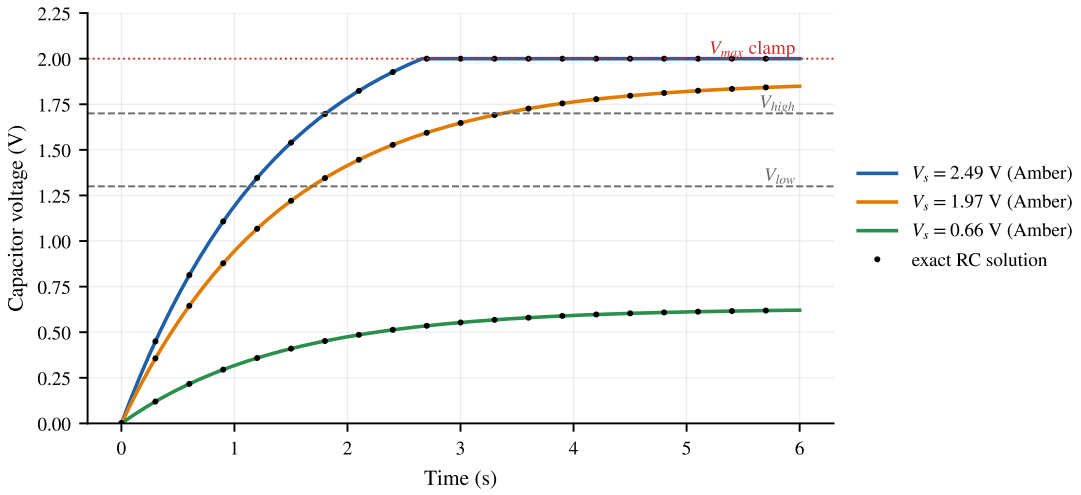


Figure 8.2: Charging from empty for three source voltages. Lines: Amber's discrete update; dots: the exact solution Eq. (8.4) (they coincide). The top curve is clipped by $V_{\max} = 2\,\mathrm{V}$. The bottom curve ($V_{\mathrm{src}} = 0.66\,\mathrm{V}$) saturates far below V_{low} and never activates a node.

Activation times of the demo nodes

From $V_0 = 0$ with $V_{\mathrm{high}} = 1.7\,\mathrm{V}$ and no load:

Node	V_{src} (V)	V_∞ (V)	$t_{1.7\,\mathrm{V}}$ (s)	Remark
Node 0, WPT, 20 m	2.486	2.37 (clamped to 2.0)	1.81	
Node 1, WPT, 30 m	1.971	1.88	3.37	slower
Node 2, RF only	0.662	0.63	never	$V_\infty < V_{\mathrm{high}}$
Node 2, solar (mean)	2.665	2.54 (clamped to 2.0)	1.58	what the demo uses

The measured activations in Figure 2.2 (≈ 1.8, ≈ 3.4 and $\approx 1.4\,\mathrm{s}$) agree; Node 2's slightly earlier activation reflects the trace's initial values ($\approx 2.7\,\mathrm{V}$ to $3.0\,\mathrm{V}$). A random start-up delay of up to 2 s (Chapter 15) can postpone activation further.

8.4 Discretisation: how Amber steps the equation

Amber advances the capacitor in steps of $\Delta t = 1\,\mathrm{ms}$. Because $\Delta t/\tau \approx 7 \times 10^{-4} \ll 1$, even a crude scheme is accurate; Amber uses a hybrid of two methods per step. For the charge-only case:

$$V_{k+1} = V_k\,\mathrm{e}^{-\Delta t/(R_{\mathrm{l}}C)} + \frac{\Delta t}{R_{\mathrm{s}}C}\left(V_{\mathrm{src}} - V_k\right). \tag{8.7}$$

The leakage term is integrated *exactly* (an exponential decay), the source term by one *explicit-Euler* step; this is an *operator-splitting* scheme with error $O(\Delta t)$ per unit time. Explicit Euler is stable here because $\Delta t/(R_{\mathrm{s}}C) \ll 2$.

Verified: the charge-only step is accurate to 2×10^{-4} V

Starting from 1.0 V with $V_{\mathrm{src}} = 2.5\,\mathrm{V}$ (unclamped), after 1 s Amber gives 1.695 36 V; the exact solution Eq. (8.4) gives 1.695 19 V.

8.4.1 The four step types

The `Capacitor` class implements four updates; the controller selects among them by sending a SimPy *interrupt* with a string cause (Chapter 13). Each ends with a clamp to $[0, V_{\max}]$ ($V_{\max} = 2\,\mathrm{V}$, "physical clamp / PMIC max").

Cause (interrupt)	Update (per Δt)
`capacitor_charging`	Eq. (8.7): charge from V_{src} with leakage. Used while off or waiting for a slot.
`capacitor_full`	Leakage only: $V \leftarrow V\,\mathrm{e}^{-\Delta t/(R_{\mathrm{l}}C)}$. Sent once, at activation.
`capacitor_discharge`	Load only: $\Delta E = VI\Delta t$, $V \leftarrow \sqrt{V^2 - 2\Delta E/C} - V(1 - \mathrm{e}^{-\Delta t/R_{\mathrm{l}}C})$. *Not sent by the shipped controller.*
`capacitor_charge_and_discharge`	Simultaneous: $I_c = \max\{(V_{\mathrm{src}} - V)/R_{\mathrm{s}}, 0\}$; $E \leftarrow \max\{E + V_{\mathrm{src}} I_c \Delta t - VI_L\Delta t,\, 0\}$; $V \leftarrow \sqrt{2E/C}\,\mathrm{e}^{-\Delta t/R_{\mathrm{l}}C}$. Used in every active state.

The capacitor is passive. The `Capacitor`'s own process merely *waits*; its voltage changes *only when the controller interrupts it*. Because the controller issues exactly one interrupt per millisecond, one controller tick equals one capacitor step. As we verified experimentally, SimPy schedules an interrupt with *urgent* priority, so the capacitor reacts within the *same* millisecond, before ordinary events at that time. A capacitor created without a controller would hold its voltage forever, with no leakage.

8.4.2 Discharge

Under a constant-current load I_L the leakage-free solution of Eq. (8.2) is $V(t) = V_0 - I_L t/C$. Amber states this via *energy*: the load draws power VI_L, so in one step $\Delta E = VI_L\Delta t$ and $V' = \sqrt{V^2 - 2\Delta E/C} \approx V - I_L\Delta t/C$ for small steps—the same result. A 15 ms, 5 mA transmission therefore lowers a capacitor of 300 µF by

$$\Delta V = \frac{I_L\,T}{C} = \frac{5\times10^{-3}\cdot 15\times10^{-3}}{300\times10^{-6}} = 0.25\,\mathrm{V},$$

which is exactly the step height of the sawtooth in Figure 2.2 (we measured 1.5 → 1.248 V for the load-only step, against 1.25 V for ideal constant current).

8.5 Thresholds, hysteresis and the energy window

A node cannot operate at an arbitrarily low voltage, and turning it on at the *same* voltage at which it turns off would cause *chattering*: starting up draws current, the voltage sags below the threshold, the node switches off, the voltage recovers, and so on. The remedy is *hysteresis*, familiar from the Schmitt trigger and from the under-voltage lock-out of energy-harvesting power-management ICs: two thresholds.

- V_{high} (demo: 1.7 V): the node switches *on* when $V \geq V_{\mathrm{high}}$.
- V_{low} (demo: 1.3 V): the node switches *off* when $V \leq V_{\mathrm{low}}$.

The energy available between the thresholds is

$$\Delta E = \tfrac{1}{2}C\left(V_{\mathrm{high}}^2 - V_{\mathrm{low}}^2\right) = \tfrac{1}{2}(300\,\mu\mathrm{F})(1.7^2 - 1.3^2) = 180\,\mu\mathrm{J}. \tag{8.8}$$

The stored energy at 1.7 V is 433.5 µJ, of which only 41% is usable; the rest remains as the unusable floor $\frac{1}{2}CV_{\mathrm{low}}^2 = 253.5\,\mu\mathrm{J}$. A larger C gives a larger window (longer bursts, longer

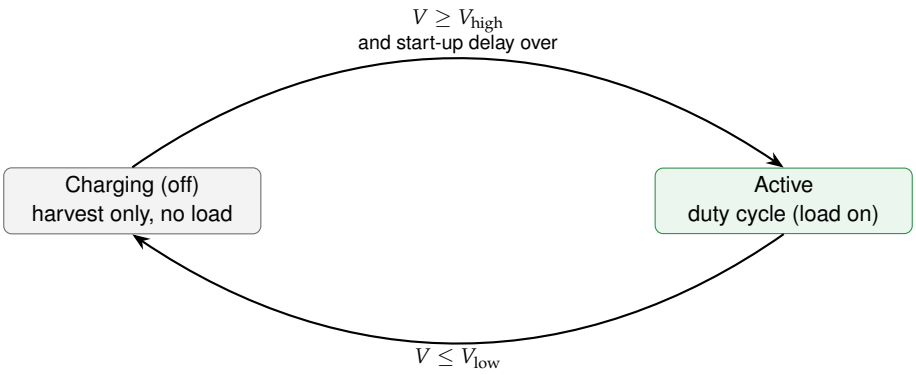


Figure 8.3: Hysteretic two-state behaviour of an energy-harvesting node. The 0.4 V gap between thresholds sets the size of the usable energy window.

recharge); a wider gap does too.

How many packets fit in one window? We will see in Chapter 15 that one full node cycle (listen, sense, process, transmit) consumes about 125 µJ, of which 90% is the 15 ms transmission. The window of 180 µJ would therefore hold only ≈ 1.4 cycles without harvesting. The measured behaviour is a useful check on this arithmetic. In a 20 s run of the broadcast demo, Node 0 completed 23 charge cycles and sent 47 packets, Node 1 sent 18 in 9 cycles and Node 2 sent 56 in 27 cycles: **about two packets per saw-tooth tooth**. The first transmission of a tooth runs to completion (15 ms); the second is typically *cut short*, because V reaches V_{low} after only 9–11 ms (visible in Figure 15.2). We return to this in Chapter 15.

Design rule: the window must cover an atomic burst

If an operation cannot be interrupted (a radio transmission, a flash write), the energy window $\frac{1}{2}C(V_{\text{high}}^2 - V_{\text{low}}^2)$ minus what the node has already spent must exceed the energy of that atomic burst, or the node will die in the middle of it. Amber does *not* enforce this: the node simply powers down whenever $V \leq V_{\text{low}}$, wherever it happens to be in its state machine.

8.5.1 The duty-cycle theorem

Let the net charging power into the capacitor be P_{in} while the node is off, and let the node draw $P_{\text{act}} > P_{\text{in}}$ while active (approximating both as constant). One cycle consists of recharging by ΔE and then spending it:

$$T_c = \frac{\Delta E}{P_{\text{in}}}, \qquad T_a = \frac{\Delta E}{P_{\text{act}} - P_{\text{in}}}.$$

The fraction of time the node is active is

$$D = \frac{T_a}{T_a + T_c} = \frac{\dfrac{1}{P_{\text{act}} - P_{\text{in}}}}{\dfrac{1}{P_{\text{act}} - P_{\text{in}}} + \dfrac{1}{P_{\text{in}}}} = \frac{P_{\text{in}}}{P_{\text{act}}}. \tag{8.9}$$

Duty-cycle theorem

The long-run fraction of time an intermittently powered node is active equals the ratio of harvested power to active-mode power, *independent of the energy window* ΔE. The window (C, V_{high}, V_{low}) determines only the *granularity*: how long each active burst and each recharge

pause lasts, not how much activity there is on average.

Energy neutrality ($\overline{P}_{\text{consumed}} = \overline{P}_{\text{in}}$) is Eq. (8.9) in disguise. For Node 0 at $V \approx 1.5\,\text{V}$ the capacitor receives about 0.30 mW (Section 7.3); if the node transmitted continuously (7.5 mW at 1.5 V) it could be active only $D \approx 4\%$ of the time.

8.5.2 What the node forgets

In the language of *intermittent computing* [16] a power failure erases volatile state. Amber's controller resets its state machine to *listening* at each reactivation, but *keeps* the last processed sensor value and the node's protocol state (for example, whether it is registered). That models non-volatile memory for registration; a design that loses registration at every outage would behave differently.

The active-mode step credits source energy, not delivered energy

The charge-only step, Eq. (8.7), is the exact RC behaviour: the power delivered into the capacitor is $P_h\,x(1-x)$ (Figure 8.4). The *charge-and-discharge* step used in every active state computes the energy gained as $V_{\text{src}}\,I_c\,\Delta t$, where $I_c = (V_{\text{src}} - V)/R_{\text{s}}$. That is the energy *drawn from the source*, which includes the heat dissipated in R_{s}; the energy actually stored is $V\,I_c\,\Delta t$. The simulator therefore over-credits harvesting by the factor V_{src}/V whenever a node is active.

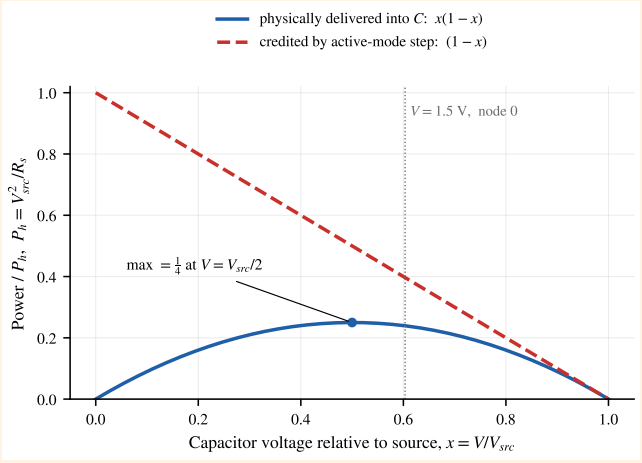


Figure 8.4: Power delivered into C (solid) versus the power credited by the active-mode step (dashed), both relative to $P_h = V_{\text{src}}^2/R_{\text{s}}$.

We measured the effect directly. With zero load the two steps should agree; instead, starting from 1 V with $V_{\text{src}} = 2.5\,\text{V}$, after 1 s the charge-only step reaches 1.695 V (exact: 1.695 V) and the charge-and-discharge step 1.948 V (Figure 8.5).

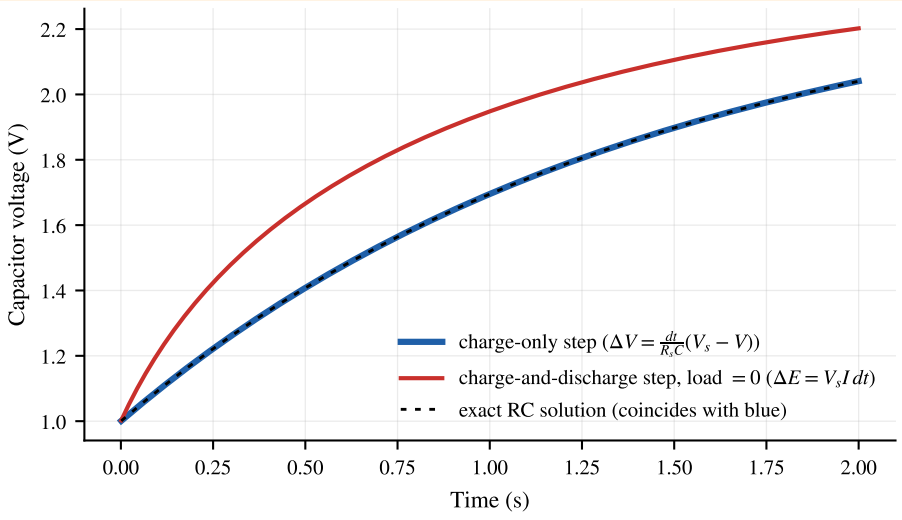


Figure 8.5: Zero-load charging from 1 V: the charge-only step coincides with the exact RC solution; the charge-and-discharge step charges faster.

For the shipped demos the practical impact is small because the load dominates the active-state energy budget. Re-running the broadcast scenario ten times with a patched step that credits $V I_c \Delta t$ reduced the mean number of packets sent in 20 s from 124.4 to 121.0 ($\approx -2.7\%$). The effect would be larger for light loads or long active phases. Keep it in mind when interpreting duty-cycle numbers.

8.6 Chapter summary

- $C\dot{V} = (V_{\text{src}} - V)/R_{\text{s}} - V/R_{\text{l}} - I_L$ gives $V(t) = V_\infty + (V_0 - V_\infty)\mathrm{e}^{-t/\tau}$; $V_\infty = (V_{\text{src}} - I_L R_{\text{s}})R_{\text{l}}/(R_{\text{l}} + R_{\text{s}})$; $\tau = (R_{\text{s}} \parallel R_{\text{l}})C$.
- A node can sustain a current $I_{\max} = (V_{\text{src}} - V)/R_{\text{s}} - V/R_{\text{l}}$; nodes whose typical load exceeds it must duty-cycle or starve.
- Amber advances the capacitor with a 1 ms hybrid step (exact leakage, Euler source). The charge-only step matches the exact solution to 2×10^{-4} V.
- Hysteresis ($V_{\text{high}} > V_{\text{low}}$) avoids chattering; $\Delta E = \frac{1}{2}C(V_{\text{high}}^2 - V_{\text{low}}^2) = 180\,\mu\text{J}$ in the demos.
- Duty-cycle theorem: $D = P_{\text{in}}/P_{\text{act}}$.
- The active-mode step over-credits harvest by V_{src}/V; $\approx 3\%$ effect on demo packet counts.

Where this lives in the code

Concept	Code (`amber/capacitor.py`, `controller.py`)
$R_s, R_l, C, \Delta t$	`CapacitorParams`
Eq. (8.7)	`Capacitor.capacitor_charging`
Leakage only	`Capacitor.capacitor_selfdischarge`
Load only	`Capacitor.load_discharging`
Charge + discharge	`Capacitor.charge_and_discharge`
Clamp $V_{\max}$	`voltage_max`
Thresholds	`VoltageThresholdsV(high,low)`
Who steps the capacitor	`Controller.run`, via `action.interrupt(cause)`

Exercises

Exercise 1

For $R_s = 5\,\text{k}\Omega$, $R_l = 100\,\text{k}\Omega$, $C = 200\,\mu\text{F}$ (the SIC demo): compute τ, and the time to charge from 0 to 1.7 V with $V_{src} = 2.486\,\text{V}$.

Exercise 2

A node draws a constant 300 µA while active. For what source voltages V_{src} is it energy-starved at $V_{low} = 1.3\,\text{V}$ (use $V_\infty < V_{low}$ with the load included)?

Exercise 3

Compute ΔE for $C = 200\,\mu\text{F}$, $V_{high} = 1.7\,\text{V}$, $V_{low} = 1.3\,\text{V}$. By what factor must C change to double the usable window? What if instead V_{low} were lowered to 1.1 V?

Exercise 4

Derive Eq. (8.6) and verify the entry $t = 3.37\,\text{s}$ for Node 1 ($V_{src} = 1.971\,\text{V}$).

Exercise 5

A node harvests $P_{in} = 200\,\mu\text{W}$ into its capacitor and, while active, draws 4 mW. What is its long-run duty cycle? Using the energy window $\Delta E = 180\,\mu\text{J}$, estimate T_c, T_a and the number of active bursts per second.

Exercise 6

Show that the maximum of $V(V_{src} - V)/R_s$ over V is $V_{src}^2/(4R_s)$, and use this to explain why the model never delivers more than 25% of the nominal harvested power.

Part III

Sharing the Medium

9

Random Access: From ALOHA to Framed Slots

In this chapter you will learn

- why devices that cannot listen before they talk need *random access*;
- how to derive the throughput of pure and slotted ALOHA from the Poisson model;
- *framed* slotted ALOHA, the scheme used by RFID readers and by Amber's registration and custom protocols: expected idle, success and collision slots, and the optimal frame size;
- how a reader can estimate the number of contenders and adapt its frame (Schoute, EPC Gen2's Q-algorithm), and how Amber's custom demo does it;
- why an *energy-limited* network wants different parameters from a throughput-limited one, and how subcarrier shifts multiply capacity.

9.1 The problem: many talkers, one channel

A base station illuminates, say, a hundred nodes. When it asks "who is there?", every node would like to answer at once; all answers share the same carrier. If two reflections overlap in time on the same frequency they interfere, and (unless capture or interference cancellation rescues them; Chapter 10) neither is decoded.

The classical remedies are *scheduling* (the base station assigns each node a slot) and *random access* (each node picks a slot by itself). Scheduling needs the base station to know who exists; discovering who exists is precisely the problem. Hence random access. For a battery-free node there is a second reason: *it cannot sense the channel*. Carrier-sense multiple access (CSMA), the workhorse of Wi-Fi, requires a receiver that is awake and sensitive before every transmission—far beyond a microwatt budget. So the only practical rule is *"transmit once, at a random time the reader has authorised"*.

9.2 Pure ALOHA

Norman Abramson's ALOHA system [5] let radio terminals in Hawaii transmit whenever they liked. Model it as follows. Packets have duration T. New transmission attempts form a Poisson process with an average of g attempts per packet time (the *offered load*). A packet starting at time t_0 is successful if no other packet starts in the *vulnerable interval* $(t_0 - T,\ t_0 + T)$, which has length $2T$. For a Poisson process the probability of no arrival in an interval of length $2T$ is e^{-2g}.

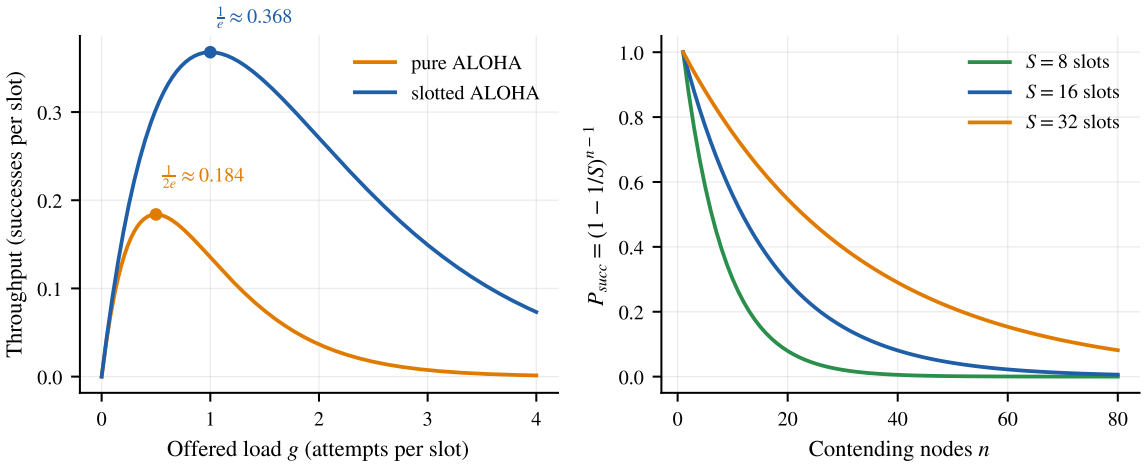


Figure 9.1: Left: throughput of pure and slotted ALOHA. Right: the success probability of a single node in a frame of S slots when n nodes contend (Eq. (9.7)).

Throughput (successes per packet time) is

$$s_{\text{pure}}(g) = g\,\mathrm{e}^{-2g}, \qquad s_{\max} = \frac{1}{2\mathrm{e}} \approx 0.184 \text{ at } g = \tfrac{1}{2}. \tag{9.1}$$

9.3 Slotted ALOHA

If time is divided into slots of length T and everyone must start at a slot boundary, two packets collide only if they choose the *same* slot. The vulnerable interval shrinks to T, and

$$s_{\text{slot}}(g) = g\,\mathrm{e}^{-g}, \qquad s_{\max} = \frac{1}{\mathrm{e}} \approx 0.368 \text{ at } g = 1. \tag{9.2}$$

Synchronisation doubles the efficiency [19].

Why does the curve fall for large g?

Throughput = load × probability of success. Raising the load first helps (more attempts) and then hurts (each attempt is more likely to collide). The maximum balances the two at $g = 1$: on average exactly one attempt per slot.

9.4 Framed slotted ALOHA

RFID readers, and Amber's registration, use *framed* slotted ALOHA (FSA). The base station announces a *frame* of S slots; each of the n contending nodes picks one slot uniformly at random and answers there. Because the number of nodes is finite and they choose independently, the analysis is a balls-into-bins problem.

Slot statistics. The number of nodes in a given slot is Binomial$(n, 1/S)$. Hence

$$\Pr[\text{slot idle}] = \left(1 - \tfrac{1}{S}\right)^{n}, \tag{9.3}$$

$$\Pr[\text{slot has exactly one node}] = \frac{n}{S}\left(1 - \tfrac{1}{S}\right)^{n-1}, \tag{9.4}$$

$$\Pr[\text{collision}] = 1 - (\text{the two above}). \tag{9.5}$$

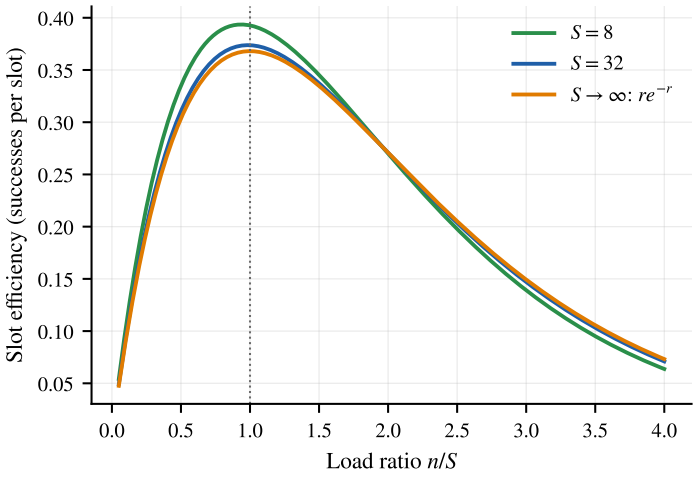


Figure 9.2: Slot efficiency of framed slotted ALOHA against the load ratio n/S. All curves peak near $n/S = 1$ at about 0.37–0.39; the $S \to \infty$ limit is re^{-r}.

The expected numbers of idle, successful and collision slots in a frame are

$$E_0 = S\left(1 - \tfrac{1}{S}\right)^{n}, \qquad E_1 = n\left(1 - \tfrac{1}{S}\right)^{n-1}, \qquad E_c = S - E_0 - E_1, \tag{9.6}$$

and a single node succeeds with probability

$$P_{\text{succ}} = \left(1 - \tfrac{1}{S}\right)^{n-1} \approx \mathrm{e}^{-n/S} \tag{9.7}$$

(nobody else chose its slot). The *slot efficiency* is

$$\eta_S = \frac{E_1}{S} = \frac{n}{S}\left(1 - \tfrac{1}{S}\right)^{n-1} \xrightarrow{S\to\infty} \frac{n}{S}\mathrm{e}^{-n/S}. \tag{9.8}$$

Maximising over S gives the **optimal frame size** $S = n$, with efficiency $\approx 1/\mathrm{e} = 0.368$. Figure 9.2 shows the curves.

A frame with 20 nodes and 16 slots

By Eq. (9.6): $E_0 = 16(15/16)^{20} = 4.40$ idle slots; $E_1 = 20(15/16)^{19} = 5.87$ successes; $E_c = 5.73$ collision slots; efficiency $5.87/16 = 0.367$. Each node succeeds with probability $(15/16)^{19} = 0.293$. Roughly seven of the twenty nodes are *not* served in this frame, and a second frame is needed for them.

Multi-frame inventory. In an inventory the reader repeats frames until every node is identified. Suppose it could always choose $S = n_k$, the number of nodes still unidentified. A frame then identifies $E_1 = n_k(1 - 1/n_k)^{n_k - 1} \approx n_k/\mathrm{e}$ of them in expectation, so

$$n_{k+1} \approx n_k\left(1 - \tfrac{1}{\mathrm{e}}\right) = 0.63\, n_k,$$

and the unidentified population decays geometrically. Going from $n = 100$ to below one node takes $\ln 100 / \ln(1/0.632) \approx 10$ frames. The practical lesson is that *the right frame size shrinks as nodes are identified*, so the reader has to keep estimating n_k.

9.5 Estimating the number of contenders

The reader observes only the *outcome* of each slot: idle (c_0 of them), success (c_1) or collision (c_c). Schoute [20] showed that at the optimal operating point a collision slot holds on average 2.39 nodes, giving the estimate

$$\hat{n} = c_1 + 2.39\, c_c. \tag{9.9}$$

A simple lower bound is $n \geq c_1 + 2c_c$. EPC's Gen2 standard [9] implements a variant of dynamic framing through its *Q-algorithm*: the frame has 2^Q slots and the reader adjusts a floating-point Q_{fp} by $\pm C$ after each collision (increase) or idle (decrease) slot.

Amber's custom protocol. The shipped `custom_protocol` demo implements an adaptive FSA in the spirit of the Q-algorithm (its docstring says "EPC-Gen2 Q-algorithm style"). After each frame the base station

1. counts heard (decoded replies) and collided (replies lost to collisions) and forms the number of contenders $m = \text{heard} + \text{collided}$ (each transmission increments exactly one of the two counters);
2. smooths it, $\ell \leftarrow 0.75\,\ell + 0.25\,m$;
3. sets a target frame size $\min\{32, \max\{2, \text{round}(2\ell)\}\}$ and moves the actual frame size *one slot* toward it, with a one-slot dead band above the target.

Two comments. First, the base station here knows m *exactly*, because the simulator logs every colliding packet individually; a real reader sees only that a slot collided (and would estimate n with Schoute's formula). This is an idealisation. Second, the target $S = 2\ell$ is not the throughput-optimal $S = n$; the next section shows why that can be a good choice here.

9.6 An energy-aware twist

In RFID a collision wastes *airtime*. In an Ambient IoT node it also wastes *joules*: the node has already paid for the transmission. With the demo's numbers, a transmission costs about $E_{\text{tx}} \approx 112\,\mu\text{J}$ (Chapter 15). The expected energy per *successfully delivered* packet is

$$\bar{E}_{\text{deliv}} = \frac{E_{\text{tx}}}{P_{\text{succ}}} = \frac{E_{\text{tx}}}{(1 - 1/S)^{n-1}}. \tag{9.10}$$

$n = 30$ nodes	slot efficiency	P_{succ}	energy per delivered packet
$S = n = 30$	0.374	0.374	299 µJ
$S = 2n = 60$	0.307	0.614	182 µJ

Doubling the frame costs about 18% of slot efficiency but cuts the energy spent per delivered packet by 39%.

Key idea

A frame-size policy that is optimal for *airtime* is not optimal for *energy*. When every transmission drains a scarce capacitor, a larger frame ($S > n$) is worth its idle slots. This is exactly the trade-off that Amber lets you explore, and a reason the demo's rule-of-thumb $S \approx 2\ell$ is sensible.

There is a second coupling: the *effective* n is the number of nodes that are *awake* when the frame is announced, which varies with their capacitors. The base station cannot observe that

number (Chapter 11).

9.7 Several orthogonal channels: subcarrier shifts

With K mutually orthogonal subcarrier shifts (Chapter 6) and nodes spread evenly over them, collisions occur only among nodes with the same shift. Each group of n/K nodes runs an independent FSA in the same S time slots, so the per-node success probability is $(1-1/S)^{n/K-1}$. If instead each node picked its (slot, shift) pair uniformly at random, the number of independent resources would be KS and $P_{\text{succ}} = (1-1/(KS))^{n-1} \approx \mathrm{e}^{-n/(KS)}$: the same to first order. Shifts and slots are interchangeable currencies: K shifts of S slots give the collision behaviour of KS slots at the airtime cost of S. For $n = 60$, $S = 10$, $K = 3$ the success probability is $(1-1/30)^{59} = 0.135$.

9.8 From the model to the simulator

Amber reproduces the framed structure literally. The base station's schedule is a list of RX windows, *each window being one slot*; a node chooses `random.randint(0,len(rx_slots)-1)`, uniformly, among the slots advertised (Chapter 16). Three simulator details modify the textbook picture and will be analysed in Chapters 10 and 17:

1. the base station advertises the slots of the *current and the next* frame, so only about half of the contenders can answer in a given frame;
2. two nodes in the same slot collide only if they transmit within a 5 ms *time window* of each other (they are not forced to transmit at the slot start), and only if they share a subcarrier shift;
3. capture and SIC resolve some collisions that the classical model counts as lost.

All three make Amber *more optimistic* than Eq. (9.7) for the *collision* aspect, and the first makes it *slower* per frame.

9.9 Chapter summary

- Battery-free nodes cannot carrier-sense, so the reader must authorise random-access slots.
- Pure ALOHA peaks at $1/(2e) \approx 0.184$, slotted ALOHA at $1/e \approx 0.368$.
- Framed slotted ALOHA: $E_1 = n(1-1/S)^{n-1}$, $P_{\text{succ}} = (1-1/S)^{n-1} \approx \mathrm{e}^{-n/S}$, optimum $S = n$.
- Schoute's estimator $\hat{n} = c_1 + 2.39\,c_c$ underlies adaptive framing; Amber's custom demo uses its own exact counts.
- For energy-limited tags a larger frame ($S \approx 2n$) can minimise energy per delivered packet.
- K orthogonal subcarrier shifts act like K times as many slots.

Where this lives in the code

Concept	Code
Random slot choice	`BackscatterModule.handle_command` (`random.randint`)
Frame = TX command + S RX windows	schedule entries `("tx",...)`, `("rx",...)` in `bsengine.py`
Adaptive frame size	`adaptive_aloha_policy` in `custom_protocol_demo.py`
Contender count	`decoded_this_frame`, `total_collided`

Exercises

Exercise 1

Derive the pure-ALOHA throughput and its maximum. Why is the vulnerable period twice as long as in the slotted case?

Exercise 2

For $n = 20$ nodes and $S = 16$ slots compute E_0, E_1, E_c and the efficiency (verify the worked example). How many slots would maximise efficiency, and what would it be?

Exercise 3

After a frame of $S = 16$ slots the reader observes $c_0 = 6$, $c_1 = 5$, $c_c = 5$. Estimate the number of contenders using Schoute's formula and the lower bound. Which frame size should it use next?

Exercise 4

Compute the energy per delivered packet for $n = 30$ and $S \in \{30, 45, 60, 90\}$ when $E_{\text{tx}} = 112\,\mu\text{J}$ and compare with the slot efficiency. Where would you set the frame size if idle slots cost nothing but airtime counts against latency?

Exercise 5

$n = 60$ nodes are divided equally among $K = 3$ subcarrier shifts and all reply in a frame of $S = 10$ slots. Compute the per-node success probability, and compare with $K = 1$.

10

Capture, SINR and Successive Interference Cancellation

In this chapter you will learn

- the *physical* (SINR-threshold) model of a collision and the *capture effect*;
- the principle of *successive interference cancellation* (SIC), why it works best when received powers differ, and how imperfect cancellation changes the SINR sequence;
- exactly how Amber groups, resolves and logs colliding packets, step by step;
- three behaviours of that pipeline that differ from textbook expectations, each verified by experiment, including a measured *SIC on/off* ablation.

10.1 From "collision" to SINR

Chapter 9 treated a collision as a binary event: if two packets share a slot, both are lost. This is the *protocol model*. Real receivers are more forgiving. A packet can be decoded despite overlapping with another if it is *strong enough* relative to everything else. This is the *physical model*: packet i is decoded if its signal-to-interference-plus-noise ratio

$$\gamma_i = \frac{P_i}{\sum_{j \neq i} P_j + N} \;\geq\; \gamma_{\text{req}}. \tag{10.1}$$

The threshold γ_{req} is a single number that stands in for the modulation, coding and receiver design (Amber's default in the base station is 3 dB).

The capture effect. When two packets collide and one is much stronger, the receiver locks onto it and ignores the other. Roberts called this *capture* [19]. For two packets of power $P_1 \geq P_2$ and negligible noise, capture requires

$$\frac{P_1}{P_2} \geq \gamma_{\text{req}} \quad \Longleftrightarrow \quad (P_1 - P_2)_{\text{dB}} \geq (\gamma_{\text{req}})_{\text{dB}}.$$

Noise changes this when the weaker packets are near the noise floor, as Figure 10.1 shows: a strong pair (-60 dBm) is interference-limited and the SINR grows without bound with the gap; a weak pair (-80 dBm) is noise-limited and saturates at about 8 dB no matter how small the interferer.

Why capture matters for backscatter. In Chapter 6 we saw that the uplink loses $2 \times 10\, n \log_{10} d$ dB: for a pair of nodes whose distances differ by a factor of 2, the received powers differ by

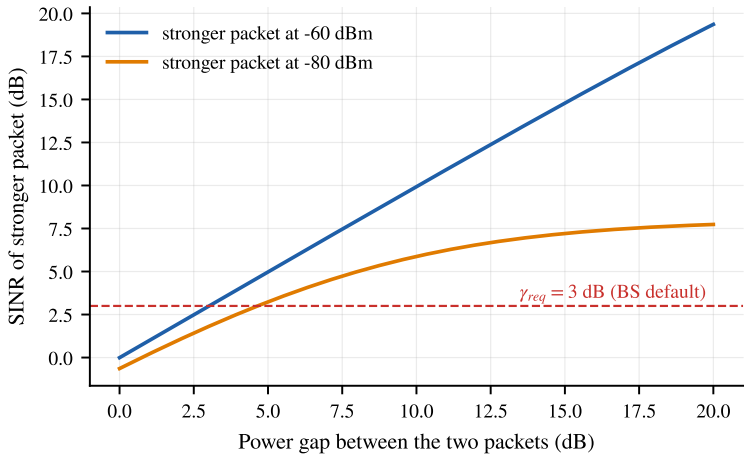


Figure 10.1: SINR of the stronger of two colliding packets against the power gap, for thermal noise -88 dBm. At -60 dBm the curve is interference-limited; at -80 dBm it saturates because noise dominates. The dashed line is Amber's default requirement of 3 dB.

$2 \cdot 22 \log_{10} 2 = 13.2$ dB below the UMa breakpoint and $2 \cdot 40 \log_{10} 2 = 24.1$ dB beyond it. This *near–far diversity* makes capture frequent: random placement itself supplies power differences.

10.2 Successive interference cancellation

10.2.1 The principle

Capture decodes the strongest packet and discards the rest. But the others may be perfectly decodable *once the strongest is out of the way*. SIC makes this explicit [7, 24]:

1. sort the received signals by power;
2. decode the strongest, treating all others (and noise) as interference;
3. if decoding succeeded, *reconstruct and subtract* that signal from the received waveform;
4. repeat on the remainder; stop at the first failure.

In information theory, SIC with ideal cancellation achieves the corner points of the multiple-access capacity region, which is why it is considered the receiver of choice in non-orthogonal multiple access (NOMA).

10.2.2 The SINR sequence

Let the powers sorted in decreasing order be $P_{(1)} \geq P_{(2)} \geq \cdots \geq P_{(m)}$ and let $\varepsilon \in [0, 1]$ be the fraction of an already-decoded signal that is *not* removed ($\varepsilon = 0$: perfect cancellation). When decoding the k-th strongest,

$$\gamma_{(k)} = \frac{P_{(k)}}{\sum_{j>k} P_{(j)} + \varepsilon \sum_{j<k} P_{(j)} + N}. \tag{10.2}$$

With perfect cancellation only *weaker* packets interfere. A useful design rule follows: if successive powers decrease by a constant ratio ρ and noise is negligible, the SINR of every stage is $1/(\rho^{-1} + \rho^{-2} + \cdots) = \rho - 1$, so SIC decodes all packets iff $\rho \geq 1 + \gamma_{\text{req}}$. *Each packet must be stronger than the sum of all weaker ones by the decoding threshold.* For $\gamma_{\text{req}} = 3$ dB this means $\rho \geq 3$ (4.8 dB) between neighbours.

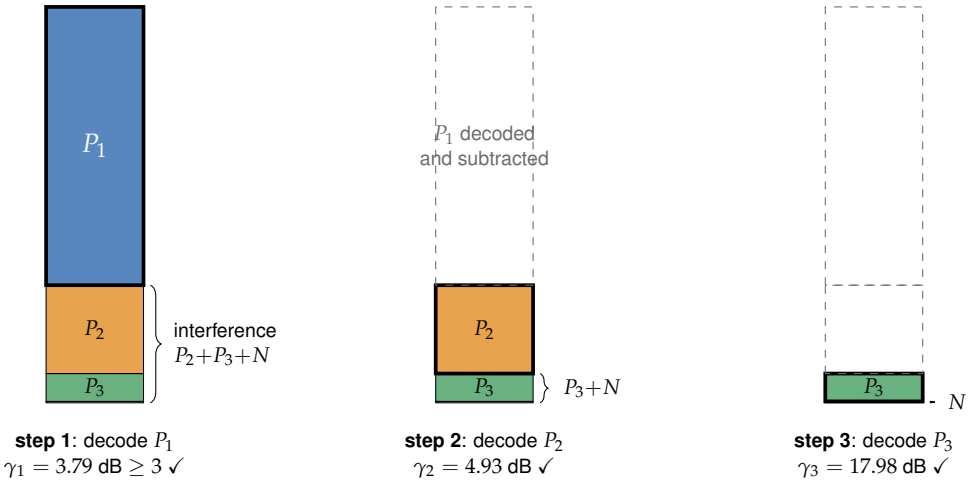


Figure 10.2: SIC on three colliding packets at -60, -65 and -70 dBm with a -88 dBm noise floor and $\gamma_{\text{req}} = 3$ dB. Bar heights are *linear* power. Without cancellation only the strongest would be decoded (capture); with SIC all three are.

Three packets, with and without SIC

Powers -60, -65, -70 dBm, i.e. 1.00, 0.316, 0.100 nW, noise 1.59 pW. Step 1: $\gamma_1 = 1/(0.316 + 0.100 + 0.0016) = 2.39 = 3.79\,\text{dB} \geq 3$, decoded. After removal, step 2: $\gamma_2 = 0.316/(0.100 + 0.0016) = 3.11 = 4.93\,\text{dB}$, decoded. Step 3: $\gamma_3 = 0.100/0.0016 = 62.8 = 17.98\,\text{dB}$, decoded. All three succeed; capture alone would deliver only the first. If the packets were -60 and -63 dBm, $\gamma_1 = 2.99\,\text{dB} < 3$: *nothing* is decoded, a razor-edge illustrating how sharp the threshold model is. With 10% residual ($\varepsilon = 0.1$) step 2 becomes $\gamma_2 = 0.316/(0.100 + 0.0016 + 0.1 \cdot 1.0) = 1.57 = 1.96\,\text{dB} < 3$: the second packet is *lost*. SIC is very sensitive to cancellation quality.

10.3 How Amber resolves collisions

The base station resolves collisions at the *end of each RX window*. Algorithm 1 states the procedure in plain steps; we then explain each design choice.

Design choice 1: the buffer is per RX window. Each `("rx",d,id)` schedule entry is a separate window, and the buffer is cleared at its start. Consequently *packets in different windows never collide*: a window is one *slot* in the sense of Chapter 9.

Design choice 2: collision means time-proximity, not overlap. A simulated backscatter packet has no duration in the buffer—it is a point event at the instant of transmission. The simulator therefore declares two packets to collide if they are within 5 ms of each other (the `collision_window_ms` argument). Within one slot of 8 ms–15 ms, two nodes that choose the same slot but start 6 ms apart (for instance because one was still in its *processing* state when the slot opened) do *not* collide.

Design choice 3: the group is built greedily. For each unused packet i, the group consists of i and *all unused packets within 5 ms of i*. This is a greedy neighbourhood, not transitive clustering. We tested arrivals at 10, 14, 18 ms: packets at 10 and 14 form a group and collide, but the packet at

Algorithm 1 Processing of one RX window (`BSBehavior._process_rx_buffer`)

Input: buffer $\mathcal{B}$ of arrivals $(t_i, \text{node}_i, P_i, \text{sector}_i, c_i)$ collected during this window; noise N; γ_{req}

1: group the arrivals by subcarrier shift c
2: **for** each shift group $\mathcal{G}$ **do**
3: sort $\mathcal{G}$ by arrival time; used $\leftarrow \emptyset$; coll $\leftarrow [\,]$
4: **for** each packet i in time order, $i \notin$ used **do**
5: $\mathcal{C} \leftarrow \{i\} \cup \{j \notin \text{used},\ j \neq i :\ |t_i - t_j| \leq 5\,\text{ms}\}$
6: **if** $|\mathcal{C}| > 1$ **then** append $\mathcal{C}$ to coll **end if**; used $\leftarrow$ used $\cup\, \mathcal{C}$
7: **for** each collision group $\mathcal{C}$ in coll **do**
8: dec $\leftarrow$ SIC(powers of $\mathcal{C}$, N, γ_{req})
9: **if** SIC is disabled **then** dec $\leftarrow$ first element of dec **end if** ▷ capture only
10: **for** each packet $p \in \mathcal{G}$ **do**
11: collided $\leftarrow$ (p is in a collision group) $\wedge$ ($p \notin$ dec)
12: **skip** p if its sector index is invalid or $P_p <$ sensitivity of its sector
13: log `RxPacket`(..., collided); update counters
14: **if not** collided **then** record node as decoded; queue an ACK if new **end if**

Algorithm 2 SIC as implemented (`packet_analysis.apply_sic`)

Input: powers P_i (W), noise N, γ_{req}, cancellation factor κ

1: $R_i \leftarrow P_i$; cand $\leftarrow$ all packets; dec $\leftarrow [\,]$
2: **while** cand $\neq \emptyset$ **do**
3: $k \leftarrow \arg\max_{i \in \text{cand}} R_i$
4: $\gamma_k \leftarrow R_k / \left(\sum_{j \in \text{cand}, j \neq k} R_j + N\right)$
5: **if** $\gamma_k \geq \gamma_{\text{req}}$ **then**
6: append k to dec; remove k from cand; $R_k \leftarrow (1-\kappa) P_k$ ▷ residual stored, but never summed again
7: **else break**
8: **return** dec

18 (4 ms from the second one, 8 ms from the first) is *not* in it and is decoded as a lone packet. Exercise 10.6 asks you to find the logic.

Design choice 4: capture-only without SIC. With SIC off the base station runs the same routine with $\kappa = 0$ and keeps only the first decoded packet (the strongest, if its SINR passes). So "SIC off" means *capture only*, "SIC on" means *every packet that passes the successive test is kept*.

Design choice 5: subcarriers are perfectly orthogonal. Packets with different `subcarrier_shift` values are placed in different groups and *cannot* collide, as discussed in Chapter 6.

Three things the pipeline does that you might not expect

(1) The cancellation factor is inert. The `apply_sic` docstring explains that $\kappa = 0.9$ leaves 10% of a decoded signal as residual interference. In the implementation (Algorithm 2, line 7) the residual $(1-\kappa)P_k$ is stored, but the decoded packet has just been removed from the candidate set, and interference is summed only over candidates; the residual is never used. We confirmed this by running `apply_sic` on several power sets with $\kappa \in \{0, 0.5, 0.9, 0.99, 1\}$: the decoded sets were *identical in every case*. Amber's "SIC" is therefore *perfect* cancellation; imperfect cancellation (Eq. (10.2) with $\varepsilon > 0$) is not simulated. The fix would be a one-line change that adds the residuals of already-decoded packets to the interference sum.

Table 10.1: Delivery error rate (DER = 1 − decoded/sent) and throughput of the 3-node broadcast scenario over 20 s, mean over 10 seeds (standard deviation of DER in brackets).

Configuration	SIC	DER	Throughput (pkt/s)
Broadcast demo parameters	off	2.57% (0.85)	6.06
Broadcast demo parameters	on	0.00% (0.00)	6.22
SIC-demo parameters	off	2.89% (1.61)	6.41
SIC-demo parameters	on	0.00% (0.00)	6.60
One factor changed from the broadcast demo (SIC off)			
Capacitance 200 µF	off	2.36% (0.71)	6.80
RX slot 15 ms	off	2.55% (1.18)	5.93
3 slots	off	2.12% (1.16)	5.99
Beamwidth 30°	off	2.57% (0.85)	6.06
Sector sensitivity −80 dBm	off	2.57% (0.85)	6.06

(2) A lone packet bypasses the SNR test. The noise power enters *only* the SINR computation of colliding groups. A single packet that passes the sector's sensitivity gate is logged as decoded regardless of its SNR. We injected a packet at −105 dBm (SNR = −17 dB against −88 dBm of noise) and it was decoded.
(3) Sub-sensitivity packets still interfere. The sensitivity filter is applied *after* the SIC decision, so a packet below the sector's sensitivity is dropped from the log but still counted as an interferer for the packets around it.

10.4 Does SIC matter? A controlled experiment

The two shipped broadcast demos differ in *six* ways at once: `enable_sic`, capacitance (300 → 200 µF), RX slot length (10 → 15 ms), slots per frame (2 → 3), sector beamwidth (65° → 30°) and sector sensitivity (−100 → −80 dBm). Their results therefore cannot isolate the effect of SIC. We rebuilt the scenario with a single function and varied the factors one at a time over ten random seeds (script `textbook/code/sic_ablation.py`). Table 10.1 shows the result.

The conclusions are clean. (i) Enabling SIC removes *all* delivery errors, from ≈ 2.6–2.9% to 0.00%, in both parameter sets. (ii) None of the other factors changes DER meaningfully; beamwidth and sector sensitivity have *exactly* the same value because all three nodes sit on the sector's boresight and far above −80 dBm. (iii) SIC raises throughput slightly (≈ +2.6%) simply because the lost packets are recovered. The improvement shown by the shipped SIC demo is thus attributable to SIC itself.

Why SIC rescues exactly these collisions

The three demo nodes arrive at the base station with −45.7, −49.7 and −68.7 dBm. Node 0 against Node 1 alone: gap 4.0 dB, so $\gamma_0 = 4.0\ \text{dB} \geq 3$: capture works for Node 0 and Node 1 is lost, unless SIC is on, when Node 1 is decoded next against only the noise. For all three together, $\gamma_0 = 1/(0.395 + 0.005) = 2.50 = 3.98\ \text{dB}$, again passing, and the remaining two are separated by 19 dB. Without SIC at most *one* packet of any colliding group survives; with SIC *all* do. This near-perfect separation is an artefact of the nodes' very different distances and the perfect cancellation of the previous reality check; with equal-distance nodes SIC would help far less.

10.5 What the threshold model abstracts away

The SINR threshold γ_{req} replaces a lot of physics: the modulation and coding scheme, the receiver's ability to *lock onto a preamble* while another signal is present (capture in real receivers depends on arrival order and synchronisation), the length of the overlap, and the decoding of partially overlapping packets. It is a good *system-level* abstraction when one wants to compare protocols, and a poor basis for predicting the absolute packet-error rate of a particular radio.

Where this lives in the code

Concept	Code
SINR, Eq. (10.1)	`apply_sic` (`packet_analysis.py`)
Noise power	`thermal_noise_watts`
Per-window buffer	`BSBehavior._rx_buffer`, `_buffer_rx`
Grouping and resolution	`BSBehavior._process_rx_buffer`
SIC on/off	`enable_sic` (constructor)
γ_{req}	`required_sinr_db` (default 3)
κ	`cancellation_factor` (default 0.9; *inert*)

10.6 Chapter summary

- The physical model decodes a packet iff its SINR exceeds γ_{req}; capture follows when one packet is stronger by roughly γ_{req}.
- SIC decodes packets in descending power order and removes each; all packets can be decoded if successive powers differ by a factor $\geq 1 + \gamma_{\text{req}}$.
- Amber buffers packets per RX window, groups by subcarrier and a 5 ms greedy proximity window, then applies SIC.
- Verified quirks: the cancellation factor has no effect (SIC is perfect), lone packets skip the SNR test, sub-sensitivity packets still interfere.
- In a controlled experiment SIC cut DER from $\approx$ 2.6–2.9% to 0%; other demo parameter changes had no effect.

Exercises

Exercise 1

Two packets arrive at -62 and -70 dBm with noise -88 dBm and $\gamma_{\text{req}} = 3$ dB. Compute both SINRs under SIC. Would capture alone decode the first? The second?

Exercise 2

Three equal packets at -70 dBm collide. What is the SINR of the first decoded packet? What does SIC do?

Exercise 3

For $\gamma_{\text{req}} = 3$ dB and negligible noise, how much stronger (in dB) must the strongest of four packets be than the weakest if consecutive powers are in a constant ratio and SIC is to decode all four?

Exercise 4

Four packets on one subcarrier arrive at $3, 7, 11, 15$ ms. Apply Algorithm 1: how many collision groups are formed and which packets are in them? How would transitive clustering differ?

Exercise 5

Recompute the example of Figure 10.2 for $\varepsilon = 0.05$ and $\varepsilon = 0.1$ using Eq. (10.2). Which packets are decoded in each case? Compare with what Amber's implementation would report.

11

Protocols: Discovery, Registration and Polling

In this chapter you will learn

- why Ambient IoT protocols are *command-driven*, and the two-phase structure (discovery, then data collection) used by the paper;
- Amber's built-in protocol as a message sequence: `send_id`, random slot, backscattered ID, `ack`, `send_data`;
- the trade-offs between *broadcast* and *unicast* polling, and why a well-scheduled network is usually *energy-limited*, not airtime-limited;
- a mean-field model of registration success versus node density, compared with simulation;
- the problem of *limited visibility*: what a base station cannot know.

11.1 Command-driven networks

Every protocol in this book is *reader-talks-first*, for the three reasons we met in earlier chapters:

1. a passive node cannot afford a receiver that listens all the time, so commands are rare, explicit and short;
2. a node can transmit only by reflecting a carrier, so it should transmit only when the reader says the carrier is on and a slot is open;
3. the reader is the only entity that can coordinate: it has power, a clock and a view of the whole cell.

The reader's repertoire is small: *broadcast a command*, *open reply windows (slots)*, *acknowledge*. A node's repertoire is smaller still: *if I heard a command and my slot is now, reflect my message*.

The paper describes the base stations of its experiment as operating *in two phases*: "a discovery phase, where nodes register by responding with their identifiers in random slots, followed by a data collection phase, where the base station periodically requests sensor readings from each registered node individually by its identifier." The first phase is framed slotted ALOHA (Chapter 9); the second is *polling*.

11.2 Amber's built-in protocol

The default protocol is implemented jointly by the node's `BackscatterModule` and the base station's schedule. It has three commands:

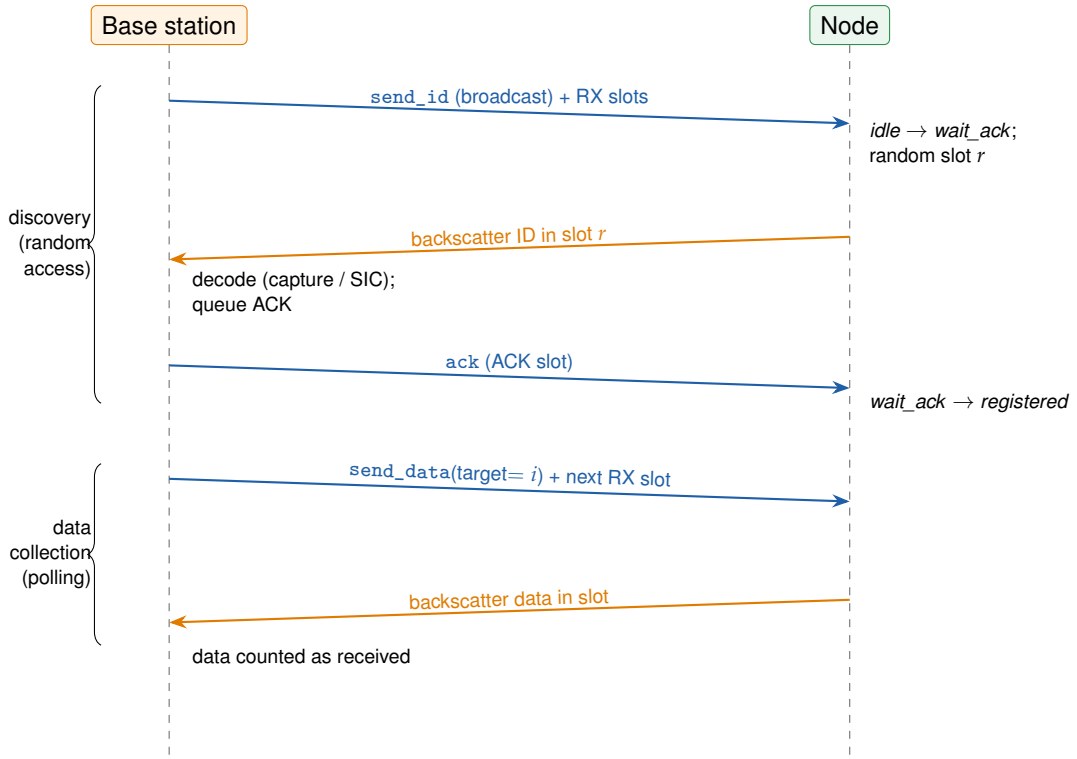


Figure 11.1: Message sequence of Amber's default protocol for one node. The two phases correspond to those of the paper. In each phase the *reply* is a backscatter packet inside an RX window the base station advertised in the command.

Command	Sent by base station	Node reaction
`send_id`	broadcast, with the list of upcoming RX slots	unless already registered: pick a slot at random; state → *wait_ack*
`ack`	in an ACK slot, to every node whose ID was decoded	if in *wait_ack*: state → *registered*
`send_data`	broadcast (`target=-1`) or to one node `target=i`; with RX slots	if registered (and addressed): choose a slot (random for broadcast, the first for unicast)

A node thus lives in three *protocol states*: *idle* (never heard a command), *wait_ack* (has chosen a slot to send its ID) and *registered*.

The sequence in Figure 11.1 has several details to appreciate.

- Commands carry the *timing of future RX windows*, so the node knows when its reply is permitted. The base station obtains this by *looking ahead* in its schedule (Chapter 17).
- An ACK is not itself a radio-simulated packet: it is a direct method call on every node whose ID was decoded. In a real system it would be a downlink message that could be lost.
- A node that is *already registered* ignores `send_id`, which is what lets discovery converge: as more nodes register, fewer contend.

11.3 Broadcast versus unicast polling

Once nodes are registered, the base station collects data in one of two ways.

Broadcast. One command `send_data(target=-1)` followed by S RX slots; each registered node picks a slot at random. This costs one command per frame, but the nodes contend: it is again FSA with n contenders, with all the collisions of Chapter 9.

Unicast. One command per node, `send_data(target=i)`, each followed by the *next* RX slot. Because only node i is allowed to answer, there are *no collisions*. The paper notes exactly this: "No collisions occur during data collection since nodes are queried individually." The price is overhead: N commands per round.

	Broadcast (FSA)	Unicast (polling)
Commands per round	1	N
Collisions	yes, $P_{\text{succ}} \approx e^{-n/S}$	none by construction
Airtime per round	$T_{\text{cmd}} + S\,T_{\text{slot}}$	$N(T_{\text{cmd}} + T_{\text{slot}})$
Needs to know the nodes?	no	yes (IDs from discovery)
Wasted energy on collisions	yes	none

11.3.1 Airtime is rarely the bottleneck

Consider the shipped `unicast` demo: three nodes, each poll is $T_{\text{cmd}} = 5\,\text{ms}$ plus a 10 ms slot, so a round takes 45 ms. In 20 s the base station issues about $20\,001/45 \times 3 \approx 1333$ polls. The measured result is 125 packets decoded: only $\approx 9\%$ **of the polls are answered**. The other 91% went to nodes that were *not ready*: asleep recharging, or in a state other than *wait_slot*. The system is *energy-limited*: throughput is set by how often the capacitors allow a node to transmit (we measured between 0.9 and 2.8 packets per second per node in the demos, Chapter 8), not by how fast the base station can poll.

Energy, not airtime, limits throughput

In a well-scheduled Ambient IoT network without collisions, delivered throughput is

$$\text{throughput} \approx \sum_{\text{nodes}} (\text{rate at which the capacitor permits a transmission}).$$

Making the MAC faster does nothing once it can poll faster than the nodes can recharge. The MAC's job is to avoid *wasting* the scarce transmissions: no collisions, no polls to sleeping nodes if that can be predicted.

11.4 Limited visibility

The base station sees a poll succeed or fail. It does *not* see why. A silent node could be

- asleep (capacitor below V_{low} or still charging);
- awake but in its *listening*/*sensing*/*processing* states, not yet at a slot;
- out of range of the carrier (the command was below its sensitivity);
- involved in a collision;
- or simply broken or absent.

The paper names this directly in its introduction: "limited visibility into the device state". It is the root of most protocol-design difficulties in this field. The three possible responses—(i) *over-provision* (more frames, bigger frames), (ii) *infer* (use past replies to predict when a node will be awake), (iii) *make it observable* (let nodes report an energy indicator)—are all research topics that Amber can be used to explore.

11.5 The discovery phase in detail

11.5.1 What the paper reports

The paper's experiment places up to 100 passive nodes, each assigned one of three subcarrier shifts, in a 500 m × 500 m area served by three base stations (30 dBm, 17 dBi sectors, 924 MHz; nodes with 1 dBi omni antennas in hybrid power mode), within 100 m of the base stations; the node count varies from 5 to 100 with 10 runs per density. Its Figure 2 shows the percentage of nodes successfully registered: reading the box plot, the median falls from roughly 88% at 5 nodes to roughly 35% at 100 nodes. The authors explain this by "a higher collision probability during random access".

11.5.2 A mean-field model

We build a simple model of r discovery rounds, each a frame of S reply slots with K subcarrier shifts. Let n_k be the number of nodes still unregistered at the start of round k and let p_a be the probability that a given unregistered node actually *participates* in a round (it is awake at the right moment and holds a slot inside the current frame). A participating node picks one of the SK (slot, shift) bins uniformly. It succeeds if no other participant picked its bin:

$$P_{\text{succ}}(k) \approx \left(1 - \frac{p_a}{SK}\right)^{n_k - 1}, \qquad n_{k+1} = n_k - n_k\, p_a\, P_{\text{succ}}(k). \tag{11.1}$$

The registered fraction after r rounds is $1 - n_{r+1}/n_1$. This is a *lower bound*: it counts every same-bin pair as a collision, whereas the simulator also allows capture and SIC and treats nodes that transmit more than 5 ms apart as non-colliding.

11.5.3 Re-creating the experiment with Amber

Because the paper does not provide the frame length or the number of discovery rounds, we cannot reproduce its numbers. We can reproduce its *mechanism*. We built a one-base-station version (script `textbook/code/registration_experiment.py`) using Amber's *default* registration protocol (none of the shipped demos exercises it): 30 dBm, 17 dBi, three 65° sectors, 924 MHz; nodes with 1 dBi in hybrid mode with the solar trace, uniformly placed in a 100 m disc, shifts i mod 3, the standard capacitor and thresholds; a 4 s warm-up so all nodes are charged; then four rounds of (5 ms `send_id`, $S = 10$ slots of 8 ms, 5 ms ACK). For every density we ran 10 random seeds, with and without SIC.

Three observations from Figure 11.2.

1. **The trend reproduces the paper's.** Registration falls monotonically with density: after four rounds ≈ 88% at 5 nodes, but only 69% (no SIC) or 80% (SIC) at 100 nodes. More rounds recover more nodes (the paper does not say how many rounds it used).
2. **At 5 nodes only ≈ 44% register in round one—not ≈ 100%.** With five nodes collisions are rare, so something else limits participation. That something is $p_a \approx \frac{1}{2}$: the base station advertises the slots of the *current and the next* frame, and a node that picks one

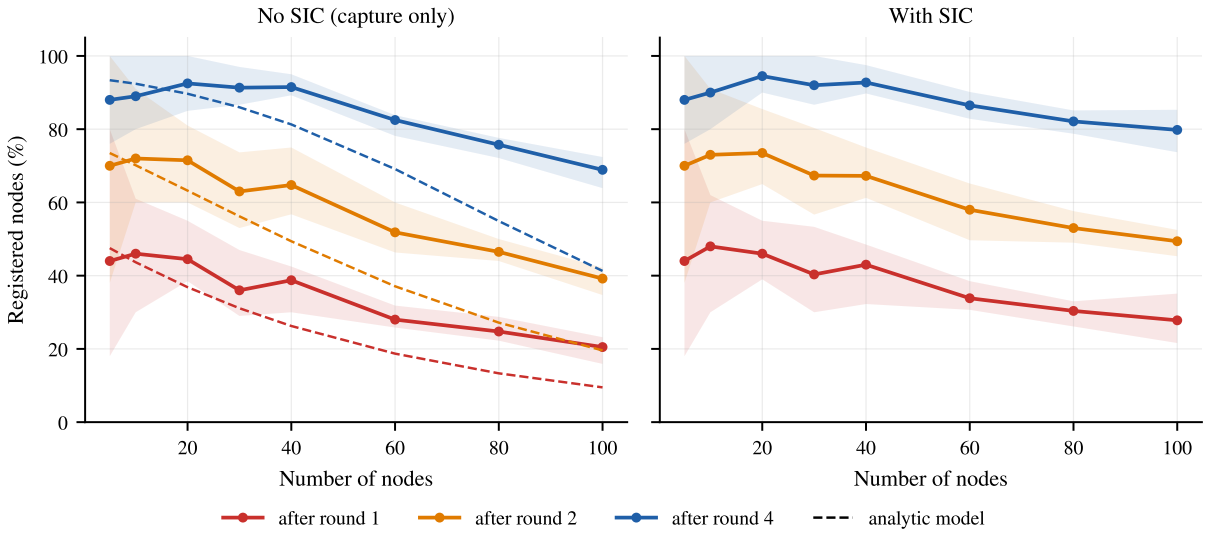


Figure 11.2: Percentage of nodes registered after one, two and four discovery rounds against node count; mean over 10 runs with a 10–90% band; left without SIC, right with SIC. Dashed: the mean-field model of Eq. (11.1) with $p_a = \frac{1}{2}$, $S = 10$, $K = 3$. This is *our* re-creation of the paper's experiment (the paper's frame length and number of rounds are unknown), so only the qualitative trend is comparable.

in the next frame is re-drawn by the next command and effectively sits the round out (Chapter 17). The per-round gain then follows $1 - 0.5^k$: 50%, 75%, 87.5%; we measured 44%, 70%, 82%, 88%.

3. **The model is a lower bound that tracks the shape.** It agrees at low density (47.5% against 44% at $n = 5$, round 1), then falls below the simulation at high density (9.5% against 21–28% at $n = 100$, round 1) because it ignores capture, SIC and the 5 ms rule.

Why even five nodes may not all register in one frame

Observation 2 is not a radio effect at all. It is a consequence of the engine's two-frame RX look-ahead combined with a uniform random choice over the advertised slots. A protocol designer who tunes S by Eq. (9.7) will find that the effective number of slots seen by a node is *the advertised list*, i.e. twice the frame size, of which only half lie in the current frame. We discuss this in detail in Chapter 17.

11.6 Design principles for energy-aware protocols

Putting the pieces together suggests a short checklist, each item testable in Amber.

1. **Don't waste transmissions.** Prefer polls (no collisions) once nodes are known; during discovery, use frames somewhat larger than the expected number of contenders (Section 9.6).
2. **Exploit frequency orthogonality.** Assign shifts to spread nodes; collisions then occur only within a group.
3. **Use the capture/SIC gain.** Near–far diversity is free; do not assume a collision is a loss.
4. **Respect duty cycles.** A node is reachable only for a fraction $D = P_{\text{in}}/P_{\text{act}}$ of the time (Eq. (8.9)); schedule redundancy accordingly.
5. **Adapt.** Estimate the active population from observed outcomes (the custom demo's loop) and resize frames.

11.7 Chapter summary

- Ambient IoT protocols are reader-driven; Amber's default one has three commands (`send_id`, `ack`, `send_data`) and three node states (idle, wait_ack, registered).
- Broadcast polling is cheap in commands but collides; unicast polling never collides but costs N commands. The shipped unicast demo answers only $\approx 9\%$ of polls: it is energy-limited.
- The base station cannot observe node energy states; silent nodes are ambiguous.
- A mean-field FSA model gives a lower bound on registration; our re-creation of the paper's experiment reproduces its downward trend.
- Engine behaviour (two-frame look-ahead) halves per-round participation: $\approx 44\%$ registered after one round even at five nodes.

Where this lives in the code

Concept	Code
Command handling	`BackscatterModule.handle_command`
Node protocol states	`BackscatterModule.state`: `idle`, `wait_ack`, `registered`
ACK delivery	`BSBehavior._send_pending_acks`
Pending / registered lists	`nodes_pending_ack`, `nodes_registered`
Broadcast vs unicast	`target` field: -1, int, or list

Exercises

Exercise 1

Using Eq. (11.1) with $p_a = 1$, $S = 10$, $K = 3$ and $n_1 = 50$, compute the expected registered fraction after one round. Repeat with $p_a = \frac{1}{2}$. By how many rounds does the latter lag?

Exercise 2

A unicast round for N nodes takes $N(T_{\text{cmd}} + T_{\text{slot}})$. For $T_{\text{cmd}} = 5$ ms, $T_{\text{slot}} = 10$ ms and $N = 100$, how long is a round? If each node can transmit at most once every 2 s on average, what fraction of polls will be answered?

Exercise 3

Argue why unicast polling may nonetheless be preferable to broadcast even though it needs N commands. Consider energy per delivered packet (Section 9.6).

Exercise 4

List three observable quantities from which a base station could infer that a silent node is asleep rather than out of range, and say what each would cost the node.

Part IV

Simulation Methodology and Architecture

12

Discrete-Event Simulation and SimPy

In this chapter you will learn

- what a simulation model is, and the difference between time-stepped and event-driven simulation;
- the SimPy concepts Amber uses: processes as generators, timeouts, and *interrupts* as message passing;
- the rule that decides the order of events at the same instant, verified by experiment;
- why Amber pre-computes the radio channel, the measured cost per node and millisecond, and what limits scalability;
- how to think about randomness, replication, verification and validation.

12.1 Models, state and time

A simulation is an *executable abstraction*. Its anatomy is always the same: a set of *state variables* (a capacitor voltage, a node's protocol state, a buffer of packets), a notion of *time*, and *rules* that change the state as time advances. The central design question is how time advances.

Time-stepped simulation. Advance the clock by a fixed Δt; at each step update every entity. This is simple and natural for continuous dynamics such as a capacitor ($C\,\dot{V} = \dots$ integrated with step Δt). Its cost is proportional to (simulated time)$/\Delta t$ whether or not anything happens.

Event-driven (discrete-event) simulation. Keep a *future-event list* ordered by time. Repeatedly remove the earliest event, set the clock to its time, execute it (which may schedule new events) and continue [6, 13]. Nothing is computed between events, so a mostly idle system is cheap and an event's time can be arbitrary (not on a grid).

	Time-stepped	Event-driven
Clock advances by	fixed Δt	time of the next event
Natural for	continuous dynamics	sparse, asynchronous activity
Cost	$\propto T/\Delta t$	$\propto$ number of events
Ordering at ties	implicit (loop order)	explicit rules

Amber is a hybrid

The *base station* is event-driven: it sleeps until a slot ends or a node interrupts it. The *node* is time-stepped: every node's controller schedules an event exactly 1 ms ahead, forever, because the capacitor must be integrated at a fixed step and thresholds must be checked continuously. Consequently one controller wake-up per node per millisecond is the *floor* of

the simulator's cost: idle milliseconds cannot be skipped.

12.2 SimPy in ten minutes

Amber is built on SimPy [21], a process-based discrete-event framework for Python. Its ideas fit on one page.

- An `Environment` holds the clock (`env.now`) and the future-event list.
- A *process* is a Python *generator function*. It runs until it hits a `yield`, which hands an *event* to the scheduler; the process is suspended until that event occurs.
- `yieldenv.timeout(d)` means "wake me up d time units from now". Amber's time unit is the millisecond.
- `env.process(gen)` starts a process; `env.run(until=T)` runs the clock to T.
- `proc.interrupt(cause)` *throws* a `simpy.Interrupt` exception into the target process at its current `yield`. The target can catch it, read `cause` (any object, in Amber a string), and carry on. This is how processes send each other messages.

A complete toy example: a charger adds one unit per millisecond; a supervisor interrupts it after 5 ms.

```
import simpy

def charger(env, cap):
    while True:
        try:
            yield env.timeout(1)          # wait 1 ms
            cap["q"] += 1                 # then add charge
        except simpy.Interrupt as it:     # a message arrives
            print(f"t={env.now}: charger interrupted ({it.cause})")
            return

def supervisor(env, proc):
    yield env.timeout(5)
    proc.interrupt("enough")              # send a message

env = simpy.Environment(); cap = {"q": 0}
p = env.process(charger(env, cap))
env.process(supervisor(env, p))
env.run(until=20)
print("final charge:", cap["q"])
```

Running it prints

```
t=5: charger interrupted (enough)
final charge: 4
```

Notice that the charge is 4, not 5. At $t = 5$ both the charger's own timeout and the supervisor's wake-up are due, yet the interrupt *pre-empts* the charger's increment. This is not an accident, as we show next.

12.2.1 Order of events at the same instant

SimPy keeps events ordered by (time, priority, insertion counter). Ordinary events have priority `NORMAL` (1); *interrupts are scheduled with priority* `URGENT` (0). We verified experimentally that an

interrupt issued at time t is handled *at time* t, immediately after the issuing process yields and before any other ordinary event at t. Three consequences are central to Amber:

1. When the controller interrupts the capacitor, the capacitor's voltage update happens *within the same millisecond*, before anything else at that instant; the controller's next reading (one millisecond later) sees the updated value.
2. When a node's `do_transmit` interrupts the base station, the base station buffers that packet *before* any other node's controller runs. The base station's "scratch" attributes (`received_tag_id`, `data_from_tag`, ...) therefore act as a safe one-element mailbox, even if two nodes transmit in the same millisecond.
3. Among *ordinary* events at one instant the order is the order of scheduling, which is the order in which processes were created. Results can in principle depend on that order; Amber's controllers all wake at the same instants, so the order in which nodes are created is the order in which they run.

12.3 How Amber uses SimPy

Three kinds of process run concurrently.

Process	Wakes on	Does
`Controller`	timeout of 1 ms, always	recompute harvest power; apply thresholds; advance the node FSM; interrupt its capacitor
`Capacitor`	interrupt from its controller (idle timeout of 10 ms otherwise)	apply one Δt step selected by the interrupt's cause
`BSBehavior`	timeout at the end of each TX/RX slot; interrupt `"receive_data"` from nodes	run the schedule; buffer packets during RX; resolve collisions at the end of the window
`EnvEnergySource`	timeout of 1 ms	move to the next row of the solar trace

Inter-process communication uses two idioms: **interrupts with a string cause** as a signal (`"capacitor_charging"`, `"receive_data"`), and **shared attributes** as the payload (`capacitor.current`, `capacitor.voltage_source`, `bs.rssi_dbm`). The base station additionally calls node methods *directly* (`receive_command`); that call is synchronous and instantaneous.

12.4 Precomputation and scalability

The paper states that "since nodes and base stations are stationary, coverage maps and link budgets are pre-computed once before execution to minimize runtime overhead." With static geometry the downlink power P_{dl} of every node and the uplink power to every sector are constants. Computing them once costs $O(N \cdot B \cdot S_B)$ (nodes $\times$ base stations $\times$ sectors) and is vectorised with NumPy. In our measurement the pre-computation for 300 nodes took 3.4 ms.

What remains at run time is the per-millisecond controller step. We measured the wall-clock cost of Amber with a fixed number of nodes over 5 s of simulated time (hybrid nodes, idle base station) on a 4-core cloud container:

Nodes	Pre-computation	Run time (5 s sim.)	µs per node-ms
10	0.9 ms	0.57 s	11.5
30	0.8 ms	1.87 s	12.5
100	1.4 ms	6.29 s	12.6
300	3.4 ms	21.3 s	14.2

The cost is *linear* in the number of nodes at about 12 µs − −14 µs per node-millisecond, so a 60 s run with 1000 nodes takes roughly $1000 \times 60\,000 \times 13\,\mu s \approx 13$ minutes. This supports the paper's claim of "efficient simulation of hundreds of nodes" (about four times slower than real time for 300 nodes in our measurement; your machine will differ).

Memory grows with time unless you ask it not to

Every capacitor appends a `(time,voltage)` tuple to `voltage_history` *every millisecond* while `keep_logs=True` (the default), and the base station and backscatter modules store every packet. For long runs pass `keep_logs=False` (controller, capacitor, backscatter, base station) and use the base station's `stream_log` option, which writes packets to disk as they arrive; counters (`total_rx`, `total_collided`, ...) are always maintained at constant memory.

12.5 Randomness and replication

Several mechanisms in Amber are random: the start-up delay of each controller (0–2 s, uniform), the sensed value, the slot chosen by a node, and node placement in experiments. They all use Python's global `random` module, *unseeded by default*. Only the custom-protocol demo calls `random.seed`. Therefore two runs of, say, the broadcast demo differ slightly (we measured 122 and 121 packets sent in two runs; over ten seeds the mean is 124.4 with standard deviation 1.6). Good practice, followed by the paper's experiment:

- treat a single run as one sample; *replicate* (10 runs per density in the paper) and report distributions (box plots) or means with confidence intervals;
- `random.seed(s)` for reproducibility of any one run;
- vary only one factor at a time when attributing an effect (our SIC ablation, Section 10.4).

12.6 Verification, validation and what Amber has had

Verification asks *"Did we build the model right?"* (code implements the intended equations). Throughout this book we verified Amber against closed-form results: the exact RC solution (agreement to 2×10^{-4} V), hand-computed link budgets (exact to two decimals), SIC arithmetic, and ALOHA-based expectations for registration.

Validation asks *"Did we build the right model?"* (it represents reality well enough for its purpose), usually by comparison with measurements. The paper is a demo abstract and presents no hardware comparison; the radio models are standard (Friis, 3GPP UMa) but the harvester, capacitor and node loads are idealisations. Treat Amber's *relative* comparisons (protocol A versus B under identical assumptions) as more trustworthy than its *absolute* numbers.

Key idea

A simulator is a *microscope for mechanisms*, not a crystal ball. Use it to see *why* a protocol behaves as it does (energy gating, collision structure, capture), then confirm key numbers experimentally.

12.7 Chapter summary

- Time-stepped simulation suits continuous dynamics; event-driven suits sparse events; Amber combines both: an event-driven base station and 1 ms time-stepped nodes.
- In SimPy a process is a generator, `timeout` suspends it and `interrupt(cause)` sends it a message; interrupts are *urgent*, so they are handled at the same instant before ordinary events.
- Static geometry lets Amber pre-compute all link budgets; run time is linear in the number of nodes at $\approx 13\,\mu s$ per node-ms.
- Randomness is unseeded by default; replicate and report distributions.
- Amber is verified against analytic cases in this book but not validated against hardware in the paper.

Exercises

Exercise 1

Modify the toy example so that the supervisor interrupts at $t = 5.5$ (use floating-point time). What is the final charge, and why?

Exercise 2

Explain why an event-driven simulator cannot skip idle milliseconds in Amber, and how you would restructure the node model so that it could. What accuracy would you lose?

Exercise 3

Using 13 µs per node-ms, estimate the run time for 500 nodes over 120 s. How many replications of a 100-node, 20 s experiment fit in one hour?

Exercise 4

Two nodes transmit in the same millisecond. Describe, step by step, how SimPy's ordering rules ensure that the base station buffers *both* packets rather than overwriting the first.

Exercise 5

Why would you not report a protocol's performance from a single unseeded run? Design a minimal replication plan for the node-density experiment of the paper.

13

The Architecture of Amber

In this chapter you will learn

- the runtime object graph of an Amber scenario and who calls whom;
- the "interface contracts" between modules (duck typing), and the extension points that let you swap a capacitor, a protocol or a policy;
- the data dictionaries that carry pre-computed channel state;
- the eight-step recipe that every demo follows to wire a scenario;
- what happens in a single simulated millisecond.

13.1 Objects at run time

Figure 2.3 showed the *import* structure. A running scenario has a different shape: for every node a trio of cooperating objects, and shared channel state.

Who owns time? Each of `Controller`, `Capacitor`, `BSBehavior` and `EnvEnergySource` registers one SimPy process in its constructor (`self.action=env.process(self.run())`). The `Node`, `Sector`, `BaseStation` are plain *data records* (Python dataclasses); the `CoverageMap` is a calculator, not a process. The `BackscatterModule` has *no process of its own*: it is passive logic that other processes call.

13.2 Interface contracts

Amber uses *duck typing*: modules interact through a handful of attributes and methods rather than formal base classes. Knowing the contracts is knowing how to replace a module.

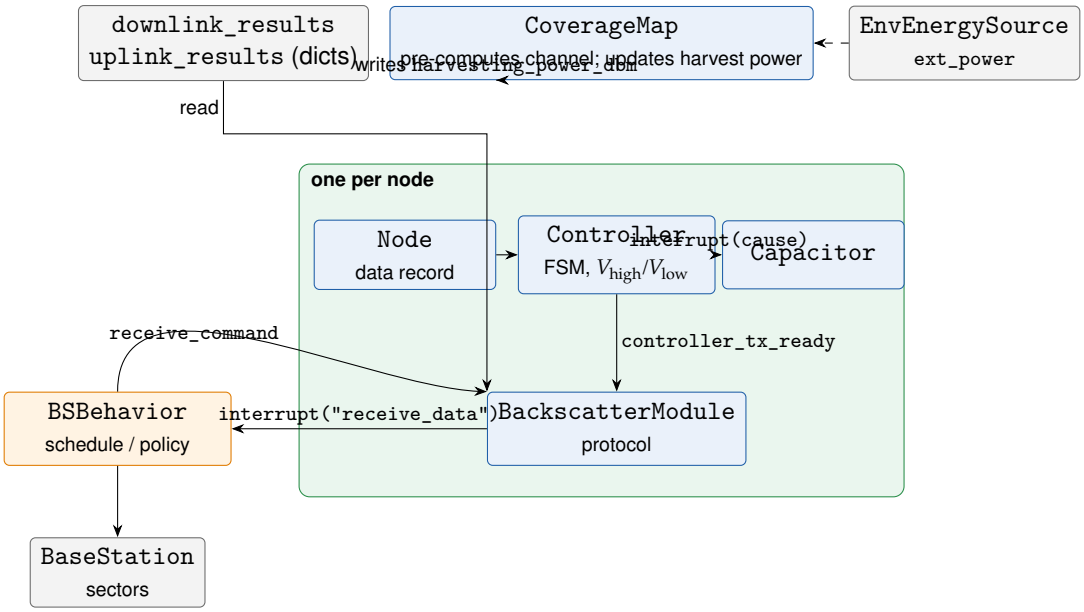


Figure 13.1: Runtime object graph. Solid arrows are calls or attribute reads; the arrows labelled `interrupt` are SimPy messages. The coverage map is shared by all nodes; the `Capacitor`, `Controller` and `BackscatterModule` are instantiated once per node; a `BSBehavior` exists per base station.

Consumer	Needs from…	Provided by default
Controller	*capacitor*: `action.interrupt(cause)` for causes `capacitor_charging`, `capacitor_full`, `capacitor_charge_and_discharge`; writable `current` (A) and `voltage_source` (V); readable `voltage`, `internal_time` (s)	`Capacitor`
Controller	*backscatter*: `state`; `controller_tx_ready(data)` returning a dict or `None`; `do_transmit(tx_info)`	`BackscatterModule`
Controller	*node*: `id`, writable `state`, `harvesting_power_dbm` (set by the coverage map)	`Node`
BackscatterModule	*base-station process*: attributes `received_tag_id`, `data_from_tag`, `rssi_dbm`, `best_sector_idx`, `subcarrier_shift`, and `action.interrupt("receive_data")`	`BSBehavior`
BSBehavior	*modules*: `node.id`, `receive_command(cmd,bs_id,data)`	`BackscatterModule`
CoverageMap	*nodes*: `id`, `x`, `y`, `height`, `efficiency`, `antenna_*`, `state`; *base stations*: `id`, `x`, `y`, `sectors`	dataclasses

Extension points. Three seams are designed for user code:

1. **Node protocol:** subclass `BackscatterModule` and override `handle_command` (what to do on a command) and `build_tx_payload` (what to send in your slot). Everything else—the sensitivity gate, logging, slot validation, uplink delivery—stays.
2. **Base-station behaviour:** either give `BSBehavior` a static `schedule` list, or a `policy` generator that yields one frame at a time and can adapt to `decoded_this_frame`, `total_collided`, etc.
3. **Energy models:** supply any object with the capacitor contract, a different `node_ext_power_fn`, or another trace to `EnvEnergySource`.

Chapter 20 demonstrates (1) and (2) with the adaptive-ALOHA demo.

13.3 Data dictionaries

Channel state is passed around as plain dictionaries produced once by the `CoverageMap`.

Downlink results (`compute_bs_to_point(nodes)`), keyed by node id:

Key	Content
`owner`	name of the best serving sector, e.g. `"BS0_S0"`
`best_pw_dbm`, `best_pw_w`	strongest received power over all base stations and sectors
`total_pw_dbm`	power summed over all sectors (interference-like measure)
`per_node_powers`	{node id : {sector name : dBm}}, every sector

Uplink results (`compute_point_to_bs(nodes)`), keyed by sector name: `per_sector_powers[sector][node_id]` (dBm; the power that sector would receive from that node), plus per-sector `owner` (the strongest node), `best_pw_dbm`, `best_pw_w`, `total_pw_dbm`.

Per-node attributes written by the coverage map : `harvesting_power_dbm` and `backscatter_power_dbm`. These are *not* fields of the `Node` dataclass; they are attached dynamically, so a controller will fail at its first step with an `AttributeError` unless `calculate_node_power` has been called for its node beforehand (as every demo does at set-up).

13.4 The wiring recipe

All four demos, and the paper's experiment, follow the same eight steps. Learn them once.

Step 1. Define the topology. Create `Nodes` (position, height, antenna, sensitivity, efficiency, subcarrier shift) and `BaseStations` with `Sectors` (azimuth, beamwidth, power, gain, sensitivity).

Step 2. Create the environment and energy source. `env=simpy.Environment()`; optionally an `EnvEnergySource` reading the solar trace with `resistance=R_SERIES`.

Step 3. Build one `CoverageMap` per energy mode. `node_energy_mode` is per map, so WPT-only and hybrid nodes use different maps (the demos do so).

Step 4. Pre-compute the channel. `dl=cov.compute_bs_to_point(nodes)`; `cov.calculate_node_power(nodes,dl)`; `ul=cov.compute_point_to_bs(nodes)`. (Order matters: the uplink needs the backscatter power that `calculate_node_power` writes.)

Step 5. Instantiate the per-node trio. `Capacitor(env,id,params,initial_voltage)`; `BackscatterModule(env,node,bs_processes=[],uplink_results=ul,downlink_results=dl)`; `Controller(env,capacitor_ctrl,node,backscatter,params,coverage_map=cov,downlink_results=dl)`.

Step 6. Instantiate the base station behaviour. `BSBehavior(env,base_station,schedule=...orpolicy=...,backscatter_modules=[...],enable_sic=...)`.

Step 7. Close the loop. Set every module's `bs_processes=[bs_behavior]` (the module needs the process to interrupt, the process needs the modules to command; neither exists when the other is built). For demos that skip discovery: `bsm.state="registered"` and `bs.nodes_registered=[...]`.

Step 8. Run and read counters. `env.run(until=T_ms)`; then use `bs.rx_packets`, `bsm.tx_records`, `bs.total_rx`, `bs.total_collided`, ...

Forgetting Step 7 produces the quietest of failures: a node whose `bs_processes` is empty "transmits" (it logs a TX record with `delivered=False`) but no base station ever hears it.

13.5 One simulated millisecond

Figure 13.1 shows who talks to whom; this table shows the *order* within a single millisecond for one node (Chapter 15 gives the details of each step).

#	Actor	Action
1	Controller wakes	publishes its state name on the node (`node.state`)
2	Controller	`coverage_map.calculate_node_power([node],downlink_results)`: new P_h (depends on `node.state` and the solar trace)
3	Controller	$V_{\text{src}} = \sqrt{P_h R_s}$ written to `capacitor.voltage_source`
4	Controller	reads V (the value left by the *previous* millisecond's update); applies the hysteresis rule; runs `working_mode()`, one FSM step, which sets `capacitor.current` and calls `capacitor.action.interrupt(cause)`
5	Controller	in state *wait_slot*: `controller_tx_ready`; if ready, `do_transmit`, which sets the base station's scratch fields and interrupts it
6	Controller	`yieldenv.timeout(1)`: suspends
7	Capacitor (urgent, same ms)	applies one Δt step and appends to its history
8	Base station (urgent, same ms)	if in an RX window, buffers the packet

Steps 7 and 8 are SimPy *urgent* events: they run at the same simulated instant, but only after the controller has yielded, and in the order in which the interrupts were issued. At the *end* of each RX window the base station processes its buffer (Algorithm 1); at the *start* of each TX window it calls the target modules' `receive_command`, a synchronous call.

Where this lives in the code

Concept	Where
`Per-node trio`	constructed in every demo's `run_simulation()`
`Node protocol seam`	`BackscatterModule.handle_command`, `.build_tx_payload`
`BS policy seam`	`BSBehavior(policy=...)`, `_run_policy`
`Channel dictionaries`	`CoverageMap.compute_*`
`Time-owning objects`	`Controller`, `Capacitor`, `BSBehavior`, `EnvEnergySource`

13.6 Chapter summary

- Four classes own SimPy processes; `BackscatterModule` and `CoverageMap` are passive helpers; `Node`, `Sector`, `BaseStation` are data.
- Modules interact by duck-typed contracts: a capacitor must accept three interrupt causes and expose `voltage`, `current`, `voltage_source`; a protocol module must provide `state`, `controller_tx_ready`, `do_transmit`.
- Channel state lives in two dictionaries computed once; harvest and backscatter power are per-node attributes attached by the coverage map.
- Every scenario follows the same eight wiring steps; the commonest mistake is not attaching `bs_processes`.

Exercises

Exercise 1

Sketch the sequence diagram (UML style) of a unicast poll between `BSBehavior`, `BackscatterModule`, `Controller` and `BSBehavior`'s buffer, using the objects of Figure 13.1.

Exercise 2

You want to replace the capacitor by a supercapacitor with an equivalent series resistance and a voltage-dependent leakage. Which attributes and interrupt causes must your class provide? What else would need to change?

Exercise 3

Why must the uplink computation follow `calculate_node_power`? What would the base station receive if it were computed first?

Exercise 4

List the objects you would create for a scenario of two base stations of three sectors and 50 nodes, and count the SimPy processes that exist.

Part V

Inside the Simulator

14

Topology and the Coverage Engine

In this chapter you will learn

- every parameter of `Node`, `Sector` and `BaseStation`, with its unit, default and role;
- what the coverage map computes, and which of its outputs the *simulation* actually uses;
- the algorithms for downlink, harvest/backscatter bookkeeping and uplink, step by step;
- how multiple base stations interact (and where the model simplifies);
- numerical conventions: sentinels, floors and the meaning of "owner".

14.1 The topology objects

Three small dataclasses in `radiodevices.py` describe the geometry and radio hardware. Coordinates are in metres on a flat plane (x east, y north); heights are separate.

14.1.1 `Node`

Field	Default	Unit	Role
`id`	`required`	–	identifier used in all dictionaries and logs
`x, y`	`required`	m	position
`height`	`1.5`	m	h_{UT} in UMa path loss
`antenna_type`	`"omni"`	–	`"omni"` or `"3GPP"`
`antenna_gain_dbi`	`0.0`	dBi	peak gain (omni: constant)
`azimuth_deg, beamwidth_deg`	`0, 360`	deg (compass)	only for 3GPP node antennas
`efficiency`	`1.0`	ratio	η: fraction of received power reflected
`sensitivity_dbm`	-45	dBm	weakest downlink command the node notices
`subcarrier_shift`	`0`	integer	frequency-domain group for uplink
`node_type`	`"passive"`	–	cosmetic (plot colour)
`radius, color, label`	`2, "", True`	plot	drawing only
`state`	`"listening"`	–	written by the `Controller` each ms; *not* a constructor argument

14.1.2 `Sector` and `BaseStation`

Field	Default	Unit	Role
Sector			
`azimuth_deg`	`required`	deg (compass)	boresight
`beamwidth_deg`	`required`	deg	$\varphi_{3\text{dB}}$ of the 3GPP pattern
`power`	`43.0`	dBm	transmit power (the paper uses 30)
`antenna_gain_dbi`	`17.0`	dBi	$G_{\max}$
`antenna_type`	`"3GPP"`	–	or `"omni"` (0 dBi, ignores gain field)
`height`	`25.0`	m	h_{BS}
`sensitivity_dbm`	-100	dBm	weakest backscatter packet logged
BaseStation			
`id, x, y`	`required`	m	identity and position of the mast
`sectors`	`[]`	list	any number of sectors

A subtle asymmetry. An `"omni"` *node* uses its `antenna_gain_dbi`; an `"omni"` *sector* is forced to 0 dBi (`antenna_from_sector` ignores the gain field). Use `"3GPP"` with a wide beamwidth if you want a high-gain wide beam.

14.2 The CoverageMap

`CoverageMap` is a configuration object plus four calculators.

Parameter	Default	Meaning
`base_stations, nodes`	`None`	the topology
`freq_hz`	`925e6`	carrier (demos: 924e6)
`pathloss_model`	`"macro"`	`"fspl"` or `"macro"`
`los`	`True`	LOS or NLOS branch of UMa
`extra_np_per_m`	`0`	medium attenuation (power, Np/m)
`node_energy_mode`	`"wpt"`	`"wpt"`, `"external"`, `"hybrid"`
`node_ext_power_fn`	`None`	callback node → watts
`combine_mode`	`"max"`	hybrid: `"max"` or sum
`bandwidth_hz, noise_figure_db`	`100e6, 6`	*declared "for SNR" but never used in this class*

14.2.1 The coverage grid (for pictures)

`compute_coverage_map(xmin,xmax,ymin,ymax,step)` evaluates, on a regular grid, the received power from every sector of every base station (receiver antenna 0 dBi, receiver height 1.5 m) and stores the *maximum* over sectors (`best_pw_dbm`), the *sum* (`total_pw_dbm`) and the name of the best sector (`owner`). `plot_rssi` draws it. The paper's Figure 1 is this picture. Crucially, **the simulation does not use the grid**; it is a diagnostic, which is why the demos' coverage maps are computed but never read at run time.

14.2.2 Downlink to the nodes

`compute_bs_to_point(nodes)` is the workhorse. For each base station, sector and node:

1. horizontal distance d_{2D} from the mast to the node;

2. bearing from the mast to the node, angular error to the sector's boresight, sector gain $G_s(\varphi)$ (Chapter 4);
3. bearing from the node back to the mast, angular error to the node's own boresight, node gain $G_n(\varphi')$;
4. received power $P = P_{\text{tx}}\, G_s\, L(d)\, e^{-\alpha d}\, G_n$ with h_{BS} from the sector and h_{UT} from the node (Chapter 5);
5. bookkeeping: store each sector's power in `per_node_powers`; keep the running maximum (`best_pw_w`, with the winning sector name as `owner`) and the sum (`total_pw_dbm`).

The calculation is vectorised over nodes with NumPy and looped over sectors.

"Best" means maximum, not sum

The power a node harvests is the *strongest single sector's* power, not the sum of all carriers reaching it. Real carriers from several sectors at the same frequency would add (incoherently); Amber ignores them. This is conservative for energy but also means *there is no inter-cell interference in the downlink.*

14.2.3 Harvest and backscatter power

`calculate_node_power(nodes,downlink_results)` converts the best downlink power into each node's `harvesting_power_dbm` and `backscatter_power_dbm` following the table of Section 7.3 (three energy modes). It is called *once* during set-up, and thereafter by each node's controller *every millisecond* with nodes = [itself], because the harvest depends on the node's state name and on the solar trace.

14.2.4 Uplink to the base stations

`compute_point_to_bs(nodes)` takes each node's `backscatter_power_dbm` (**evaluated at set-up, with the node in its initial state**) as a transmit power and propagates it to every sector: node gain toward the mast, path loss, sector receive gain. The result `per_sector_powers[sector][node]` is the RSSI that sector would observe. The aggregates (`owner`, `best_pw_dbm`, `total_pw_dbm`) are per sector.

14.2.5 What this means for multiple base stations

Three consequences of the dictionaries' design:

1. **Every base station can hear every node.** `per_sector_powers` contains an entry for each (sector, node) pair; no entry is dropped for being weak. When a node transmits, `do_transmit` interrupts *each* base station in its `bs_processes`, using the strongest sector of that base station. Whether a packet is *logged* is decided afterwards by each sector's sensitivity. A cell-edge node is thus heard by several base stations at once (macro-diversity), and the paper's `nodes_multi_bs` statistic counts exactly this.
2. **Command reception ignores which base station spoke.** A node accepts a command if its *best* downlink power over *all* base stations exceeds its sensitivity, whichever base station issued the command.
3. **Static link budgets.** Nothing changes with time except the harvest; in particular the uplink RSSI is a constant for each (sector, node) pair.

14.3 Numerical conventions

- **Floor at −120 dBm**: `watts_to_dbm` clamps at 10^{-15} W; a node with zero harvest has −120 dBm and `dbm_to_watts` returns 10^{-15}, not zero.
- **Sentinel −999 dBm**: used when a node has no recorded power (e.g. `received_power=best_pw_dbm.get(id,-999.0)`); it ensures commands are rejected and uplinks vanish rather than crash.
- **Tie-breaking**: the best sector is chosen by a strict ">", so ties go to the earlier-evaluated sector (first base station, lowest sector index).
- **Sector names** are strings `BS<id>_S<index>`. The node module finds a base station's sectors by prefix matching on `BS<id>_` and recovers the sector index by splitting on `_S`; use integer base-station ids and let sectors be numbered by list order.

Where the coverage engine differs from physics

(i) No inter-cell interference or carrier summation ("max" not "sum"); (ii) the uplink assumes the node reflects its *initial-state* backscatter power at all times; (iii) the path loss near the mast has the artefacts of Chapter 5; (iv) `bandwidth_hz` and `noise_figure_db` on the map are not used (noise is computed in the base station from its own arguments, Chapter 17); (v) static geometry and no shadowing.

Where this lives in the code

Concept	Function (`propagation.py`)
Grid for pictures	`compute_coverage_map`, `plot_rssi`
Downlink to nodes	`compute_bs_to_point`
Harvest/backscatter	`calculate_node_power`
Uplink to sectors	`compute_point_to_bs`
Device drawing	`_draw_radio_devices`

14.4 Chapter summary

- Three dataclasses describe the hardware; units are metres, degrees (compass), dBm and dBi.
- The coverage grid is for plotting; the simulation uses `compute_bs_to_point`, `calculate_node_power` and `compute_point_to_bs`.
- A node harvests the *best sector's* power; every base station hears every node, gated by sector sensitivity.
- The uplink is computed once and uses the node's initial-state backscatter power.

Exercises

Exercise 1

For a node at $(60, 80)$ and a base station at the origin with sectors at compass azimuths 0° and 120° (65°, $G_{\max} = 15$ dBi, 46 dBm, $h_{BS} = 25$ m), which sector is the owner? Give the angular errors and the downlink power with UMa LOS and a 0 dBi node.

Exercise 2

Why does `compute_point_to_bs` not need the base station's transmit power? What does it need from a previous call?

Exercise 3

A node lies equidistant from two base stations with identical sectors and both can hear it. Describe, with reference to the data structures, what the node's transmission triggers in each base station.

Exercise 4

Modify the harvest model (conceptually) so that a node harvests the *sum* of all sectors' powers. Which function changes, and what data does it already have?

15

The Node Controller

In this chapter you will learn

- the controller's per-millisecond loop, as an algorithm;
- the five states of the working-mode state machine, what each costs in charge and energy, and how states are chained;
- the start-up behaviour and the activation/shut-down hysteresis, from the code's point of view;
- the *wait_slot* state and its idealisations;
- the "delivered-before-paid" behaviour, measured: nearly half of the transmit states in the demo end in brown-out.

The `Controller` is the heart of a node. It owns the node's simulated *behaviour*: it turns the capacitor voltage into duty-cycled activity (Chapter 8), and activity into energy consumption and radio transmissions. Every node has one; each is a SimPy process that wakes up every millisecond.

15.1 Configuration

`ControllerParams` bundles three frozen dataclasses (plus a maximum start-up time):

State	Duration d (ms)	Current I (mA)	Charge $d\,I$ (µC)	Energy at 1.5 V (µJ)
listening	5	0.14	0.70	1.05
sensing	2	0.512	1.02	1.54
processing	5	1.28	6.40	9.60
transmitting	15	5.0	75.0	112.5
cycle	27	–	83.1	124.7

(demo values; the last column assumes a constant 1.5 V). Transmitting is 90% of the energy of a cycle. Thresholds are $V_{\text{high}} = 1.7$, $V_{\text{low}} = 1.3$ V. `max_startup_time_ms` (default 2000) bounds a random start-up delay.

These demo loads are far from microwatt tags

A transmit current of 5 mA at 1.5 V is 7.5 mW, and the cycle averages 4.6 mW. These are the electrical currents of a microcontroller-and-radio sensor node, three to four orders of magnitude above the ~ 1 µW class that 3GPP's first Ambient IoT release targets (Chapter 1). They are parameters: nothing in the model prevents you from entering 1 µA-class numbers, but the *shipped demos* should not be read as representative of Ambient-IoT-class power.

Algorithm 3 Controller main loop (every millisecond)

1: **each ms:** publish `node.state`← `state_name`
2: recompute harvest: `calculate_node_power([node],downlink_results)` ▷ only if a map and results exist
3: $V_{\text{src}} \leftarrow \sqrt{P_h \cdot R_s}$, write to `capacitor.voltage_source`
4: **if** `capacitor.internal_time` $\leq$ `startup_delay` **then**
5: interrupt capacitor with `charging`
6: **else**
7: $v \leftarrow$ `capacitor.voltage`
8: **if** $v \geq V_{\text{high}}$ **and not** active **then**
9: interrupt `full` (leak only); state← listening; counter← 0; active← true
10: **else if** not active **then**
11: interrupt `charging`; state← listening
12: **else if** $v > V_{\text{low}}$ **then**
13: WORKINGMODE ▷ one FSM step; sets load and interrupts
14: **else** ▷ $v \leq V_{\text{low}}$ and active
15: active← false; record FSM cycles of this charge cycle; interrupt `charging`
16: **wait** 1 ms

15.2 The main loop

Algorithm 3 is the `Controller.run` generator in plain words.

A few observations from the algorithm.

- The **start-up delay** (uniform in $[0, 2000)$ ms, drawn once per node) is measured on the *capacitor's* own clock. During it the node charges even if V already exceeds V_{high}. Its purpose is to de-synchronise nodes that would otherwise all become active at the same simulated instant.
- **Activation** requires $v \geq V_{\text{high}}$ *and* the delay to have elapsed. At that moment the controller sends one *leak-only* step (`capacitor_full`), resets its state machine to *listening* and sets the node active.
- **Shut-down** happens at the first millisecond where $v \leq V_{\text{low}}$, *wherever the state machine happens to be*.
- The voltage that the controller reads at step k is the one produced by the capacitor update of step $k-1$ (Section 13.5); the one-millisecond lag is harmless.
- The harvest power is recomputed in *every* step: it changes when the node enters the *transmitting* state (the $(1-\eta)$ rule of Chapter 6) and with the solar trace.

The series resistance is not read from the capacitor

Line 3 converts power to voltage using `getattr(capacitor_ctrl,"R_series",5e3)`. But `Capacitor` stores its resistance inside `params` (`capacitor.params.R_series`); it has no attribute `R_series`. The `getattr` therefore *always returns the default* 5 kΩ, whatever `CapacitorParams.R_series` says. We verified this with a capacitor configured for 1 kΩ: `hasattr(cap,"R_series")` is false and the lookup returns 5000. In the shipped demos both values are 5 kΩ, so nothing goes wrong; if you change R_s in `CapacitorParams` without also changing the constant in `Controller.run`, the equivalent source voltage and the capacitor's charging resistance will silently disagree.

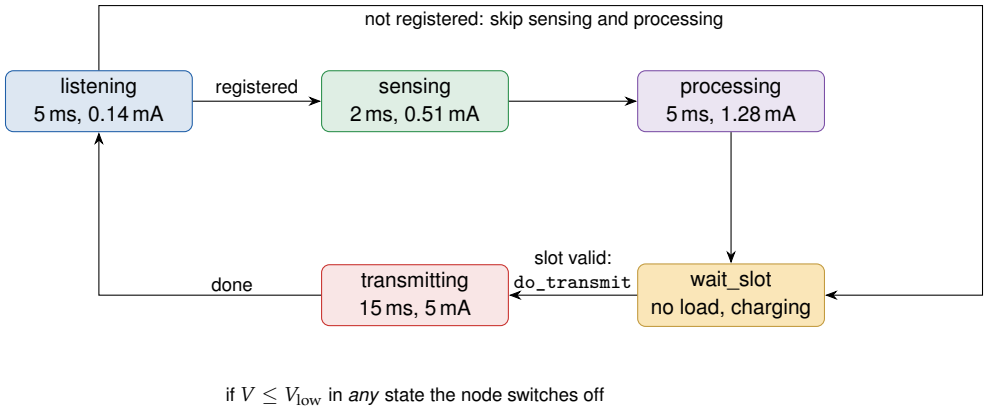


Figure 15.1: The working-mode FSM. Numbers are the demo durations and currents. A node that is not yet registered skips sensing and processing (upper loop) and waits for a command (*send_id*) to give it a slot. A freshly registered node that has no sensor value yet returns from *wait_slot* to *sensing* first.

15.3 The working-mode state machine

Once the node is active and $V > V_{\text{low}}$, each millisecond runs one step of the finite-state machine in Figure 15.1.

Mechanics common to all states. A state is advanced by a *counter* that increases by one each millisecond; the state ends when the counter reaches its configured duration. In every state except *wait_slot* the controller sets `capacitor.current` to the state's current and sends the `charge_and_discharge` interrupt: the node *draws* its load while the harvester *charges* simultaneously.

listening ($d = 5$): the receive window in the model's vocabulary. At its end a *registered* node moves to *sensing*; an unregistered one to *wait_slot*, where it can send its ID if a slot was offered.

sensing ($d = 2$): produces a sensor value (a random integer in $[100, 255]$ in the stub); then *processing*.

processing ($d = 5$): copies the sensed value to `processed_data`; then *wait_slot*.

wait_slot : **draws no load and only charges**. Each millisecond the controller asks the protocol module whether a transmission is permitted now (`controller_tx_ready`). If so, it *immediately* calls `do_transmit`, records the data and moves to *transmitting*.

transmitting ($d = 15$): draws 5 mA for 15 ms; at the end the cycle count is incremented and the node returns to *listening*.

Because *wait_slot* has no end condition other than a transmission (or the voltage dropping to V_{low}, impossible while no load is drawn and the harvester is charging), a node with nothing to say simply *rests at full charge* until the base station gives it a slot. This is why the node-0 voltage in Figure 2.2 climbs above 1.7 V during waiting.

Idealisations of *wait_slot*

(i) Waiting draws *zero* current. A real node must keep its wake-up receiver or timer alive: even 1 µA is not nothing. (ii) The listening state's "receive window" has no effect on whether commands are received (Chapter 16): commands are accepted in any state, including when the node is off.

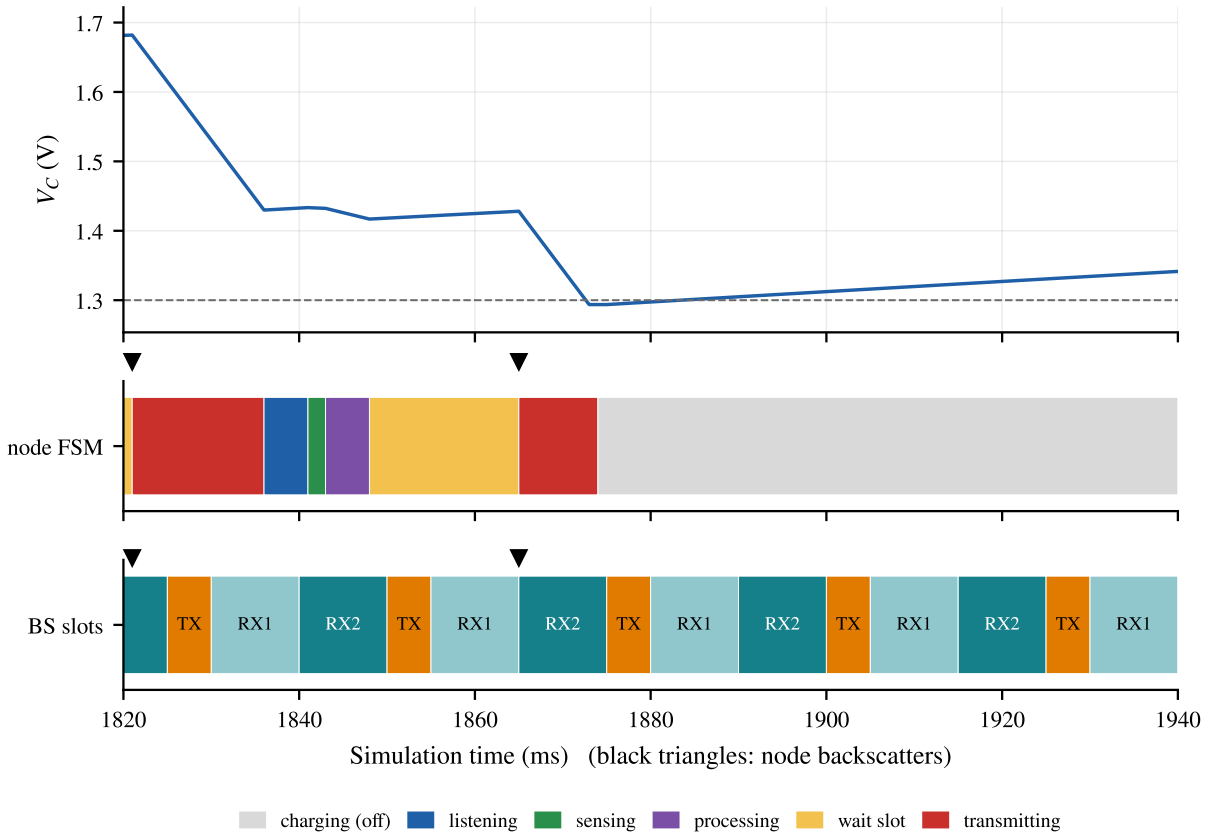


Figure 15.2: A 120 ms window of Node 0 (measured). Top: capacitor voltage; middle: the node's FSM state per millisecond (gold: waiting for a slot; grey: switched off and charging); bottom: the base station's frame (TX 5 ms, two RX windows of 10 ms). Black triangles mark the instants at which the node backscatters. Each packet is delivered at the *start* of the node's transmit state, inside an RX window.

15.4 One real energy cycle

Figure 15.2 zooms into two transmissions of Node 0 from the broadcast demo ($\approx$ 120 ms), with the base-station slot structure underneath.

We can read off the entire mechanism.

1. The first red bar is a full 15 ms transmit state: V falls from 1.68 to about 1.43 V, i.e. $\Delta V = I\,T/C = 0.25$ V as predicted in Chapter 8.
2. Listening (blue), sensing (green) and processing (purple) follow, with a small drop each.
3. During the gold *wait_slot* bar the voltage *rises* (charge only). The wait ends at 1 865 ms, the first millisecond of the base station's next RX window in which the node holds a valid slot.
4. The second transmit state starts, but V reaches $V_{\text{low}} = 1.3$ V after only 9 ms; the node *switches off in the middle of its transmission*, and then charges (grey) for the next second.

Delivered before paid

The packet of the second transmission was handed to the base station at the first millisecond of the transmit state, *before* the node paid the remaining 15 ms of energy. The controller's source says so explicitly: "the `transmitting` state … only models the transmit energy draw; … the reception has already happened." The consequence is energy *causality*: nothing prevents a node from starting a transmission that its capacitor cannot finish, and it is still

counted as delivered. We measured how often it happens in the shipped broadcast demo (20 s, one run): Node 0 had 25 complete and 22 truncated transmit states (47% truncated), Node 1 9 and 9 (50%), Node 2 32 and 24 (43%). Truncated states lasted only 9–11 ms instead of 15 ms. A model in which a truncated burst counted as a *lost* packet would report roughly half the throughput. This also explains the "two packets per saw-tooth tooth" of Chapter 8. The design rule of that chapter applies: choose C, V_{high} and V_{low} so that the energy window covers a full burst, or add a minimum-start voltage check before entering *transmitting*.

15.5 Bookkeeping and logging

- `fsm_count_in_current_cap_cycle` counts *completed* FSM cycles (incremented at the end of *transmitting*); on each shut-down it is appended to `fsm_counts_per_cap_cycle`. In the demo these lists contain mostly 1s, which is the footprint of the truncated second transmission above.
- `data_history` records `(capacitortime,value)` for every delivered datum of a registered node.
- `acc_current_*` lists store the current of every step per state when `keep_logs=True`.
- `keep_logs=False` suppresses all per-step lists (flat memory for long runs).

Where this lives in the code

Concept	Code (`controller.py`)
Currents, durations, thresholds	`ControllerParams` (`CurrentsA`, `DurationsMs`, `VoltageThresholdsV`)
Main loop	`Controller.run`
State machine	`Controller.working_mode`
Charge-and-discharge request	`Controller._enter_charge_and_discharge`
Registered?	`Controller._is_registered`
Start-up delay	`startup_delay_s`, from `max_startup_time_ms`

15.6 Chapter summary

- The controller wakes every ms, recomputes harvest, applies hysteresis and steps a five-state FSM; each active state draws its current while charging.
- A cycle costs $\approx 83\,\mu C$, $\approx 125\,\mu J$ at 1.5 V, 90% in the transmit state.
- *wait_slot* idles for free and charges; a node can rest above V_{high} indefinitely.
- Packets are delivered at the *entry* of the transmit state; in the demo 43–50% of transmit states are cut short by brown-out.
- The controller's conversion $V_{\text{src}} = \sqrt{P_h R_s}$ always uses $R_s = 5\,k\Omega$, not the capacitor's parameter.

Exercises

Exercise 1

Compute the energy of one FSM cycle at 1.7 V and at 1.3 V (charge is the same). By how much does the cycle's energy cost depend on the voltage at which it is run?

Exercise 2

A node is at 1.35 V when it enters *transmitting*. Ignoring harvest and leakage, how long can it transmit before V_{low}? Express as a fraction of the 15 ms burst. What minimum entry voltage guarantees a complete burst?

Exercise 3

Propose a one-line change to `wait_slot` that prevents transmissions from starting when the capacitor cannot complete them. What effect on throughput and on "delivered" statistics would you expect?

Exercise 4

Explain why the start-up delay is needed, and what the demo's collision statistics would look like for N nodes if every node had a delay of zero and identical parameters.

16

The Backscatter Module: Node-Side Protocol

In this chapter you will learn

- the division of labour between `Controller` (energy and timing) and `BackscatterModule` (protocol);
- the full path of a command: sensitivity gate, logging, slot bookkeeping, protocol handler;
- the four conditions a transmission must satisfy before it is allowed (`controller_tx_ready`), and the exact slot-window arithmetic;
- what `do_transmit` computes and writes into the base station, and what the `delivered` flag *really* means;
- how to override the protocol with two small methods.

16.1 Division of labour

The class docstring states it in one line: "Controller handles energy/timing, BackscatterModule handles protocol." The module is *passive*: it owns no SimPy process, only state and methods called by (i) the base station, to deliver commands, and (ii) the controller, to ask whether it may transmit and to perform the transmission.

Attribute	Meaning
`state`	`"idle"`, `"wait_ack"`, `"registered"`
`rx_slots`	list of (start, end) ms tuples advertised by the last command
`chosen_slot_idx`	index into `rx_slots` chosen by this node, or -1
`last_tx_command_time`	time of the last command that enables a reply, or -1
`tx_records`, `rx_records`	logs of transmissions and received commands
`packets_sent`, `packets_delivered`, `acks_received`	counters

16.2 Receiving a command

A command arrives as a synchronous call `receive_command(cmd,bs_id,data)` made by the base station at the start of its TX slot. The method is a short *pipeline*.

1. **Sensitivity gate.** Look up the node's best downlink power (`downlink_results["best_pw_dbm"][node.id]`, default -999) and compare with `node.sensitivity_dbm`. If weaker, *return*: the command is ignored and nothing is logged.
2. **Log.** Append an `RxRecord(end_ms,cmd,from_bs)` if `keep_logs`.
3. **Slot bookkeeping.** Store `data["rx_slots"]` in `self.rx_slots`, replacing any previous list.
4. **Protocol handler.** Call `handle_command(cmd,bs_id,data)`.

Steps 1–3 are *framework* behaviour common to every protocol; step 4 is the *policy* you may override.

Energy does not gate reception

Nothing in this pipeline looks at whether the node is awake: not the capacitor, not the controller's `is_active`, not the state name. We confirmed it by instantiating a module with *no controller and no capacitor at all*: it accepted a `send_id` command, chose a slot and moved to `wait_ack`. A node whose capacitor is empty therefore still "hears" commands and updates its protocol state. Its *transmission* is still gated by energy, because only a running controller in the *wait_slot* state can call `do_transmit`. In real hardware the hear/transmit asymmetry would be different: a node with no energy hears nothing at all.

16.3 The default handler

`handle_command` implements the registration-and-polling protocol of Chapter 11.

`send_id`. If already *registered*: clear `chosen_slot_idx` and `rx_slots` (so nothing is transmitted by accident) and return. Otherwise draw `chosen_slot_idx` uniformly in $\{0, \ldots, |\texttt{rx_slots}| - 1\}$ (or -1 if no slots were advertised), set `state="wait_ack"` and `last_tx_command_time=now`.

`ack`. If `state=="wait_ack"`: set `state="registered"` and increment `acks_received`. In any other state: ignore.

`send_data`. Read `target` (default -1). If `target` is neither -1 nor this node's id: return. If registered: choose a slot (for a unicast target, index $|\texttt{rx_slots}| - 1$, which is the only slot because the base station truncates the list to one; for a broadcast, uniformly at random); set `last_tx_command_time=now`.

List targets are accepted by the base station but ignored by the node

The base station's `_deliver_command` supports a *list* of target ids ("send to these nodes") and sends the command to each listed node. But the default `handle_command` compares `target!=self.node.id`, and a list is never equal to an integer, so *every addressed node returns without choosing a slot*. We tested with `target=[0,1]`: both nodes had `chosen_slot_idx=-1` and `last_tx_command_time=-1` afterwards; no reply is ever sent. If you need multicast, override `handle_command` and test `self.node.idintarget`.

16.4 When may the node transmit?

The controller, while in *wait_slot*, calls `controller_tx_ready(sensor_data)` every millisecond. The method returns a dictionary describing what to send, or `None` for "not now". It applies *four* checks in order:

1. **A command has enabled a reply:** `last_tx_command_time>=0`.
2. **A valid slot was chosen:** $0 \leq$ `chosen_slot_idx` $<$ |`rx_slots`|.
3. **Now is inside the chosen slot:** slot_start $\leq t <$ slot_end. The upper bound is *strict*: at $t =$ slot_end the base station's window (`whilenow<end_time`) has already closed and would miss the packet.
4. **The slot is not stale:** slot_start $\geq$ last_tx_command_time, i.e. the slot belongs to the command, not to an earlier cycle.

If all pass, it returns `build_tx_payload(sensor_data)`: for the default protocol a dictionary with keys `type` and `payload`—the node's id (type `id`) while in *wait_ack*, the sensor value (type `data`) when *registered*—and `None` otherwise.

A node that is late for its slot

The base station sends `send_data` at $t = 100$ ms; its RX windows are $[105, 115)$ and $[115, 125)$. Suppose a node chooses $[105, 115)$ but its controller is in *processing* until $t = 112$ ms. At $t = 112$ it enters *wait_slot*, `controller_tx_ready` finds $105 \leq 112 < 115$ and the node *does* reply, at 112 ms, late in the slot. If it had finished at $t = 115$, it would have missed the slot for good: the stale slot is never reused, `chosen_slot_idx` stays set, and the node must wait for a new command. This is why a command can hit a node *without* producing a reply, and why base stations "see" fewer replies than commands.

16.5 Transmitting

`do_transmit(tx_info)` performs the uplink *in zero simulated time*:

1. For each base station process B in `bs_processes`: scan the pre-computed uplink table `per_sector_powers` for sectors named `BS<id>_*` and pick the sector with the *strongest* RSSI for this node.
2. Write into B the *scratch* fields: `received_tag_id`, `data_from_tag` (the payload), `rssi_dbm`, `best_sector_idx`, `subcarrier_shift`.
3. Interrupt B with cause `"receive_data"`. Since interrupts are urgent (Chapter 12), B buffers this packet *before* any other process runs, so the scratch fields cannot be overwritten.
4. Log a `TxRecord(end_ms,payload_type,payload,slot_idx,delivered)` and update counters.
5. **Clear** the slot state (`chosen_slot_idx=-1`, `rx_slots=[]`, `last_tx_command_time=-1`). A node must receive a *new* command before it can transmit again; this prevents repeated transmissions in the same slot.

`delivered` does not mean "decoded"; it barely means "heard"

`TxRecord.delivered` is `True` if the packet was *handed to at least one base-station process*. We tested a node whose uplink RSSI was -150 dBm: its record said `delivered=True`, yet the base station dropped the packet at the sensitivity gate and logged nothing. With the default tables every base station in `bs_processes` has an entry for every node, so `delivered` is essentially *always* true when the module is wired. All the shipped CSVs show `delivered=True` for every transmission. To measure real delivery you must look at the *base station's* `rx_packets` and their `collided` flag, as the demos do.

16.6 Customising the protocol

Two methods express a complete protocol:

```
class MyNode(backscatter.BackscatterModule):
    def handle_command(self, cmd, bs_id, data):
        # called after sensitivity gate, logging, rx_slots stored
        if cmd == "collect":
            self.chosen_slot_idx = (random.randint(0, len(self.rx_slots)-1)
                                    if self.rx_slots else -1)
            self.state = "active"
            self.last_tx_command_time = self.env.now

    def build_tx_payload(self, sensor_data):
        # called only when now is inside a valid, fresh slot
        return {"type": "data", "payload": sensor_data} \
               if self.state == "active" else None
```

This is the node half of the adaptive-ALOHA demo (Chapter 20). Note the *obligations* the base class leaves to you: set `chosen_slot_idx` and `last_tx_command_time` when you want to allow a reply, and return a dict from `build_tx_payload`. Everything else—the slot validity checks and the uplink delivery—is inherited.

16.7 Logging

`save_logs(clear=True)` writes tab-separated files `node_<log_file>_tx.txt` and `node_<log_file>_rx.txt` if a `log_file` stem was given, and then clears the lists. With `keep_logs=False` no records are stored; the counters `packets_sent`, `packets_delivered`, `acks_received` are always maintained.

Where this lives in the code

Concept	Code (`backscatter.py`)
Command pipeline	`receive_command`
Protocol policy (override)	`handle_command`
Transmit permission	`controller_tx_ready`
Payload policy (override)	`build_tx_payload`
Uplink delivery	`do_transmit`
Records	`TxRecord`, `RxRecord`

16.8 Chapter summary

- The module is passive protocol logic; the controller asks it whether to transmit, the base station tells it what to do.
- Reception is gated by downlink power only; transmission additionally needs the controller to be in *wait_slot*, inside a fresh, valid slot.
- `do_transmit` writes pre-computed RSSI into each base station and interrupts it; the slot state is cleared afterwards.
- `delivered` means "handed to a base station", not "decoded"; list targets are ignored by the default handler.

- Two overridable methods, `handle_command` and `build_tx_payload`, suffice to implement a new protocol.

Exercises

Exercise 1

Trace a `send_id` command to a node with downlink power -50 dBm and `sensitivity_dbm=-45`. What happens and what is logged?

Exercise 2

A node chooses slot $[205, 213)$. At what times can `controller_tx_ready` succeed? What if the command was sent at $t = 206$?

Exercise 3

Rewrite `handle_command` so that `send_data` with a list target works. Which attribute must be set to allow the reply?

Exercise 4

Design a node protocol in which a node transmits only if its capacitor voltage exceeds 1.6 V. Where would you read the voltage, given that `BackscatterModule` has no reference to the capacitor?

17

The Base-Station Engine

In this chapter you will learn

- the schedule language and the two ways to drive a base station: a static *schedule* or a *policy* generator;
- what happens in a TX window and in an RX window, including the half-duplex rule;
- the *RX-slot look-ahead* that lets commands advertise future reply windows, and its two-frame horizon (with measured consequences);
- how acknowledgements and registration state are maintained;
- logging, counters, and the failure mode when a base-station process ends too early.

17.1 Role and constructor

`BSBehavior` is one base station's *behaviour*. The geometry and radio parameters live in the `BaseStation` dataclass; `BSBehavior` adds time: it runs a schedule, commands nodes, listens for replies, resolves collisions, and counts. It is a SimPy process.

Argument	Default	Meaning
`schedule`	`None`	list of entries (below); static frame repeated
`policy`	`None`	callable returning a generator of frames; *overrides* `schedule`
`backscatter_modules`	`None`	the nodes it commands (needed for `receive_command`)
`loop`	`True`	repeat the schedule
`required_sinr_db`	`3.0`	γ_{req} (Chapter 10)
`enable_sic`	`True`	SIC vs capture-only
`cancellation_factor`	`0.9`	*inert* (Chapter 10)
`noise_figure_db`, `bandwidth_hz`	`6.0, 100e6`	thermal noise N (Chapter 3)
`keep_logs, stream_log`	`True, None`	in-memory lists, or stream to disk
`on_rx, on_tx`	`None`	callbacks per decoded packet / sent command
`log_file`	`None`	stem for `save_logs`

17.2 The schedule language

A schedule is a list of entries `(mode,duration_ms,slot_id[,payload])`:

Entry	Meaning
`("rx", 10, "listen")`	listen 10 ms for any node
`("rx", 10, "poll_0", {"expect": 0})`	listen, expecting node 0 (sets the `matched` flag only)
`("tx", 5, "cmd", {"cmd":"send_id", "target":-1})`	broadcast a command
`("tx", 5, "ack", {"cmd":"ack"})`	send ACKs to all pending nodes
`("tx", 5, "cmd", {"cmd":"send_data", "target":3})`	unicast to node 3

The entries are executed in order, each lasting its duration; with `loop=True` the list repeats forever. The sum of durations is the *frame duration*. The demos' schedules are:

- **broadcast**: `[txsend_data(-1)5ms]+[rx10ms]` $\times 2$ (frame 25 ms);
- **unicast**: `[txsend_data(i)5ms,rx10ms]` for $i = 0, 1, 2$ (frame 45 ms);
- **custom**: a policy yielding [tx collect] + [rx 8 ms] $\times S$ with S adapting.

17.3 TX and RX windows

TX window (`_do_tx`). At its *start* the base station delivers the command: for `cmd=="ack"` it calls `_send_pending_acks`; otherwise `_deliver_command` (below). It then sleeps for the duration *ignoring any interrupt* — "half-duplex: can't receive while transmitting" — and finally logs a `TxPacket` and calls `on_tx`. A node that backscatters during a TX window is simply not heard. Because the carrier is separate and always on, this is not a physical full-duplex limitation but a modelling choice: the schedule time-multiplexes commands and replies.

RX window (`_do_rx`). At its start the buffer is cleared and the `expect` filter noted. The base station then sleeps; every `"receive_data"` interrupt appends the packet to the buffer (time, node, payload, RSSI, sector, subcarrier) and resumes the sleep for the *remaining* time. At the end it processes the buffer with Algorithm 1. Decoded packets are added to `decoded_this_frame`, queued for ACK if new, and passed to `on_rx`.

17.4 Telling nodes when to answer: the RX-slot look-ahead

A node can answer only in a window the base station has *authorised* (Chapter 16). The base station therefore attaches to each command the list of *upcoming RX windows*, found by scanning its own schedule (`_get_next_rx_slots`). Algorithm 4 states the rule.

Two details deserve emphasis.

- For a *unicast* command (`target!=-1`) the list is truncated to its *first* element: "reply in the very next RX window".
- For a *broadcast* the list contains *every* RX window found in the scan, and each registered node picks one at random.

Algorithm 4 RX-slot look-ahead (`BSBehavior._get_next_rx_slots`)

Input: current schedule position i_0, frame start T_f, frame duration F, schedule length m, time $t_{\min}$ (end of the command)

slots $\leftarrow [\,]$; $i \leftarrow i_0$; offset $\leftarrow 0$
for $k = 1, \ldots, 2m$ **do** ▷ scan up to two full frames
  $i \leftarrow (i+1) \bmod m$; **if** $i = 0$ **then** offset $\leftarrow$ offset $+ 1$
  start $\leftarrow T_f +$ offset $\cdot F + \sum_{j<i} d_j$; end $\leftarrow$ start $+ d_i$
  if mode$_i$ $=$ `rx` **and** start $\geq t_{\min}$ **then** append (start, end)
return slots

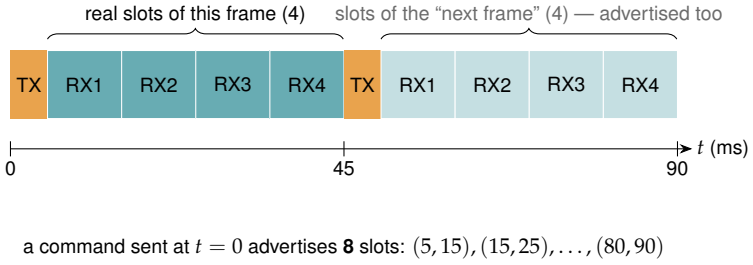


Figure 17.1: The look-ahead from a command at $t = 0$ with a frame of one 5 ms TX window and four 10 ms RX windows. The scan covers two frames, so the command advertises eight RX windows, only four of which belong to the current frame. We measured exactly this: `rx_slots` had length 8, starting with $(5, 15)$ and ending with $(80, 90)$.

17.4.1 Consequence: half of the contenders sit the frame out

A node picks `random.randint(0,len(rx_slots)-1)`: uniformly among *all* advertised windows, i.e. among $2S$ for a frame of S windows. With probability $\frac{1}{2}$ it picks a window in the *next* frame. In a periodic schedule the next frame starts with the *same* command, which re-draws the node's slot before that window arrives, so the earlier pick is never used. The practical effect is that **each ready node participates in only about half of the frames**.

This is consistent with everything we measured. In the registration experiment (Chapter 11) five nodes registered at 44%, 70%, 82%, 88% after rounds 1–4, close to $1 - 0.5^k$. In the custom-protocol demo the largest slot index ever used was 11 while the frame size ranged between 7 and 20 (an index $\geq S$ never transmits).

A look-ahead designed for static schedules, used with adaptive ones

For a *static* periodic schedule the "next frame" windows are real, and the scan of two frames "handles edge cases" (a code comment). For an *adaptive* policy the next frame may have a different size, and for the *last* frame of a finite policy it does not exist. Three practical rules follow:

1. account for the factor $\frac{1}{2}$ when you size frames (the custom demo's rule $S \approx 2\ell$, where ℓ counts *participating* contenders, implicitly does);
2. put the command and its RX windows in the *same* frame (the `_run_policy` docstring says so), or the look-ahead will not see them;
3. never let the base-station process terminate while nodes may still hold future slots (Section 17.8).

17.5 Policy mode

Instead of a static list you can supply `policy`, a function of the `BSBehavior` returning a generator of *frames*; a frame is a list of schedule entries. `_run_policy` executes each yielded frame exactly as if it were the schedule (including look-ahead within that frame), then *resumes the generator*, which can read the engine's state before deciding on the next frame:

State available to a policy	Use
`decoded_this_frame`	node ids decoded in the last frame
`total_collided`, `total_rx`, `total_tx`	cumulative counters
`nodes_pending_ack`, `nodes_registered`	registration bookkeeping
`env.now`	current time

Because the generator is *lazy*, each frame is decided only after the previous one has finished; that is what makes the protocol adaptive (the adaptive-ALOHA demo of Chapter 20).

17.6 Acknowledgements and registration state

When a non-colliding packet from a new node is decoded, its id is appended to `nodes_pending_ack`. At the next `("tx", ..., {"cmd": "ack"})` entry, `_send_pending_acks` (i) calls `receive_command("ack", ...)` on the matching node module, (ii) removes the node from *pending* and (iii) adds it to `nodes_registered`. The engine marks a node registered *when it sends the ACK*, whether or not the node accepted it. Normally the node, which just transmitted, passes the sensitivity gate and registers too; if it did not (for instance the downlink dropped below its sensitivity), the base station believes the node is registered while the node is still in *wait_ack*. Such state divergence cannot happen in a perfect simulator but is exactly what real ACK loss produces.

17.7 Counters and logs

- **Always-on counters** (constant memory): `total_rx` (every logged reception, collided or not), `total_collided`, `total_tx`, `rx_count_by_node` (decoded packets per node).
- **Lists** `rx_packets`, `tx_packets` when `keep_logs=True`.
- **Streaming**: `stream_log="path"` writes `path_rx.txt` and `path_tx.txt` (tab-separated) as packets arrive, so memory stays flat; call `close_stream()` after the run.
- **Batch**: `save_logs()` writes `bs_<log_file>_rx.txt`, `_tx.txt` using pandas.

An `RxPacket` carries `start_ms` (the start of the RX *window*), `end_ms` (the *arrival time* of the packet), the slot id, node, payload, RSSI, sector, the `expect/matched` flags, the subcarrier and the `collided` flag. Note that `start_ms` is the window start, not the packet start: the demos' CSVs `time_ms` column is this window start.

17.8 When the base station stops

If the schedule has `loop=False`, or a policy generator is exhausted, the SimPy process *terminates* (`mode` becomes `"idle"`). SimPy refuses to interrupt a terminated process. A node that later backscatters (it may still hold a slot from the final look-ahead) then triggers a `RuntimeError: ... has terminated and cannot be interrupted`, which aborts the *whole* simulation. We

hit exactly this while building the registration experiment, and avoided it by ending the policy with a long final idle window (`[("rx",10_000,"tail")]`) so that the process stays alive.

Where this lives in the code

Concept	Code (`bsengine.py`)
Main loop and frames	`BSBehavior.run`, `_run_policy`
TX window	`_do_tx`, `_deliver_command`
RX window	`_do_rx`, `_buffer_rx`, `_process_rx_buffer`
Look-ahead, Algorithm 4	`_get_next_rx_slots`
ACKs	`_send_pending_acks`
Packet records	`RxPacket`, `TxPacket`
Logging	`save_logs`, `close_stream`, `stream_log`

17.9 Chapter summary

- A base station runs a schedule (or policy) of TX and RX windows; commands are delivered synchronously at the start of a TX window, replies are buffered during RX windows and resolved at their end.
- Commands advertise upcoming RX windows by scanning *two frames* ahead; unicast keeps only the first. A uniformly random pick among $2S$ windows means each ready node takes part in about half of the frames.
- Policies are lazy generators of frames that can read `decoded_this_frame` and counters.
- ACKs mark a node registered at the base station when sent, regardless of node reception.
- A terminated base-station process crashes the simulation if a node later transmits; keep it alive.

Exercises

Exercise 1

For the unicast demo's schedule (frame $= 45$ ms), list the first three advertised slots for the command to node 1 sent at $t = 60$ ms, where the current frame started at $t = 45$ ms. Which one is kept?

Exercise 2

A broadcast frame has $S = 6$ RX windows. What is the probability that a ready node transmits in the *current* frame? What is the expected number of the $n = 12$ ready nodes that transmit?

Exercise 3

Write a policy generator that alternates between a discovery frame (`send_id` + 8 slots + ACK) until 90% of N expected nodes are registered, then switches to unicast polling of registered nodes. What must the final frame contain to avoid the termination crash?

Exercise 4

Why is `RxPacket.start_ms` unsuitable as the packet's start time when interpreting overlaps? What would you use instead?

18

Packet Analysis and Performance Metrics

In this chapter you will learn

- the standard metrics—delivery error rate (DER), throughput, per-node and per-base-station statistics—and exactly how the demos and the analysis module define them;
- the *two* ways Amber classifies collisions (real-time and post-hoc) and why they can disagree;
- a verified defect in the post-hoc path that makes almost every packet look collided;
- practical guidance on which numbers to trust and how to report them.

18.1 Metrics

A simulator is only as useful as the questions its output can answer. For Ambient IoT networks the natural metrics are the following.

Delivery and throughput

Let N_{sent} be the number of packets nodes handed to the channel, N_{dec} the number decoded without collision loss at the base station, and T the simulated time.

$$\text{PDR} = \frac{N_{\text{dec}}}{N_{\text{sent}}}, \qquad \text{DER} = 1 - \text{PDR}, \qquad \text{throughput} = \frac{N_{\text{dec}}}{T}\ [\text{packets/s}].$$

Other useful quantities: per-node success counts (fairness), the number of registered nodes versus time (discovery), collisions per frame, and energy-related ones such as the number of FSM cycles per charge cycle or the fraction of time a node is active.

18.1.1 How the demos compute them

Each demo prints

$$\texttt{total_sent} = \sum_{\text{nodes}} \texttt{packets_sent}, \qquad \texttt{decoded} = \left|\{(\text{node}, \text{window start}) : \neg\text{collided}\}\right|,$$

$$\texttt{DER} = 1 - \texttt{decoded}/\texttt{total_sent}, \qquad \texttt{throughput} = \texttt{decoded}/(\texttt{SIM_TIME_MS}/1000).$$

Note that the numerator counts *unique* (node, window) pairs, and that "sent" counts all handed-over packets including those the base station later drops for being below its sensitivity (the denominator is not reduced by them). For the unicast demo in one run: 125 sent, 125 decoded, DER $= 0\%$, 6.2 packets/s.

18.1.2 Name clash: "DER"

Two different quantities share the name "DER"

In the demos DER is an *error* rate: $1 - \text{decoded/sent}$ (smaller is better; the plots label it "Delivery Error Rate"). In `packet_analysis.analyze_packets`, `BSStats.der` is received/transmitted, a *success* ratio (larger is better), and "transmitted" there means "rows in the base station's RX log", i.e. packets *logged* by the base station, not packets *sent* by nodes. In our test run the same simulation gives demo-style DER $\approx 2.4\%$ (121 decoded of 124 sent) but analysis-module `der`$= 0.976$. Always state which you mean.

18.2 The `packet_analysis` module

Besides `apply_sic` and the noise helper used by the engine, `packet_analysis.py` provides a post-processing pipeline.

Function / class	Role
`load_packet_log(path)`	read a tab-separated BS RX log; requires `start_ms,end_ms,node_id,rssi_dbm,subcarrier_shift`
`detect_collisions(df,...)`	*post-hoc* collision detection by time overlap, then SIC per overlap
`analyze_packets(...)`	per-BS and per-node statistics; writes optional text logs
`BSStats`	`transmitted, received, errors, collisions_total, der`
`NodeStats`	`sent, received_at_bs, total_errors, heard_by_bs`
`CollisionEvent`	kind $\in$ {total, n_way, capture, sic_success}, nodes, powers, SINRs, winners

How `analyze_packets` chooses its path. If the log has a `collided` column (which the base station writes) the function *trusts it*: received = rows with `collided==False`, errors = rows with `collided==True`. Only if the column is missing does it call `detect_collisions` and re-derive the outcomes.

Post-hoc detection. For each subcarrier it builds start/end events for each packet interval, sweeps in time order, and whenever a packet starts while others are active, it forms a collision set and applies SIC (default $\gamma_{\text{req}} = 10$ dB here, versus 3 dB in the engine) to label packets `decoded`, `captured`, `sic_recovered` or `collision_loss`. Collision events are classified `total` (two packets, none decoded), `n_way` (> 2, none decoded), `capture` (one decoded) and `sic_success` (two or more).

Side effects to know about. By default `analyze_packets` *appends* a text report to `terminal_out.txt` and collision lines to `collision_out.txt` in the current directory on every call; pass `write_terminal_log=None,write_collision_log=None` to disable. The demos' own CSV files (`rx_packets.csv`) lack `start_ms/end_ms`, so `load_packet_log` rejects them ("Missing required columns"): the analysis module expects logs written by the engine's `save_logs/stream_log`.

18.3 Real-time versus post-hoc classification

We ran the same 20 s broadcast simulation (SIC off) and analysed its engine-written log in both ways.

Path	Received (decoded)	Errors	Collision events
Real-time (`collided` column)	121	3	–
Post-hoc, as logged	**1**	**123**	122
Post-hoc, zero-length intervals nudged by 1 µs	120	4	4

The real-time classification and the (repaired) post-hoc classification agree to within one packet; the unrepaired post-hoc path reports that 99% of packets were lost.

A latent defect in the post-hoc sweep

In the engine's log, a packet's `start_ms` is the start of the RX *window* and `end_ms` is its *arrival* time. A node that happens to be ready when the window opens transmits in the very first millisecond, so `start_ms==end_ms`: 100 of the 124 packets in our run have zero-length intervals. The sweep sorts events by `(time,is_start)`, i.e. *end* events before *start* events at the same time. For a zero-length packet the *end* is therefore processed first (removing nothing, marking the packet `decoded`) and the *start* afterwards, which inserts the packet into the `active` set *permanently*. Every later packet then "collides" with this ghost, producing the cascade of *n*-way collisions (122 events in our run). The one-line remedy (end events after start events for equal times, or a minimum duration) restores agreement. It does not affect the demos, which count via the engine's real-time flags, but it makes `analyze_packets` unreliable on logs lacking the `collided` column. Remember also that, for engine logs, the overlap intervals are not airtime: they are "window start to arrival", so even a repaired post-hoc detector approximates collision by an interval-overlap rule different from the engine's 5 ms rule.

18.4 Practical guidance

1. **Prefer the engine's counters** (`total_rx`, `total_collided`, `rx_count_by_node`) and `collided` flags for headline results; they are cheap, constant-memory and come from the classification actually used by the simulation.
2. **Report what you divide by.** "Sent" includes packets later dropped below sensitivity; if you want the PDR of *audible* packets, count rows in `rx_packets` instead.
3. **Report distributions.** Replicate and show box plots or confidence intervals (Chapter 12); a single unseeded run is one sample.
4. **Separate energy from radio losses.** A silent node is not a lost packet. Count *opportunities* (polls to a node that was in *wait_slot*) if you want a radio-only PDR.
5. **Mind the logging overhead.** For long runs use `keep_logs=False` and `stream_log`.

Where this lives in the code

Concept	Code
Demo metrics	`run_simulation()` in each demo; `save_packets`
Always-on counters	`BSBehavior.total_rx`, `total_collided`, `total_tx`, `rx_count_by_node`
Offline statistics	`analyze_packets` (`packet_analysis.py`)
Offline collisions	`detect_collisions`, `_analyze_collision_outcome`

18.5 Chapter summary

- DER (error) $= 1 -$ decoded/sent in the demos; `BSStats.der` is a success ratio over logged receptions. Do not mix them.
- Prefer the engine's real-time `collided` flags; the post-hoc detector is unreliable on engine logs because of zero-length intervals (we measured 1 vs 121 decoded).
- `analyze_packets` writes text files by default and does not read the demos' CSV files.
- Report distributions, and separate energy-induced silence from channel-induced loss.

Exercises

Exercise 1

A run reports `total_sent` $= 130$, `decoded` $= 117$ over 20.001 s. Compute DER, PDR and throughput. If 6 of the missing 13 packets were dropped below sensitivity, what is the PDR of audible packets?

Exercise 2

Explain why `start_ms==end_ms` for a packet that arrives in the first millisecond of its window, and sketch the event order of the sweep for two such packets in consecutive windows.

Exercise 3

Design a metric that separates "lost to energy" from "lost to collisions" using only data the base station can log. Which additional information from the simulator (not available in a real network) would you use as ground truth?

Part VI

Experiments and Critical Assessment

19

Reading the Experiments

In this chapter you will learn

- how the four shipped demos are configured and what each is designed to show;
- how to *interpret* their output with the theory of Parts II and III: a hand-built budget table that predicts who is awake and who collides;
- what the numbers actually were (measured, with run-to-run variation);
- a worked analysis of the adaptive-ALOHA demo, including a surprising finding about fairness that traces back to the path-loss model;
- how to read the paper's Figure 2 and what a faithful reproduction requires.

19.1 The four demos at a glance

	broadcast	broadcast_sic	unicast	custom_protocol
Question	what do collisions cost?	can SIC remove them?	does polling avoid them?	can a user protocol adapt?
Nodes	3 (2 WPT, 1 hybrid)	3	3	12, hybrid
Placement	$y = 20, 30, 100$ m north	same	same	random, $r \in [5, 40]$ m, seeded
BS	1 mast, 3 sectors, 46 dBm, 15 dBi	same	same	same
Beamwidth	65°	30°	30°	65°
Capacitor	300 µF	200 µF	300 µF	300 µF
Schedule	TX + 2×RX_{10}	TX + 3×RX_{15}	3×(TX+RX_{10})	policy: TX + S×RX_8
SIC	off	on	on (default)	on
Registration	forced	forced	forced	implicit
Horizon	20 s	20 s	20 s	15 s

All use 924 MHz, UMa LOS, $\eta = 0.7$, node and sector sensitivities of -100 dBm, thresholds 1.7/1.3 V, and the solar trace through a hybrid coverage map for one node (or all, in the custom demo). The first three run in $\approx$ 2–4 seconds of wall-clock time.

19.2 A budget table that predicts the results

Everything the first three demos do follows from five numbers per node, all of which we derived by hand in Parts II. Collect them:

	Node 0	Node 1	Node 2
Distance north (m)	20	30	100
UMa LOS path loss (dB)	60.08	62.10	71.57
Downlink power P_{dl} (dBm)	+0.92	−1.10	−10.57
Energy mode	WPT	WPT	hybrid (solar wins)
Source voltage V_{src} (V)	2.49	1.97	2.66 (solar)
Time to first 1.7 V (s)	1.8	3.4	1.6
Sustainable current at 1.5 V (µA)	182	79	218 (solar)
Uplink RSSI (dBm)	−45.7	−49.7	−68.7

Prediction 1: who sends how much? Node 1 has the weakest source (1.97 V), so it charges slowest and sends least; Node 2 draws on the strong solar source and sends most; Node 0 is in between. The measured packet counts (20 s, one run): Node 0 46, Node 1 18, Node 2 58. The ordering is exactly that of the sources, *not* that of the distances.

Prediction 2: who loses collisions? With SIC off a collision keeps only the *strongest* packet and only if it exceeds the rest by 3 dB. Node 0 (−45.7) is 4.0 dB above Node 1 (−49.7) and 23 dB above Node 2 (−68.7), so it always wins; Node 1 beats Node 2 by 19 dB; Node 2, the weakest, is the casualty. The measured outcome: decoded 46/46, 18/18, 53/58: *all* errors fall on Node 2.

Prediction 3: SIC removes all errors. After capturing Node 0 the remaining pair is 19 dB apart; SIC recovers it. Measured: 133 sent, 133 decoded.

Prediction 4: unicast has no collisions but low answer rate. Polling cannot collide; the 20 s run issued ≈ 1333 polls and decoded 125 (≈ 9%), the rest hitting nodes that were charging.

19.3 Measured results

Single-run counts vary with the random start-up delays (unseeded). We give the single-run values quoted above together with the ten-seed statistics from our ablation (broadcast scenario, Chapter 10).

Demo	Sent	Decoded	DER	Throughput	Notes
broadcast (SIC off)	122	117	4.1%	5.8 pkt/s	errors only on Node 2
broadcast_sic	133	133	0.0%	6.6 pkt/s	
unicast	125	125	0.0%	6.2 pkt/s	≈ 9% of polls answered
custom_protocol	753	692	8.0%†	46.1 pkt/s	12 nodes, 199 frames
10-seed mean, broadcast	124.4	–	2.57% (±0.85)	6.06 pkt/s	SIC off
10-seed mean, broadcast	124.4	–	0.00%	6.22 pkt/s	SIC on

† collided share of logged receptions (60 of 752).

19.4 Case study: adaptive framed-slotted ALOHA

The custom demo runs 12 hybrid nodes with random placement ($r \in [5, 40]$ m, seeded, hence *exactly reproducible*: our run matched the committed CSV to the packet: 753 sent, 692 decoded, 60 collided). The base station starts with $S = 20$ slots and adapts (Chapter 9).

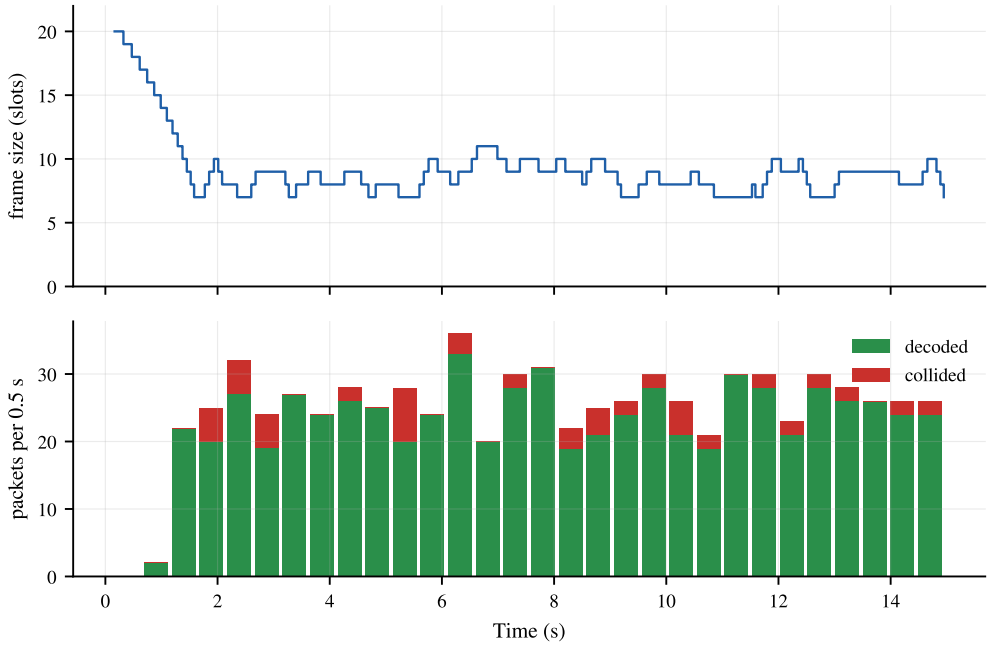


Figure 19.1: Adaptive framed-slotted ALOHA over 15 s (from the repository's committed output). Top: frame size per frame; it collapses from 20 to about 8 within 1.5 s as the BS learns how few nodes are contending. Bottom: decoded and collided packets per 0.5 s bin.

Dynamics. For the first second no node has activated (they are charging), so frames are empty and the policy shrinks the frame by one slot per frame ($20 \to 7$). From 0.8 s nodes start answering; the frame size settles around $S \approx 8.4$ with an average of 3.9 contenders per frame (3.6 decoded, 0.3 collided). Only ≈ 4 of the 12 nodes contend in any frame: the others are charging, or sit the frame out because of the look-ahead effect of Chapter 17.

Collisions are rarer than ALOHA predicts. With $n = 3.9$, $S = 8.4$ the textbook probability that a given node's slot is shared is $1 - (1 - 1/S)^{n-1} \approx 31\%$. The measured collided share is 8%. The difference is due to (i) capture and SIC resolving most pairs (the nodes' powers span -66.7 dBm to -14.6 dBm), and (ii) the 5 ms rule, which lets same-slot packets that start ≥ 6 ms apart coexist.

A surprising fairness result. The per-node decoded counts are very uneven:

node	r (m)	P_{dl} (mW)	decoded	node	r (m)	P_{dl} (mW)	decoded
3	7.0	32.6	**94**	0	16.3	0.63	41
4	6.3	8.2	**95**	1	27.8	0.11	41
5	7.4	6.1	**90**	2	23.8	0.33	40
7	9.3	44.3	**77**	6	19.9	0.63	41
				8,9,10,11	25–39	0.1–0.8	42–44

Eight nodes decode an almost identical 40–44 packets, and four decode about twice as many. Both facts follow from the theory.

1. The eight "ordinary" nodes receive less RF power than the solar trace delivers (1.43 mW). With `combine_mode="max"` their energy source is the *same* solar trace, so they have identical energy budgets and, in an uncongested system, identical rates: *energy-limited throughput* (Chapter 11).

2. The four "rich" nodes lie within 10 m of the mast, where the RF power (6–44 mW) *exceeds* the solar power by up to 30×, so the max-rule makes RF their source and they recharge much faster.

But 44 mW at 9.3 m is +16.5 dBm, absurdly high for a 46 dBm transmitter with a 25 m mast: it is the *<10 m path-loss artefact* of Chapter 5 (UMa falls through to its second branch and understates the loss by $\sim$ 14 dB). This is an excellent worked example of how a numerical edge case in the channel propagates, through the energy model, into a protocol-level fairness result. **The unevenness is not a property of adaptive ALOHA; it is a property of a path-loss branch.** The base station's frame adaptation is working correctly in both cases.

19.5 The paper's experiment

We summarised the experiment and our re-creation in Chapter 11. Here are practical notes for reading its Figure 2.

- The figure is a *box plot*: for each node count, the box spans the interquartile range of the 10 runs, the centre line (or marker) the median, whiskers the range of non-outliers, dots the outliers. The wide boxes at low density ($\sim$ 80–100%) reflect the discrete nature of small populations (with 5 nodes, one failure is 20 percentage points).
- A faithful reproduction needs the *discovery frame parameters*: slot count, slot length, number of rounds, how nodes choose subcarrier shifts, and thresholds/electrical constants; none is given in the abstract. We therefore reproduce *mechanisms*: monotone decay, effect of SIC, role of participation.
- With three base stations, a node may be heard by several; the paper's registration counts presumably count a node once. In Amber, ACKs are per base station (each `BSBehavior` has its own lists), so you must merge `nodes_registered` across stations.

19.6 Things to try

Each is a one-line change that tests a piece of theory.

1. **Capacitance.** Double C: the window ΔE doubles, charge cycles lengthen, packets per cycle rise but the long-run rate (duty-cycle theorem) barely changes.
2. **Efficiency η.** Lower it from 0.7 to 0.2: every uplink weakens by 5.4 dB; the margin of the 100 m node over the −100 dBm sector sensitivity shrinks from 31 to 26 dB, and its collisions with stronger nodes become harder to resolve.
3. **Remove the solar source.** Set `node_energy_mode="wpt"` for all nodes: Node 2 never activates.
4. **Move nodes beyond 10 m** in the custom demo and watch the fairness gap close.
5. **Sector power.** Reduce power by 10 dB; the downlink of Node 1 falls from −1 to −11 dBm, its V_∞ below V_{high}: it stops.
6. **Seeds.** Run any demo ten times and report mean ± s.d.

19.7 Chapter summary

- A five-row budget table (distance, P_{dl}, source voltage, activation time, uplink RSSI) predicts the demos' outcome: who sends how often (by energy source) and who loses collisions (by received power).

- The three-node demos are deterministic functions of the budget plus random start-up delays; ten-seed means are the right basis for comparison.
- In the adaptive-ALOHA demo only ~ 4 of 12 nodes contend per frame; collisions are rarer than textbook ALOHA predicts because of capture, SIC and the 5 ms rule.
- Per-node unevenness in that demo is traced to the $< 10\,\text{m}$ path-loss artefact and the max-rule of hybrid mode, not to the protocol.

Exercises

Exercise 1

Rebuild the budget table for the unicast demo's SIC variant (30° beamwidth). Does the 30° beam change any node's gain? (All nodes lie on the boresight.)

Exercise 2

Using the measured source voltages, estimate the ratio of packets sent by Node 0 and Node 1 from the duty-cycle theorem and compare with 46 : 18. Which assumption of the theorem is most violated?

Exercise 3

In the custom demo, the frame size converges to ≈ 8.4 slots with ≈ 3.9 contenders. Evaluate the slot efficiency of this operating point, and compare it with the $S = n$ and $S = 2n$ cases of Section 9.6.

Exercise 4

Design a ten-seed experiment that isolates the effect of the $< 10\,\text{m}$ artefact on the custom demo's throughput (hint: compare node positions drawn from $[5, 40]$ and from $[10, 40]$).

20

Extending Amber: Writing Your Own Protocol

In this chapter you will learn

- how the adaptive-ALOHA demo implements a complete user protocol with two small pieces of code and no engine changes;
- how to write and evaluate a new base-station policy, through a measured case study of *energy-aware polling*;
- a checklist for building experiments, and a gallery of the commonest mistakes with their symptoms;
- where the model invites extension.

20.1 Anatomy of a user protocol: adaptive ALOHA

The shipped `custom_protocol` demo is the template for everything in this chapter. Its docstring states the contract: "Nothing in Amber's engine is edited." It consists of a *node half* and a *base-station half*.

20.1.1 Node half

We met it in Chapter 16: subclass `BackscatterModule` and override two methods.

```
class AdaptiveAlohaNode(backscatter.BackscatterModule):
    def handle_command(self, cmd, bs_id, data):
        if cmd == "collect":              # my protocol's one command
            self.chosen_slot_idx = (random.randint(0, len(self.rx_slots) - 1)
                                    if self.rx_slots else -1)
            self.state = "active"
            self.last_tx_command_time = self.env.now

    def build_tx_payload(self, sensor_data):
        return ({"type": "data", "payload": sensor_data}
                if self.state == "active" else None)
```

The protocol vocabulary is whatever strings you choose for `cmd` (here "`collect`"). There is no registration phase: the first `collect` activates a node. The framework supplies the sensitivity gate, the slot validity checks and the uplink.

20.1.2 Base-station half

The policy is a generator that yields one frame at a time and adapts between frames.

```
def adaptive_aloha_policy(bs):
    n_slots, prev_collided, load_ema = START_SLOTS, 0, 0.0
    while True:
        yield [("tx", 5, "collect", {"cmd": "collect", "target": -1})] + \
              [("rx", 8, "slot") for _ in range(n_slots)]
        # execution resumes here after the whole frame has played out
        heard = len(bs.decoded_this_frame)
        collided = bs.total_collided - prev_collided
        prev_collided = bs.total_collided
        load_ema = 0.75 * load_ema + 0.25 * (heard + collided)
        target = min(MAX_SLOTS, max(MIN_SLOTS, round(2 * load_ema)))
        if n_slots < target:       n_slots += 1
        elif n_slots > target + 1: n_slots -= 1
```

Read it as a controller. The *measurement* is the number of contenders $m = \text{heard} + \text{collided}$ in the last frame; it is smoothed with an exponential moving average ($\alpha = 0.25$) to avoid chasing noise; the *set-point* is $S^\star = 2\ell$ (Section 9.6), clamped to $[2, 32]$; the *actuator* moves S by at most one per frame, with a one-slot dead band above the target (so S does not oscillate around the target). **Command and slots are yielded in the same frame**, a requirement of the look-ahead (Chapter 17).

20.1.3 Wiring

```
nodes_modules = [AdaptiveAlohaNode(env=env, node=n, bs_processes=[],
                                   uplink_results=ul, downlink_results=dl)
                 for n in nodes]
bs_behavior = bsengine.BSBehavior(env=env, base_station=bs,
                                  policy=adaptive_aloha_policy,
                                  backscatter_modules=nodes_modules,
                                  enable_sic=True)
for m in nodes_modules: m.bs_processes = [bs_behavior]
```

Everything else (capacitors, controllers, coverage maps) is identical to the other demos (Chapter 13).

20.2 Case study: does energy-aware polling help?

The hypothesis. In Chapter 11 we argued that a collision-free polling network is *energy-limited*: the base station polls faster than nodes can recharge, so most polls fall on sleeping nodes ($\approx 91\%$ in the three-node unicast demo). A natural optimisation is to *avoid polling nodes that just failed to answer*: they are probably recharging. If throughput were limited by airtime, this should help substantially; if it is limited by harvested energy, it cannot help much. This is a clean test of the claim.

The policies. Both are written as base-station policies on the unmodified engine. A frame is one poll: `("tx", 5, "poll", {"cmd": "send_data", "target": i})` followed by `("rx", 10, "slot", {"expect": i})`, i.e. 15 ms per poll, so the airtime budget is $\lfloor 20\,001/15 \rfloor = 1333$ polls in 20 s for *both* policies.

```
def policy(bs):                          # skip_silent (penalty = P polls)
    skip = {i: 0 for i in range(N)}; i = 0
    while True:
        if skip[i] > 0:                  # recently silent: leave it alone
```

Table 20.1: Round-robin versus skip-silent polling, $N = 12$ nodes, 20 s, mean over 10 seeds. Paired differences use the same seeds.

Policy	Decoded	Polls	Decoded/poll	Throughput	Gain vs RR
round-robin	667.2	1333	0.501	33.4 pkt/s	–
skip, penalty 2	684.7	1333	0.514	34.2 pkt/s	+2.6% ($t = 11.6$)
skip, penalty 4	683.6	1333	0.513	34.2 pkt/s	+2.5% ($t = 8.1$)
skip, penalty 8	680.6	1333	0.511	34.0 pkt/s	+2.0% ($t = 5.4$)
skip, penalty 16	679.5	1333	0.510	34.0 pkt/s	+1.8% ($t = 4.2$)

```
            skip[i] -= 1; i = (i + 1) % N; continue
        yield [("tx", 5, "poll", {"cmd": "send_data", "target": i}),
               ("rx", 10, "slot", {"expect": i})]
        if i not in bs.decoded_this_frame:      # no answer -> penalise
            skip[i] = P
        i = (i + 1) % N
```

Round-robin is the same without the `skip` logic. Scenario: $N = 12$ registered nodes, hybrid energy mode with the solar trace, placed uniformly at random in an annulus $r \in [10, 40]$ m (to stay clear of the < 10 m artefact), $\eta = 0.7$, the demo electrical parameters, 20 s horizon, SIC on; 10 seeds per policy (script `textbook/code/polling_experiment.py`).

Result. Skipping silent nodes helps, but only by $\approx$ 2–3%: small, yet statistically solid (t between 4 and 12 in the paired comparison). Half of all polls are answered even by round-robin, and the smartest policy we tried answers 51.4%. Each node decodes about 55–60 packets in 20 s ($\approx$ 2.8 per second) whatever the policy; the total $\approx$ 34 pkt/s is *half of the airtime capacity of* $1333/20 = 66.7$ polls/s.

Interpretation. The hypothesis was right in the way that matters: the policy cannot beat the energy bound, because the number of packets a node can send is set by how fast its capacitor refills, not by which polling order the base station uses. All the policy decides is *which node spends its next burst now*; avoiding sleepers saves airtime but airtime was not scarce. Think of the nodes as taps filling buckets at fixed rates: re-ordering how you visit the taps does not increase the water you collect per hour. To move the needle one must change the *energy side*: better harvesting, a larger C or lower V_{low} (longer bursts), lower per-packet energy, or a transmission cap that respects the duty-cycle theorem. This is exactly the sort of conclusion a simulator that couples energy and protocol is built to expose.

Interpret small effects with care

We changed one thing; the policies share seeds, so the paired comparison removes most of the placement variance. Nevertheless the node population is homogeneous (identical sources), the poll order is rigid, and the model's energy accounting has the caveats of Chapter 8 (over-credited harvesting while active, brown-out truncation). The qualitative conclusion—*energy dominates*—is robust to these; the 2.6% is specific to this configuration.

20.3 A checklist for new experiments

1. **State the question** and the metric (Chapter 18); decide what you will compare against.

2. **Fix the scenario** (geometry, power, energy mode, electrical parameters) in one config; keep nodes ≥ 10 m from masts.
3. **Make runs reproducible**: seed Python's `random`; log the seed.
4. **Wire the eight steps** of Chapter 13; set `bs_processes`.
5. **Warm up**: nodes need $\approx$ 2–4 s to charge and the start-up delay is up to 2 s; start protocol measurements after that or account for it.
6. **Keep the base-station process alive** (long final window).
7. **Replicate**: ≥ 10 seeds, report mean and spread; use *paired* seeds when comparing designs.
8. **Change one factor at a time**; the shipped demos change six.
9. **Sanity-check with theory**: a link-budget table, an RC charge time, an ALOHA estimate. If the simulator disagrees, find out why before trusting it.
10. **Record the caveats** of Chapter 21 that apply to your result.

20.4 A gallery of mistakes

Symptom	Likely cause	Fix
Nodes "transmit" (TX records exist) but the base station logs nothing	`bs_processes` never set (Step 7)	`m.bs_processes=[beh]` for every module
`RuntimeError:...hasterminatedandcannotbeinterrupted`	base-station process ended (`loop=False`, exhausted policy) while nodes hold slots	end with a long idle RX window
`AttributeError:harvesting_power_dbm`	controller built before `calculate_node_power`	call it before creating controllers
Nodes never activate	$V_\infty < V_{\text{high}}$ (RF only, too far, or too little power)	compute Eq. (8.3); add an external source
Registration stalls at $\approx$half per round	two-frame look-ahead	expect factor $\frac{1}{2}$; use more rounds or slots
`send_data` to a list gets no replies	default handler ignores list targets	override `handle_command`
Changing `R_series` changes nothing	controller uses a hard-coded 5 kΩ lookup	set both places, or patch the lookup
Different results on each run	unseeded randomness	`random.seed(...)`
Memory grows without bound	`keep_logs=True`	`keep_logs=False`, `stream_log`
Odd extra throughput for nodes near the mast	< 10 m path-loss branch	keep nodes ≥ 10 m
`analyze_packets` says 1 of 124 packets decoded	zero-length-interval defect in post-hoc detection	use the engine's `collided` flags

20.5 Where the model invites extension

The architecture makes the following research directions natural; each is a replacement of one module behind the contracts of Section 13.2.

- **A realistic harvester**: an RF–DC efficiency curve with a turn-on threshold and cold-start (replace the conversion in `calculate_node_power`).

- **Imperfect SIC**: add the decoded packets' residual to the interference sum (Chapter 10).
- **Energy-causal transmissions**: refuse to start a burst the capacitor cannot finish, or count truncated bursts as failures (Chapter 15).
- **Fading and shadowing**: replace the constant link budget with a per-packet random draw (this requires giving up the one-time pre-computation).
- **Mobility**: recompute channel entries as nodes move, or per slot.
- **Wake-up receivers** and non-zero idle current in *wait_slot*.
- **Smarter protocols**: energy-state reporting, schedule prediction from the node's recharge time, multi-carrier or multi-reader cooperation.

20.6 Chapter summary

- A complete protocol is a node subclass (two methods) plus a base-station policy generator.
- The energy-aware polling case study improved decoded packets by only $\approx$ 2–3% because throughput is bounded by harvested energy.
- A short checklist and a table of symptoms and causes will save hours.
- The module contracts of Chapter 13 define where extensions plug in.

Exercises

Exercise 1

Implement a node protocol in which the node replies only if it received the command with at least 3 dB margin over its sensitivity, and justify the rule physically.

Exercise 2

Modify the polling policy so that the penalty grows exponentially with consecutive silent polls (exponential back-off). Predict, using the duty-cycle theorem, whether this can exceed the $\approx$ 34 pkt/s bound.

Exercise 3

Write a base-station policy that alternates a discovery frame and a polling round and stops discovering once all N nodes are registered. List the `BSBehavior` attributes you read.

Exercise 4

Add a rule so that a node starts a transmission only if $V \geq V_{\text{low}} + \Delta V_{\text{burst}}$. How would you obtain the voltage in the node protocol given the contracts?

21

Assumptions, Limits and Verified Caveats

In this chapter you will learn

- a catalogue of every modelling assumption and code behaviour discussed in the book, in one place, each with the evidence for it;
- which ones can change a headline result and which are cosmetic;
- a checklist of validity questions to ask of any Amber result;
- where to go next.

How to read this chapter. Amber is version 0.1.0 of a research prototype, described by a two-page demo abstract. It is natural for such a tool to contain simplifications and rough edges, and the authors state its scope honestly. The point of cataloguing them is not criticism but *calibration*: you should know which results you may trust. Each row below states the behaviour, the **evidence** (*measured*: we ran an experiment; *tested*: a targeted test of the code; *read*: established by reading the source), and the **effect**. We did *not* modify the Amber source in preparing this book.

21.1 Catalogue

21.1.1 Channel and radio

#	Behaviour	Evidence	Effect (severity)
R1	UMa LOS falls through to the post-breakpoint branch for $d_{2D} < 10$ m: a 13.8 dB step at 10 m (Ch. 5)	*tested*: 44.4 vs 58.3 dB at 9.99/10 m	Nodes < 10 m get ~ 14 dB too much RF; produced 2× throughput for 4 of 12 nodes in the custom demo (Ch. 19). **High** near masts
R2	FSPL mode uses d_{2D} not d_{3D}	*tested*	14 dB error at 5 m, 0.2 dB at 100 m. **Low**
R3	Horizontal antenna pattern only; no downtilt (Ch. 4)	*read*	Overestimates gain for nodes close to the mast. **Medium** near masts
R4	Deterministic static channel: no fading, shadowing, mobility	*read* (by design)	Realistic variability absent; results are optimistic and smooth. **Design scope**
R5	A node harvests the best sector only; no carrier summation or inter-cell interference	*read* (Ch. 14)	Conservative for energy; no downlink interference. **Low–Medium**

#	Behaviour	Evidence	Effect (severity)
R6	`CoverageMap.bandwidth_hz`, `noise_figure_db` unused	*read* (grep)	Cosmetic; noise comes from the base station's own arguments. **Low**
R7	Sensitivity thresholds are independent of the noise floor; with default 100 MHz the noise (−88 dBm) exceeds sector sensitivity (−100 dBm)	*read*, *tested*	Set both consistently. **Medium**
R8	A lone packet above sensitivity is decoded regardless of SNR; noise enters only collisions	*tested*: −105 dBm, SNR −17 dB decoded	Weak-link PER is not modelled. **Medium**
R9	Command reception uses the best power over *all* base stations, not the sender's	*read*	Multi-BS command coverage is optimistic. **Low**
R10	Subcarrier shifts are perfectly orthogonal; no carrier leakage or sideband loss (Ch. 6)	*read*	Self-interference—the central practical backscatter issue—is not simulated. **Design scope**

21.1.2 Energy and node behaviour

#	Behaviour	Evidence	Effect (severity)
E1	No RF–DC efficiency curve, turn-on threshold or cold start (Ch. 7)	*read*	Energy availability is optimistic at low RF power. **High** for RF-limited studies
E2	Controller converts power to voltage with `getattr(cap,"R_series",5e3)`, which always returns 5 kΩ	*tested*	Changing R_s in `CapacitorParams` silently mismatches the source voltage. **Medium** (latent)
E3	Charge-and-discharge step credits $V_{\text{src}} I \Delta t$ (source energy), not $V I \Delta t$ (delivered): over-credit by V_{src}/V	*tested*, *measured*: 1.948 V vs 1.695 V after 1 s; −2.7% packets when corrected	Harvest overstated while active. **Medium**
E4	Packet is delivered at *entry* to the transmit state; the energy is paid afterwards. 43–50% of transmit states end in brown-out	*measured* (3 nodes, 20 s)	Delivered counts are optimistic by up to $\sim 2\times$ under tight energy budgets. **High**
E5	*wait_slot* draws zero current	*read*	Idle/wake-up-receiver cost ignored. **Medium**
E6	Energy gates transmission, not reception; protocol state survives power-down	*tested*: module with no capacitor accepts commands	Registration/slot state of "dead" nodes changes; real nodes would hear nothing. **Medium**

#	Behaviour	Evidence	Effect (severity)
E7	Shipped solar trace is 2999 rows $= 3$ s, looped; file named "300 seconds"; shared by all nodes	*measured*	Spurious periodicity possible; no spatial shading. **Low**
E8	`Capacitor.load_discharging` / cause `capacitor_discharge` never used by the shipped controller	*read* (grep)	Dead code path. **Cosmetic**
E9	Demo loads (~ 5 mA) are 10^3–$10^4\times$ above Release-19 $\sim 1\,\mu$W targets	*read*	Demos are not representative of ultra-low-power tags. **Interpretation**

21.1.3 MAC, SIC and software

#	Behaviour	Evidence	Effect (severity)
M1	`cancellation_factor` has no effect: SIC is perfect cancellation (Ch. 10)	*tested*: identical decode sets for several κ from 0 to 1	Imperfect SIC not simulated; SIC gain overstated. **High** for SIC studies
M2	Collision = arrival within 5 ms (greedy, non-transitive), packets have no duration	*tested*: 10, 14, 18 ms $\Rightarrow$ third packet escapes	Collision statistics are model-dependent. **Medium**
M3	Advertised RX slots cover *two* frames; random pick over $2S$ $\Rightarrow$ $\approx \frac{1}{2}$ participation per frame (Ch. 17)	*tested*: 8 advertised for a 4-slot frame; *measured*: 44/70/82/88% registered over rounds at $n = 5$	Slows discovery by $\sim 2\times$ per round; must be budgeted. **High** for MAC studies
M4	`TxRecord.delivered` is True when handed to any BS process; -150 dBm packet is "delivered"	*tested*	Not a delivery metric. **Medium**
M5	`send_data` with a list `target` accepted by BS, ignored by default node handler	*tested*	Multicast silently fails. **Low**
M6	ACK is a direct call, always received; BS marks registered at send	*read*	Real ACK loss absent. **Medium**
M7	A terminated BS process makes a later node transmission raise `RuntimeError`	*tested* (crash reproduced)	Aborts the simulation. **Medium** (usability)
M8	"DER" means error rate in demos, success ratio in `BSStats.der`	*read*, *tested*	Easy to misreport. **Medium**

#	Behaviour	Evidence	Effect (severity)
M9	Post-hoc `detect_collisions` mislabels almost all packets on engine logs (zero-length intervals)	*tested*: 1 vs 121 decoded; repaired: 120	Only affects logs without `collided`. **Medium**
M10	`analyze_packets` appends text files to the working directory; rejects demo CSVs	*read*, *tested*	Side effects; unusable on demo output. **Low**
M11	Randomness is unseeded by default (only the custom demo seeds)	*read*	Run-to-run variation; replicate. **Medium**

21.2 Which caveats matter for which question?

If your question concerns…	Check first…
Absolute throughput / delivery	E4, E3, E1, R1, M4
Registration / discovery speed	M3, M2, M6, E6
SIC / collision resolution	M1, M2, R8, R10
Energy-limited operation, duty cycle	E3, E4, E5, E2, E7
Coverage and range	R1–R5, R7
Relative protocol comparison under identical assumptions	usually robust; replicate (M11) and use paired seeds

21.3 Ten validity questions for any simulation result

1. Is the result a *comparison* under shared assumptions, or an *absolute* number? (Prefer the former.)
2. How many seeds, and what is the spread?
3. Do the nodes at the edge of the scenario sit in a region where the channel model is valid ($d_{2D} \geq 10\,\text{m}$)?
4. Is the source of each node's energy what you think it is (RF, solar, max, sum)?
5. Is throughput limited by energy, by airtime or by collisions? Does a hand calculation agree?
6. Does "delivered" mean decoded at a base station? Counted before or after the energy was paid?
7. Are SIC and capture assumptions realistic for your receivers?
8. Did you change one factor at a time?
9. Could the effect you see come from the 5 ms rule, the look-ahead or the node's start-up delay rather than the mechanism you study?
10. Would the conclusion survive a ± 3 dB change in the link budget and a $\pm 30\%$ change in the harvest?

21.4 Where to go next

- **Standards:** 3GPP TR 38.848, TR 38.769, TS 38.191 [1–3]; the channel model TR 38.901 [4].
- **Overview:** López et al. [15]; the backscatter survey of Huynh et al. [12]; the dual-tone downlink idea [25].
- **Textbooks:** Dobkin on UHF RFID [8]; Tse and Viswanath for SIC and multiple access [24]; Banks et al. or Law for simulation methodology [6, 13].
- **The simulator:** the repository, the generated API reference from the docstrings, and the SenSys demo abstract [17].
- **Your own experiments:** Chapter 20, with the scripts in `textbook/code`.

21.5 Chapter summary

- We catalogued 10 channel/radio, 9 energy/node and 11 MAC/software behaviours, each with its evidence.
- The most consequential for headline numbers: brown-out truncation (E4), perfect SIC (M1), the two-frame look-ahead (M3), the < 10 m path-loss branch (R1) and the missing RF–DC curve (E1).
- Relative comparisons under identical assumptions are the safest use of the simulator; always replicate.

Exercises

Exercise 1

Pick any *High* item and propose a minimal code change that removes it. State what you would measure to show the change had the intended effect.

Exercise 2

A paper reports "Amber shows SIC raises throughput by 40%". List the three caveats from this chapter you would check first, and the experiment you would run.

Exercise 3

Using the validity questions, critique the claim "the adaptive ALOHA policy gives node 4 twice the throughput of node 1 because it is better located".

A

Derivations

This appendix gathers, in compact but complete form, derivations that the main text states or sketches.

A.1 Solution of the capacitor equation

Equation (8.2): $C\dot{V} = (V_{\text{src}} - V)/R_{\text{s}} - V/R_{\text{l}} - I_L$. Collect the terms in V:

$$C\dot{V} + \Big(\frac{1}{R_{\text{s}}} + \frac{1}{R_{\text{l}}}\Big)V = \frac{V_{\text{src}}}{R_{\text{s}}} - I_L \quad\Longrightarrow\quad \dot{V} + \frac{V}{\tau} = \frac{V_\infty}{\tau},$$

with $\tau = C/(1/R_{\text{s}} + 1/R_{\text{l}}) = (R_{\text{s}} \parallel R_{\text{l}})C$ and $V_\infty = \tau(V_{\text{src}}/R_{\text{s}} - I_L)/C = (V_{\text{src}} - I_L R_{\text{s}})\, R_{\text{l}}/(R_{\text{l}} + R_{\text{s}})$. Multiplying by the integrating factor $\mathrm{e}^{t/\tau}$:

$$\frac{\mathrm{d}}{\mathrm{d}t}\big(V\mathrm{e}^{t/\tau}\big) = \frac{V_\infty}{\tau}\mathrm{e}^{t/\tau} \;\Rightarrow\; V(t)\mathrm{e}^{t/\tau} = V_0 + V_\infty(\mathrm{e}^{t/\tau} - 1) \;\Rightarrow\; V(t) = V_\infty + (V_0 - V_\infty)\mathrm{e}^{-t/\tau}.$$

Setting $V(t_{\text{th}}) = V_{\text{th}}$ and solving gives Eq. (8.6).

Discrete step. Explicit Euler on the source term alone gives $V_{k+1} = V_k + \frac{\Delta t}{R_{\text{s}}C}(V_{\text{src}} - V_k)$, with amplification factor $1 - \Delta t/(R_{\text{s}}C) \in (0,1)$ when $\Delta t < R_{\text{s}}C$ (stable, here $\Delta t/R_{\text{s}}C = 6.7 \times 10^{-4}$). Leakage alone is $V \to V\mathrm{e}^{-\Delta t/R_{\text{l}}C}$ exactly. Composing them (Eq. (8.7)) commits an $O(\Delta t^2)$ local error per step, hence $O(\Delta t)$ globally, with a coefficient of order $\Delta t/\tau \sim 10^{-3}$: we measured an error of 1.7×10^{-4} V after 1 s.

A.2 Maximum power into the capacitor

Power into the capacitor through R_{s} at voltage V is $p(V) = V(V_{\text{src}} - V)/R_{\text{s}}$. Then $p'(V) = (V_{\text{src}} - 2V)/R_{\text{s}} = 0$ at $V = V_{\text{src}}/2$ and $p_{\max} = V_{\text{src}}^2/(4R_{\text{s}}) = P_h/4$, with $P_h = V_{\text{src}}^2/R_{\text{s}}$. Equivalently $p = P_h\, x(1-x)$, $x = V/V_{\text{src}}$.

A.3 Duty-cycle theorem

See Section 8.5. With $T_c = \Delta E/P_{\text{in}}$, $T_a = \Delta E/(P_{\text{act}} - P_{\text{in}})$: $D = T_a/(T_a + T_c) = P_{\text{in}}/P_{\text{act}}$, because $\dfrac{1}{P_{\text{act}} - P_{\text{in}}} \Big/ \Big(\dfrac{1}{P_{\text{act}} - P_{\text{in}}} + \dfrac{1}{P_{\text{in}}}\Big) = \dfrac{P_{\text{in}}}{P_{\text{in}} + (P_{\text{act}} - P_{\text{in}})}$.

A.4 Framed slotted ALOHA: slot statistics

Each of n nodes picks one of S slots independently and uniformly. The occupancy X of a given slot is Binomial$(n, 1/S)$: $\Pr[X = k] = \binom{n}{k} S^{-k}(1 - 1/S)^{n-k}$. Hence $\Pr[X = 0] = (1 - 1/S)^n$,

$\Pr[X=1] = \frac{n}{S}(1-1/S)^{n-1}$. By linearity of expectation over the S slots, $E_0 = S\Pr[X=0]$, $E_1 = S\Pr[X=1] = n(1-1/S)^{n-1}$ and $E_c = S - E_0 - E_1$. A tagged node succeeds iff each of the other $n-1$ nodes avoided its slot: $(1-1/S)^{n-1}$.

Optimal frame size. Let $r = n/S$. For large S, $(1-1/S)^{n-1} \to \mathrm{e}^{-r}$, so $\eta_S \to r\mathrm{e}^{-r}$, with $\mathrm{d}\eta_S/\mathrm{d}r = (1-r)\mathrm{e}^{-r} = 0$ at $r=1$ and $\eta_S(1) = 1/\mathrm{e}$.

Pure and slotted ALOHA. A transmission starting at t_0 is vulnerable to any other start in $(t_0 - T, t_0 + T)$ (pure) or in the same slot (slotted). With Poisson arrivals of rate g per packet time, the probability of no other start in an interval of length $2T$ (resp. T) is e^{-2g} (resp. e^{-g}). The maximum of $g\mathrm{e}^{-2g}$ is at $g = 1/2$ ($1/2\mathrm{e}$) and of $g\mathrm{e}^{-g}$ at $g = 1$ ($1/\mathrm{e}$).

A.5 SIC: the ratio rule

For perfect cancellation and negligible noise, powers $P_{(k)} = P_{(1)}\rho^{-(k-1)}$ with $\rho > 1$. At stage k the interference is the sum of all weaker packets: $P_{(k)}\sum_{j\geq 1}\rho^{-j}$ for an infinite chain, which converges to $P_{(k)}/(\rho-1)$. Thus $\gamma_{(k)} = \rho - 1$ for every k, and decoding all packets requires $\rho - 1 \geq \gamma_{\mathrm{req}}$, i.e. $\rho \geq 1 + \gamma_{\mathrm{req}}$ (in linear units). For a finite chain of m packets the ratio at the last stages is larger, so $\rho \geq 1 + \gamma_{\mathrm{req}}$ is sufficient. With $\gamma_{\mathrm{req}} = 3$ dB (≈ 2.0) we need $\rho \geq 3.0$ (4.8 dB). The strongest of four packets exceeds the weakest by $\rho^3 = 27$ (14.3 dB).

A.6 Sidebands of a switched reflection

Let $s(t) \in \{0,1\}$ be a unit square wave with period $1/f_s$ and 50% duty. Its Fourier series is $s(t) = \frac{1}{2} + \frac{2}{\pi}\sum_{k=1,3,5,\ldots}\frac{1}{k}\sin(2\pi k f_s t)$. The reflection coefficient is $\Gamma(t) = \Gamma_0 + \Delta\Gamma\, s(t)$ and the reflected wave is $y(t) = \Gamma(t)\, A\cos(2\pi f_c t)$. Using $\sin a\cos b = \frac{1}{2}[\sin(a+b) + \sin(a-b)]$ on the fundamental ($k=1$) term:

$$\Delta\Gamma\frac{2}{\pi}A\sin(2\pi f_s t)\cos(2\pi f_c t) = \frac{\Delta\Gamma}{\pi}A\big[\sin 2\pi(f_c+f_s)t - \sin 2\pi(f_c - f_s)t\big].$$

Each sideband has amplitude $\Delta\Gamma A/\pi$, hence a power ratio of $(\Delta\Gamma/\pi)^2$ to the incident carrier (A). For $\Delta\Gamma = 1$ this is $1/\pi^2 = 0.101$ (-9.9 dB); for $\Delta\Gamma = 2$ it is $4/\pi^2 = 0.405$ (-3.9 dB). The constant term $\frac{1}{2}\Delta\Gamma + \Gamma_0$ multiplies the carrier itself, giving the unshifted "static" reflection of Eq. (6.6).

A.7 Friis in two lines

Isotropic power density at range d: $S = P_t G_t/(4\pi d^2)$. Effective aperture of a receive antenna of gain G_r: $A_e = G_r\lambda^2/(4\pi)$ (a standard result obtained from antenna reciprocity and thermal equilibrium; see, e.g., Pozar [18]). The received power is $SA_e = P_t G_t G_r(\lambda/4\pi d)^2$.

A.8 3GPP UMa continuity at the breakpoint

At $d_{2D} = d'_{BP}$ and ignoring the height difference in d_{3D}, $\mathrm{PL}_1 = 28 + 22\log d'_{BP} + 20\log f_c$ and $\mathrm{PL}_2 = 28 + 40\log d'_{BP} + 20\log f_c - 9\log(d'^2_{BP} + \Delta h^2) \approx 28 + 40\log d'_{BP} + 20\log f_c - 18\log d'_{BP} = 28 + 22\log d'_{BP} + 20\log f_c$ when $\Delta h \ll d'_{BP}$: the two branches agree, confirming the continuity claim. For the demo ($d'_{BP} = 147.8$ m) we measured 75.17 and 75.19 dB at 147.8 and 148 m.

B

Formula and Parameter Reference

B.1 Formula sheet

Quantity	Formula
dBm to W	$P = 10^{(P_{\text{dBm}}-30)/10}$
Link budget	$P_{\text{rx}} = P_{\text{tx}} + G_{\text{tx}} - \text{PL} + G_{\text{rx}}$ (dB)
Thermal noise	$N = k_B T_0 B F$; $-174 + 10\log B + \text{NF}$ dBm
Friis	$P_r = P_t G_t G_r (\lambda/4\pi d)^2$; $\text{PL}_{FS} = 20\log(4\pi d/\lambda)$
3GPP sector gain	$G = G_{\max} - \min[12(\varphi/\varphi_{3\text{dB}})^2, 30]$
UMa LOS	$\text{PL}_1 = 28 + 22\log d_{3D} + 20\log f_c$ $(10 \le d_{2D} \le d'_{BP})$; $\text{PL}_2 = 28 + 40\log d_{3D} + 20\log f_c - 9\log(d'^2_{BP} + \Delta h^2)$
Breakpoint	$d'_{BP} = 4(h_{BS}-1)(h_{UT}-1) f_c / c$
Backscatter uplink	$P_r = P_t + G_t + G_r + 2G_a + 10\log\eta - \text{PL}(d_1) - \text{PL}(d_2)$ (dB)
Harvest (WPT mode)	$P_h = P_{\text{dl}}(1-\eta\,\mathbb{1}_{\text{tx}})$; $V_{\text{src}} = \sqrt{P_h R_{\text{s}}}$
Capacitor	$C\dot{V} = (V_{\text{src}} - V)/R_{\text{s}} - V/R_{\text{l}} - I_L$; $V(t) = V_\infty + (V_0 - V_\infty)\mathrm{e}^{-t/\tau}$
Steady state, τ	$V_\infty = (V_{\text{src}} - I_L R_{\text{s}})R_{\text{l}}/(R_{\text{l}} + R_{\text{s}})$; $\tau = (R_{\text{s}} \| R_{\text{l}})C$
Charge time	$t = \tau \ln[(V_\infty - V_0)/(V_\infty - V_{\text{th}})]$
Energy window	$\Delta E = \tfrac{1}{2}C(V^2_{\text{high}} - V^2_{\text{low}})$
Duty cycle	$D = P_{\text{in}}/P_{\text{act}}$
Slotted ALOHA	$s = g\mathrm{e}^{-g}$; FSA: $E_1 = n(1-1/S)^{n-1}$, $P_{\text{succ}} = (1-1/S)^{n-1}$
SINR	$\gamma_i = P_i/(\sum_{j\neq i} P_j + N)$; SIC: $\gamma_{(k)} = P_{(k)}/(\sum_{j>k} P_{(j)} + \varepsilon\sum_{j<k} P_{(j)} + N)$
Mean-field registration	$n_{k+1} = n_k - n_k p_a\,(1 - p_a/SK)^{n_k - 1}$

B.2 Default values at a glance

Parameter	Class	Default	Unit / note
`dt`	`CapacitorParams`	`1e-3`	s (the simulator tick is 1 ms)
`R_series`	`CapacitorParams`	`5e3`	Ω (the controller assumes 5e3)
`R_leakage`	`CapacitorParams`	`0.1e6`	Ω
`C`	`CapacitorParams`	`300e-6`	F
`initial_voltage`	`Capacitor`	`0.0`	V
`voltage_max`	`Capacitor`	`2.0`	V clamp
`keep_logs`	`Capacitor etc.`	`True`	memory grows if True
`max_startup_time_ms`	`ControllerParams`	`2000`	ms; delay uniform in $[0, \cdot)$
`thresholds`	`VoltageThresholdsV`	`(set by user)`	V; demo high 1.7, low 1.3
`currents, durations`	`CurrentsA, DurationsMs`	`(set by user)`	A, ms; demo in Ch. 15
`efficiency`	`Node`	`1.0`	η
`sensitivity_dbm`	`Node`	-45	dBm
`height`	`Node`	`1.5`	m
`antenna_gain_dbi`	`Node`	`0.0`	dBi
`subcarrier_shift`	`Node`	`0`	integer label
`power`	`Sector`	`43.0`	dBm
`antenna_gain_dbi`	`Sector`	`17.0`	dBi
`height`	`Sector`	`25.0`	m
`sensitivity_dbm`	`Sector`	-100	dBm
`freq_hz`	`CoverageMap`	`925e6`	Hz
`pathloss_model`	`CoverageMap`	`"macro"`	or "fspl"
`node_energy_mode`	`CoverageMap`	`"wpt"`	or "external", "hybrid"
`combine_mode`	`CoverageMap`	`"max"`	hybrid combination
`required_sinr_db`	`BSBehavior`	`3.0`	dB
`enable_sic`	`BSBehavior`	`True`	
`cancellation_factor`	`BSBehavior`	`0.9`	*inert*
`noise_figure_db`	`BSBehavior`	`6.0`	dB
`bandwidth_hz`	`BSBehavior`	`100e6`	Hz
`collision_window_ms`	`_process_rx_buffer`	`5.0`	ms (not a constructor argument)
`resistance`	`EnvEnergySource`	`1.0`	Ω (demos pass 5e3)
`column`	`EnvEnergySource`	`"V_IM"`	trace column

B.3 The demos' electrical and timing parameters

Item	Value	Source
Carrier frequency	924 MHz	all demos
Path loss	UMa LOS	all demos
Thresholds $V_{\text{high}}/V_{\text{low}}$	1.7/1.3 V	all demos
$R_s/R_l/C$	5 kΩ/100 kΩ/300 µF (200 µF in SIC demo)	demos
Listening	5 ms, 0.14 mA	demos
Sensing	2 ms, 0.512 mA	demos
Processing	5 ms, 1.28 mA	demos
Transmitting	15 ms, 5 mA	demos
TX command / RX slot	5 ms / 10 ms (15 in SIC demo; 8 in custom)	demos
Sector	46 dBm, 15 dBi, azimuths 0°, 120°, 240°	demos
Paper's experiment	30 dBm, 17 dBi, nodes 1 dBi	paper

C

The Paper, Annotated

This appendix reads the SenSys demo abstract [17] *statement by statement*, linking each claim to the chapter that explains it, the code that implements it, and our verdict from reading and running the code. Quotations are from the paper. Verdict symbols: ✓ consistent with the code; ∼ consistent with caveats; ○ an abstraction (described, not simulated at that level).

C.1 Abstract and introduction

Claim	Book	Code / comment	OK?
"open-source discrete-event simulator for 6G Ambient IoT"	Ch. 1, 12	Python/SimPy, BSD-3 licence	✓
"integrates energy harvesting, capacitor storage dynamics, and backscatter communication"	Ch. 7, 8, 6	`CoverageMap`, `Capacitor`, `BackscatterModule`; no RF–DC curve (E1)	∼
3GPP Ambient IoT devices harvest RF and "communicate via backscatter communication"	Ch. 1, 6	*Device A/B/C*; Amber's node ≈ storage-equipped backscatter device	✓
"most existing simulators stop at the signal level and do not support Ambient IoT nodes" (Sionna, ray tracing)	Ch. 1	motivation, not code	–
"Network simulators . . . typically assume always-available power, while energy harvesting studies typically omit protocol-level communication"	Ch. 1	motivation	–
"first open-source simulator that integrates RF energy harvesting, capacitor storage dynamics, and backscatter communication"	Ch. 1	the authors' claim "to our knowledge"; we do not assess priority	–

C.2 Section II: Amber overview

Claim	Book	Code / comment	OK?
"system-level simulator designed for large-scale 6G Ambient IoT networks"	Ch. 2	ms-resolution, hundreds of nodes (measured: 300 nodes $\approx 4\times$ real time)	✓
Physically grounded: "voltage- and current-based capacitor models … series resistance, leakage current, and simultaneous charge/discharge"	Ch. 8	`Capacitor`: four step types; exact RC for charge-only; active-mode over-credit (E3)	∼
"free-space path loss (FSPL) or the 3GPP TR 38.901 UMa scenario, together with 3GPP-compliant sectored antenna patterns"	Ch. 4, 5	horizontal pattern only (R3); UMa < 10 m branch (R1); FSPL uses d_{2D} (R2)	∼
"Link budgets … Friis … configurable antenna gains, carrier frequency, and medium attenuation"	Ch. 3, 5	`compute_rx_power_watts`, `extra_np_per_m`	✓
Modularity: "each module as an independent class with defined interfaces"	Ch. 13	duck-typed contracts; two override seams	✓
Energy awareness: "All simulated events are conditioned on the node's instantaneous energy state"	Ch. 15, 16	transmissions are; *receptions* are not (E6)	∼
"tracks capacitor voltage at millisecond resolution … charging and active duty-cycling modes based on configurable voltage thresholds"	Ch. 8, 15	`Controller.run`, $V_{\text{high}}/V_{\text{low}}$	✓
"Current consumption is modeled separately for each operational state (listening, sensing, processing, transmitting)"	Ch. 15	`CurrentsA`, `DurationsMs`; *wait_slot* is a fifth, zero-load state (E5)	✓
Scalability: "Built on SimPy … 1 ms temporal resolution, using interrupt-driven communication"	Ch. 12	verified; interrupts are *urgent*	✓
"coverage maps and link budgets are pre-computed once … enabling efficient simulation of hundreds of nodes"	Ch. 12, 14	precompute ∼ms; run $\approx 13\,\mu$s/node-ms	✓
"three entity types: nodes, base stations, and the communication links"	Ch. 2	dataclasses + dicts; links are tables, not objects	✓

C.3 Section II (continued): the physical layer and devices

Claim	Book	Code / comment	OK?
"Physical-layer communication is based by default on the PRDCH structure … dual-tone principle: an external unmodulated carrier occupies a dedicated OFDM subcarrier, enabling both energy harvesting and backscatter reflection, while a single data-bearing subcarrier carries downlink information"	Ch. 6	carrier is implicit/always on; commands are function calls gated by RSSI; no waveform	○
Nodes "characterized by position, antenna gain, receiver sensitivity, and a power split ratio"	Ch. 6, 14	`Node`: `x`,`y`, `antenna_gain_dbi`, `sensitivity_dbm`, `efficiency`	✓
"above V_{high}, the node cycles through listening, sensing, processing, and transmitting states; below V_{low}, it enters a charge-only state"	Ch. 8, 15	hysteresis exactly as described	✓
"harvesting energy via wireless power transfer, an external source such as solar, or a hybrid of both"	Ch. 7	modes `wpt`/`external`/`hybrid`; hybrid defaults to *max*	✓
"Base stations consist of multiple sectors, each with configurable azimuth, beamwidth, height, antenna gain, transmit power, receiver sensitivity, and 3GPP-compliant antenna patterns"	Ch. 14	`Sector` fields	✓
"Communication links are computed bidirectionally, downlink for wireless power transfer and carrier provision, uplink for backscattered data"	Ch. 14	`compute_bs_to_point`, `compute_point_to_bs`	✓
"subcarrier shifting as a collision mitigation strategy … Distinct subcarrier offsets provide frequency-domain separation, and SIC resolves collisions among nodes sharing the same offset"	Ch. 6, 10	integer label, orthogonal; SIC perfect (M1)	∼

C.4 Section III: the experiment

Claim	Book	Code / comment	OK?
"three base stations and up to 100 passive nodes, each assigned one of three subcarrier shifts, … 500 m × 500 m"	Ch. 11, 19	not in the repository; our one-BS re-creation	–
"Base stations transmit at 30 dBm with 17 dBi sectorized antennas at 924 MHz, nodes … 1 dBi omni … hybrid power mode"	Ch. 6	used in our re-creation; at these powers RF alone cannot energise nodes ($V_\infty \leq 0.65$ V) so hybrid mode is essential	✓
"a discovery phase, where nodes register by responding with their identifiers in random slots, followed by a data collection phase, where the base station periodically requests sensor readings from each registered node individually by its identifier"	Ch. 11	`send_id/ack/send_data(target)`	✓
"increasing the number of nodes from 5 to 100 … within 100 m … 10 runs"	Ch. 11	reproduced qualitatively (Fig. 11.2)	∼
"registration percentage decreases as the node density increases, leading to a higher collision probability during random access"	Ch. 9, 11	confirmed; our analysis adds the $\frac{1}{2}$-participation effect (M3)	✓
"No collisions occur during data collection since nodes are queried individually"	Ch. 11	verified by construction in the unicast demo	✓
"single unicast polling protocol operating in full-duplex mode"	Ch. 6, 17	carrier continuous; the *schedule* still time-multiplexes TX/RX windows	∼
"By reconfiguring the simulator, we can test different protocols, scheduling strategies, or network topologies"	Ch. 20	node subclass + BS policy	✓
Fig. 1: RSSI coverage map with fixed BSs, random nodes	Ch. 4	`compute_coverage_map`, `plot_rssi`; our own example in Fig. 4.2	✓

C.5 What the abstract does not say

For anyone wishing to *reproduce* its Figure 2 the following are unspecified: the number of reply slots and slot length in the discovery phase, the number of discovery rounds, how subcarrier shifts are assigned (here: nodes i mod 3 in our re-creation), the capacitor and threshold values, the load currents, the exact node placement distribution (uniform in a disc, or by area), and the SIC and capture parameters. Our re-creation made explicit choices for each and reproduces the *mechanism*, not the absolute numbers.

D

Solutions to the Exercises

Numeric answers were computed with the script `textbook/code/exercise_checks.py`, which uses Amber's own functions wherever they apply. Conceptual answers are brief; they indicate the reasoning expected.

Chapter 1

1.1 $E = 2430$ J; $t = 2430/20\,\mu\text{W} = 1.215 \times 10^8$ s $= 3.85$ years. For 15 years the average load must be $2430/(15 \cdot 3.156 \times 10^7) = 5.1\,\mu$W, a factor 3.9 lower.

1.2 $2 \times 10^6/(4 \cdot 365) \approx 1370$ per day. This underestimates the labour: devices fail early, are hard to reach (travel and access dominate the cost), must be located and identified, and real cell life is shortened by self-discharge and temperature.

1.3 Device B (storage, backscatter, no signal generation). With an oscillator and a small power amplifier it can generate its own carrier: Device C.

1.4 Without storage there is no energy when the carrier is absent, and with no carrier there is nothing to reflect, so no uplink is possible. Consequences: replies must be scheduled by the reader (slots) while the carrier is on; no carrier sensing, hence reader-controlled random access; commands must be short and protocols robust to nodes that are asleep.

Chapter 2

2.1 Nodes: position, antenna gain, sensitivity, power-split ratio. Base stations: sectors with azimuth, beamwidth, height, gain, transmit power, sensitivity, antenna pattern. Links: downlink (power and carrier) and uplink (backscatter), computed once.

2.2 (i) A stronger energy source; (ii) a different random start-up delay or capacitor. The demo implements (i): Node 2 is hybrid, its solar term (≈ 1.4 mW) exceeds its RF term (≈ 0.09 mW) and the max-rule selects it.

2.3 Only if it reaches *wait_slot* while $t <$ slot end: processing ends at most 5 ms later, so it can still reply provided the remaining processing time is shorter than the time left in the (chosen) slot.

Chapter 3

3.1 (a) 1.995 W; (b) -26.99 dBm; (c) 1.995×10^{-13} W (0.2 pW).

3.2 $10\log_{10}(10^{-7} + 10^{-7.3}) = -68.24$ dBm. Four equal interferers: $-73 + 6.02 = -66.98$ dBm.

3.3 $-174 + 60 + 8 = -106$ dBm; $-174 + 80 + 6 = -88$ dBm; they differ by 18 dB.

3.4 (a) $30 + 17 - 80 + 1 = -32$ dBm $= 0.63\,\mu$W; (b) yes, 13 dB above -45 dBm; (c) $-32 - 3.01 + 1 - 80 + 17 = -97.0$ dBm.

3.5 $N = -174 + 6 + 10\log B$; need $N = S - \gamma = -103$ dBm, so $10\log B = 65$ and $B = 3.16$ MHz.

Chapter 4

4.1 $17 - 12(40/65)^2 = 12.46$ dBi; $17 - 12(90/65)^2 = -6.0$ dBi; at 150° the attenuation exceeds 30 dB, so $17 - 30 = -13$ dBi.

4.2 The floor is reached at $\varphi_{3\text{dB}}\sqrt{30/12} = 1.58\,\varphi_{3\text{dB}}$: (a) 47.4°; (b) 189.7° $>$ 180°: never; the attenuation at 180° is $12 \cdot (180/120)^2 = 27$ dB.

4.3 Math angles: 90°, 45°, 315°, 180°. A sector at compass 10° is at math 80°; for a node at math 350° the error is $|(350 - 80 + 180) \bmod 360 - 180| = 90°$.

4.4 The node's compass bearing 60° is math 30°. Errors: sector 0, 60°; sector 120, 60°; sector 240, 180°. Gains 4.78, 4.78, -15 dBi. The first two tie; Amber's strict "$>$" keeps the first: sector 0 is the owner.

4.5 $D \approx 41\,253/(90 \cdot 15) = 30.6$, i.e. 14.9 dBi.

Chapter 5

5.1 See the text. At 50 m: 65.7 dB at 924 MHz, 77.3 dB at 3.5 GHz; the difference $20\log(3500/924) = 11.6$ dB arises because the receive aperture $\lambda^2 G/4\pi$ shrinks with frequency.

5.2 $d'_{BP} = 4 \cdot 29 \cdot 1 \cdot 2 \times 10^9/3 \times 10^8 = 773$ m; the node at 200 m is *before* the breakpoint.

5.3 $d_{3D} = 300.9$: $28 + 40\log 300.9 + 20\log 0.924 - 9\log(147.8^2 + 23.5^2) = 28 + 99.14 - 0.69 - 39.15 = 87.30$ dB, matching the table.

5.4 $4.343 \cdot 0.01 \cdot 100 = 4.34$ dB per hop; 8.69 dB over the round trip.

5.5 In this regime the path loss rises by 5.93 dB from 50 to 100 m. The downlink falls by 5.93 dB, the uplink by $2 \times 5.93 = 11.86$ dB.

Chapter 6

6.1 Chain: $P_{\text{tag}} = P_t G_t G_a L(d_1)$, $P_{\text{bs}} = \eta P_{\text{tag}}$, $P_r = P_{\text{bs}} G_a G_r L(d_2)$. Doubling both distances changes each L by -6.02 dB: P_r falls by 12.04 dB; the downlink falls by 6.02 dB.

6.2 $\lambda = 0.328$ m, $\text{PL}_{FS}(4\text{ m}) = 43.71$ dB. Tag captures $36 + 9 + 2 - 43.71 = +3.29$ dBm (2.1 mW); reflects $\eta = 0.3$ (-5.23 dB): -1.94 dBm; reader receives $-1.94 + 2 + 9 - 43.71 = -34.65$ dBm.

6.3 $(0.8/\pi)^2 = 0.065$, i.e. -11.9 dB, so $\eta \approx 0.065$ (before other losses).

6.4 The harvest falls by $10\log(1 - \eta)$: -5.23 dB ($\eta = 0.7$), -0.97 dB ($\eta = 0.2$); for $\eta = 1$ the harvest is 0 (Amber floors it at -120 dBm).

6.5 Nodes with the same shift occupy the same frequency, so replies that overlap in time collide; different shifts are orthogonal in frequency and cannot. This resembles frequency-division multiple access (or a subcarrier group in OFDMA).

Chapter 7

7.1 Using the mean voltage: $2.66^2/5000 = 1.415$ mW (1.51 dBm). The code computes P per sample and the *mean of* V^2 is 1.433 mW (1.56 dBm); the difference is $\sigma^2/R = 12.5\,\mu$W (0.9%).

7.2 Max: $\sqrt{1.4\,\text{mW} \cdot 5\,\text{k}\blacksquare} = 2.65$ V. Sum: $\sqrt{2.4\,\text{mW} \cdot 5\,\text{k}\blacksquare} = 3.46$ V.

7.3 -5.23 dB ($\eta = 0.7$); -0.97 dB ($\eta = 0.2$).

7.4 $V_\infty = V_{\text{src}} \cdot 0.952 \geq 1.7 \Rightarrow V_{\text{src}} \geq 1.785$ V; $P = V_{\text{src}}^2/R_{\text{s}} \geq 637\,\mu\text{W} = -1.96$ dBm.

7.5 Insert $P_h \leftarrow \eta_{\text{rf-dc}}(P_{\text{dl}})P_{\text{dl}}$ in `calculate_node_power`, before the controller converts power to voltage. In the paper's scenario P_{dl} is -12 dBm at 20 m but below -20 dBm beyond ≈ 70 m; those nodes would harvest nothing from RF and, with $\eta_{\text{rf-dc}} = 0.3$, even nodes at 20 m would deliver 19 μW ($V_{\text{src}} \approx 0.31$ V): the dependence on the external source becomes stronger.

Chapter 8

8.1 $\tau = (4.762\,\text{k}\blacksquare)(200\,\mu\text{F}) = 0.952$ s; $V_\infty = 2.368$ V; $t = \tau \ln[2.368/(2.368 - 1.7)] = 1.21$ s.

8.2 $V_\infty = (V_{\text{src}} - 1.5) \cdot 0.952 < 1.3 \Rightarrow V_{\text{src}} < 2.865$ V.

8.3 $\Delta E = \frac{1}{2}(200\,\mu\text{F})(2.89 - 1.69) = 120\,\mu$J. Doubling needs $C = 400\,\mu$F. Lowering V_{low} to 1.1 V gives 168 μJ (+40%); doubling by V_{low} alone would need $V_{\text{low}} = 0.7$ V, impractical.

8.4 Solve $V_{\text{th}} = V_\infty + (V_0 - V_\infty)\text{e}^{-t/\tau}$ for t. For Node 1: $V_\infty = 1.971 \cdot 0.9524 = 1.877$, $t = 1.4286 \ln(1.877/0.177) = 3.37$ s.

8.5 $D = 200\,\mu\text{W}/4\,\text{mW} = 5\%$; $T_c = 180/200 = 0.9$ s; $T_a = 180\,\mu\text{J}/3.8\,\text{mW} = 47$ ms; one burst per 0.947 s, i.e. 1.06 bursts/s.

8.6 $\text{d}[V(V_{\text{src}} - V)]/\text{d}V = V_{\text{src}} - 2V = 0$ at $V = V_{\text{src}}/2$; the maximum is $V_{\text{src}}^2/4R_{\text{s}} = P_h/4$; the delivered power is never above 25% of P_h.

Chapter 9

9.1 See Appendix A.4. A pure-ALOHA packet is vulnerable to a start in either of two packet-times around it, a slotted packet only to the same slot.

9.2 $E_0 = 4.40$, $E_1 = 5.87$, $E_c = 5.73$, $\eta = 0.367$. The optimum is $S = n = 20$: $\eta = (19/20)^{19} = 0.377$.

9.3 $\hat{n} = 5 + 2.39 \cdot 5 = 17.0$; lower bound $5 + 2 \cdot 5 = 15$; use $S \approx 17$.

9.4 $S = 30$: $P_{\text{succ}} = 0.374$, $\eta = 0.374$, 299 μJ; $S = 45$: 0.521, 0.347, 215 μJ; $S = 60$: 0.614, 0.307, 182 μJ; $S = 90$: 0.723, 0.241, 155 μJ. If idle slots cost only airtime, choose S around $2n$–$3n$ as a compromise: energy falls steeply up to $\approx 2n$ and slowly after, while efficiency keeps falling.

9.5 20 nodes per shift: $(1 - 1/10)^{19} = 0.135$; with $K = 1$ and 60 nodes: $0.9^{59} = 0.0020$.

Chapter 10

10.1 $\gamma_1 = 7.93$ dB, $\gamma_2 = 17.98$ dB (after removing the first). Capture alone decodes the first only.

10.2 $\gamma = -3.04$ dB < 3 dB: nothing is decoded and SIC stops at the first step.

10.3 $\rho \geq 1 + \gamma_{\text{req}} = 3.0$ (4.76 dB) per step; three steps: $\rho^3 = 27$, i.e. 14.3 dB.

10.4 First packet $i = 3$: group $\{3, 7\}$ (since $|3 - 11| = 8 > 5$); then $i = 11$: group $\{11, 15\}$. Two groups of two. Transitive clustering would chain $3 \sim 7 \sim 11 \sim 15$ into one group of four.

10.5 $\varepsilon = 0.05$: $\gamma = (3.79, 3.19, 1.71)$ dB $\rightarrow$ first two decoded, third lost. $\varepsilon = 0.1$: $\gamma = (3.79, 1.96)$ dB $\rightarrow$ only the first. $\varepsilon = 0$: all three. Amber's implementation always behaves as $\varepsilon = 0$.

Chapter 11

11.1 $p_a = 1$: round-1 registered fraction 0.19; $p_a = \frac{1}{2}$: 0.22 (fewer collisions among participants), but 90% is reached after 5 rounds versus 6: it lags by one round.

11.2 Round $= 100 \cdot 15 = 1500$ ms. Each node is polled every 1.5 s but can answer once per 2 s: about $1.5/2 = 75\%$ of polls answered.

11.3 Unicast never wastes node energy on collisions: energy per delivered packet is E_{tx} rather than $E_{\text{tx}}/P_{\text{succ}}$ (e.g. 299 μJ vs 112 μJ). Commands cost base-station airtime, not node energy.

11.4 (i) Time since the last reply (compare with the node's recharge time); (ii) an energy flag or voltage bucket piggy-backed on data (a few bits); (iii) response to a cheap ping slot (a transmission); (iv) RSSI history: a node that answered at a good RSSI recently is probably asleep, not out of range.

Chapter 12

12.1 The charger's increments at $t = 1, \ldots, 5$ all precede the interrupt at 5.5: final charge 5.

12.2 Every controller reschedules itself 1 ms ahead, so the future-event list is never empty between ms ticks. To skip idle time, advance each capacitor analytically (Eq. (8.4)) between the *events that change its inputs* (load changes, state transitions, $V_{\text{low}}/V_{\text{high}}$ crossings); one would lose nothing in the linear regime but the harvest depends on a 1 ms trace, so the benefit is limited to RF-only or piecewise-constant sources.

12.3 $500 \cdot 120\,000 \cdot 13\,\mu\text{s} = 780$ s. A 100-node 20 s run takes 26 s, so about 138 replications per hour on one core.

12.4 A's controller runs, calls `do_transmit`: it writes the scratch fields and interrupts the base station (urgent). The controller yields; the urgent interrupt is processed immediately, so the base station buffers A. Only then does B's controller (a normal event) run and overwrite the scratch fields and interrupt. We verified this with three same-millisecond transmissions: all were buffered intact.

12.5 Each run is one sample of random placements, delays and slot choices. Plan: densities $\{5, 10, 20, 40, 60, 80, 100\}$, 10–30 seeds each, one factor varied, report median and inter-quartile range per density (box plots), same seeds across compared configurations.

Chapter 13

13.1 Sequence: BS `_do_tx` $\rightarrow$ `receive_command` $\rightarrow$ `handle_command` (node state); each ms `Controller` $\rightarrow$ `controller_tx_ready`; if ready $\rightarrow$ `do_transmit` $\rightarrow$ BS interrupt (buffer); end of window: BS `_process_rx_buffer`.

13.2 Provide `voltage`, `voltage_source`, `current`, `internal_time`, an `action` process accepting `capacitor_charging`, `capacitor_full`, `capacitor_charge_and_discharge`. Voltage-dependent leakage and ESR live inside your step functions; also reconsider the controller's hard-coded R_s.

13.3 The uplink uses `backscatter_power_dbm`, written by `calculate_node_power`. If missing, `getattr` falls back to -999 dBm, so every RSSI would be at or below the floor and no packet would pass sector sensitivity.

13.4 Objects: 2 `BaseStation` (6 `Sector`), 50 `Node`, 50 each of `Capacitor`, `BackscatterModule`, `Controller`, 2 `BSBehavior`, ≥ 1 `CoverageMap`, optionally an `EnvEnergySource`. Processes: $50 + 50 + 2 + 1 = 103$.

Chapter 14

14.1 Bearing 53.1°; errors 36.9° (sector 0) and 83.1° (sector 120); gains 11.14 and -4.63 dBi; sector 0 is the owner. $\mathrm{PL}(100\,\mathrm{m}) = 71.57$ dB, so $P_{\mathrm{dl}} = 46 + 11.14 - 71.57 = -14.43$ dBm.

14.2 It propagates the node's `backscatter_power_dbm`, which already contains the base-station power, gains and path loss of the downlink; that attribute comes from `calculate_node_power`.

14.3 `do_transmit` loops over both base-station processes: for each it selects the best sector for the node, writes the scratch fields and interrupts. Each base station buffers the packet in its current RX window (if any), resolves its own collisions, and applies its sector's sensitivity; the packet may be logged by both.

14.4 Change `calculate_node_power`: sum the sectors' powers from `downlink_results["per_node_powers"][node_id]` (in watts) instead of reading `best_pw_w`. The data is already in the dictionary.

Chapter 15

15.1 $83.1\,\mu\mathrm{C} \times 1.7 = 141\,\mu\mathrm{J}$ and $\times 1.3 = 108\,\mu\mathrm{J}$: the energy cost varies by $\pm 13\%$ with the voltage (for fixed charge, $E \propto V$).

15.2 Allowed drop 0.05 V: $t = C\Delta V / I = 3$ ms (20% of the burst). A complete burst needs $V \geq V_{\mathrm{low}} + 0.25 = 1.55$ V at entry.

15.3 In *wait_slot* refuse to call `controller_tx_ready` unless $V \geq V_{\mathrm{low}} + I_{\mathrm{tx}} T_{\mathrm{tx}} / C$ (plus margin). Delivered counts fall (by up to ≈half in the broadcast demo) but become energy-causal.

15.4 Delays de-synchronise nodes. With zero delay and identical parameters all nodes would activate and discharge in lock-step, causing synchronous contention spikes and correlated unavailability.

Chapter 16

16.1 $-50 < -45$ dBm: the sensitivity gate returns at once; no log, no slot stored, no state change.

16.2 For integer ms: $205 \leq t \leq 212$. If the command was sent at $t = 206$, slot_start $= 205 < 206$: the stale-slot check always fails (such a slot cannot occur in normal operation).

16.3 Replace the test by: `ifisinstance(target,list)andself.node.idnotintarget:return`; then proceed to set `chosen_slot_idx` and `last_tx_command_time`. Note that the base station truncates `rx_slots` to one entry for any target $\neq -1$, so all listed nodes would share one slot; send separate commands or extend `_deliver_command`.

16.4 Pass the capacitor object to your subclass constructor, keep it as an attribute, and test `self.cap.voltage` in `controller_tx_ready` (returning `None` below 1.6 V), or give the controller the check.

Chapter 17

17.1 Frame start 45; node 1's command at 60 (TX 60–65). Scanning: RX windows $(65, 75)$, $(80, 90)$, $(95, 105)$, ...; the first, $(65, 75)$, is kept.

17.2 The probability is $S/2S = \frac{1}{2}$; the expected number of the 12 ready nodes that transmit is 6.

17.3 Phase 1: while `len(bs.nodes_registered)<0.9*N`, yield [tx send_id, 8 rx, tx ack]; phase 2: loop over registered ids yielding [tx poll(i), rx]; the infinite loop keeps the process alive (a finite one needs a long final RX window).

17.4 `start_ms` is the *window* start; use `end_ms` (the arrival time), the instant the model treats as the packet (it has no duration).

Chapter 18

18.1 DER = 10%; PDR = 90%; throughput $117/20.001 = 5.85$ pkt/s; audible PDR $117/124 = 94.4\%$.

18.2 A packet arriving at the window's first ms has start = end. The sweep orders "end" before "start" at equal times, so for packets p_1 at 100 and p_2 at 125 the order is $\text{end}_{p_1}, \text{start}_{p_1}, \text{end}_{p_2}, \text{start}_{p_2}$: after start_{p_1} the set `active` contains p_1 for ever; at start_{p_2} it is non-empty, so p_2 "collides" with the ghost.

18.3 Count *opportunities*: polls to nodes that, according to the simulator's ground truth, were in *wait_slot* at poll time. Losses among opportunities are radio losses; the rest are energy losses. The ground truth (controller state) is not observable in a real network.

Chapter 19

19.1 All nodes are on boresight ($\varphi = 0$), so the beamwidth does not change any gain (15 dBi) and the budget table is unchanged.

19.2 Using $P_{\text{in}} \approx P_h x(1-x)$ at $V \approx 1.5$ V: Node 0 ≈ 0.30 mW, Node 1 ≈ 0.14 mW; ratio ≈ 2.1 versus the measured $46/18 = 2.6$. The theorem's most violated assumption is a *constant* P_{in}; it varies with V, the active-mode step over-credits it, and truncated bursts distort the counts.

19.3 Operating point ($n = 3.9$, $S = 8.4$): 0.32; $S = n$: 0.42; $S = 2n$: 0.34. The demo sits close to the $2n$ rule, with a small n.

19.4 Run the demo with positions from $[5, 40]$ and $[10, 40]$ using the same angles and seeds (10 each), compare total throughput and the spread of per-node counts.

Chapter 20

20.1 In `handle_command`: compute the margin = best_pw_dbm − node.sensitivity_dbm and return if it is below 3. Physically, a weak command is unreliable for an envelope detector, and by reciprocity a weak downlink predicts a weak uplink; it avoids wasting energy at the cell edge.

20.2 Exponential back-off cannot exceed the energy bound (≈ 34 pkt/s in this scenario); expect a gain of at most a few percent, as for fixed penalties.

20.3 `bs.nodes_registered`, `bs.decoded_this_frame`, `bs.nodes_pending_ack`.

20.4 Pass the capacitor to the module's constructor, store it, and test its voltage in `controller_tx_ready`.

Chapter 21

21.1 Example (M1): in `apply_sic` add `sum(residual_w[j]forjindecoded)` to the interference of the remaining candidates. Show, for $\kappa = 0.9$, that the second packet of Figure 10.2 is no longer decoded; run the ablation of Table 10.1 with the fix.

21.2 Check M1 (perfect SIC), M2 (the 5 ms collision definition) and E4/M4 (delivered counts). Experiment: re-run the ablation with imperfect cancellation, paired seeds, several κ.

21.3 Node 4 is 6.3 m from the mast, inside the < 10 m path-loss branch (R1): its RF power (8 mW) exceeds the solar source, so in hybrid mode it recharges faster. Location matters through *energy*, via a modelling artefact, not through a better radio link or a better protocol. Test by moving all nodes beyond 10 m.

E

Glossary

ACK
Acknowledgement. In Amber a direct call that moves a node from *wait_ack* to *registered*.

Ambient IoT
3GPP's concept of battery-free or energy-harvesting IoT devices that communicate by backscatter; studied in Releases 18–20.

Backscatter
Communication by modulating the reflection of an external carrier (Chapter 6).

Brown-out
Voltage falling below the operating threshold during activity; in Amber the node switches off when $V \leq V_{\text{low}}$.

Capture effect
Decoding the strongest of several overlapping packets because it exceeds the others by the SINR threshold.

Carrier (tone)
The unmodulated sinusoid radiated by the base station; powers and illuminates nodes.

Cold start
The problem that a rectifier produces no usable DC below a minimum input power.

Collision window
Amber's 5 ms rule: packets on the same subcarrier arriving within 5 ms of each other are treated as colliding.

Coverage map
A grid of best-sector received power, used for plots; not used by the simulation itself.

Device A / B / C
3GPP Ambient IoT device categories: no storage and backscatter; storage and backscatter; storage and active RF.

DER
Delivery error rate: $1 -$ decoded/sent in the demos (but a success ratio in `BSStats.der`).

dBi
Antenna gain in decibels relative to an isotropic radiator.

dBm
Power in decibels relative to one milliwatt.

Discrete-event simulation
Simulation advancing time from event to event.

Duty cycle
Fraction of time a node is active; $D = P_{\text{in}} / P_{\text{act}}$ in the long run.

Duck typing
Interfaces defined by the attributes and methods an object has, not by its class.

Energy window
$\frac{1}{2}C(V_{\text{high}}^2 - V_{\text{low}}^2)$: energy usable between the two thresholds.

Envelope detector
A diode and low-pass filter that recovers the amplitude of a received signal; a passive receiver for OOK.

ESR
Equivalent series resistance of a capacitor (not modelled separately in Amber).

FSA
Framed slotted ALOHA: the reader announces a frame of S slots; each node picks one at random.

Friis equation
$P_r = P_t G_t G_r (\lambda/4\pi d)^2$.

FSPL
Free-space path loss.

Hybrid mode
Harvest from both RF and an external source; Amber's default combination is *max*.

Hysteresis
Two different thresholds for switching on and off.

Interrupt
A SimPy message that throws an exception into a process; scheduled urgently.

Intermittent computing
Computation that survives frequent power failures.

Link budget
The accounting of gains and losses from transmitter to receiver.

LOS / NLOS
Line of sight / non line of sight.

Look-ahead
The base station's scan of its schedule to advertise upcoming RX windows (two frames in Amber).

Monostatic / bistatic
Carrier source and receiver co-located / separate.

Neper
Natural-log unit of attenuation; 8.686 dB per neper for amplitudes.

Noise figure
Ratio by which a receiver degrades the SNR.

OOK
On–off keying.

Owner (sector)
The sector delivering the highest power at a point.

Policy
A generator of frames driving a base station adaptively.

PRDCH
Physical reader-to-device channel (3GPP); in Amber's paper a dual-tone OFDM-based structure.

Power split ratio η
Fraction of received power reflected rather than harvested (`Node.efficiency`).

RSSI
Received signal strength indicator; here the received power in dBm.

Rectenna
Rectifying antenna: antenna plus rectifier converting RF to DC.

SIC
Successive interference cancellation.

SINR
Signal-to-interference-plus-noise ratio.

Sector
One directional antenna panel of a base station.

Sensitivity
Weakest signal a receiver can use; a configured threshold in Amber.

SimPy
Python process-based discrete-event simulation library.

Slot
One RX window advertised by a command.

Subcarrier shift
Frequency offset (integer label in Amber) at which a node reflects its reply.

Thermal noise
Noise power $k_B T B F$.

UMa
Urban macro scenario of 3GPP TR 38.901.

Unicast / broadcast
A command addressed to one node / to all.

WPT
Wireless power transfer: harvesting from the base station's carrier.

F

Reproducing the Results

Everything quantitative in this book can be regenerated. The scripts live in `textbook/code` next to the book sources; they use the unmodified `amber` package and the shipped demo files.

F.1 Setting up

```
git clone https://github.com/mrnmir/Amber.git && cd Amber
pip install -e .            # numpy, pandas, simpy, matplotlib, openpyxl
python demo_experiments/broadcast/broadcast_demo.py
```

The four demos write CSV files into their `output` folders (overwriting the committed ones); to avoid this, run them from a copy of the folder as we did.

F.2 Scripts

Script	What it produces
`make_figures.py`	all data-driven figures (path loss, antenna, budgets, capacitor, saw-tooth, FSM zoom, ALOHA, coverage, registration, ...)
`registration_experiment.py`	Fig. 11.2: registration versus density, SIC on/off (about 2 min on 4 cores)
`sic_ablation.py`	Table 10.1: one-factor ablation of the broadcast demos
`polling_experiment.py`	Table 20.1: round-robin versus skip-silent polling
`energy_accounting_test.py`	the $\approx 2.7\%$ effect of the active-mode energy credit
`scalability_benchmark.py`	run time versus node count (Chapter 12)
`exercise_checks.py`	numeric answers of Appendix D
`normalize_code_macros.py`	LaTeX hygiene helper used while writing

Typical invocation (from `textbook/code`):

```
python make_figures.py                          # regenerate figures
python registration_experiment.py 10 results/registration_results.json
python sic_ablation.py 10 results/sic_ablation.json
python polling_experiment.py 10 results/polling.json
```

F.3 Building the book

```
cd textbook
latexmk -pdf main.tex        # needs a TeX Live with tikz, circuitikz, tcolorbox, siunitx,
    algorithmicx, natbib
```

F.4 What is deterministic

The custom-protocol demo seeds Python's `random` (`SEED=7`) and reproduces its committed output exactly; our ablations use explicit seeds per run, so their numbers reproduce; the three-node demos are unseeded, so single runs differ by a few packets (use the ten-seed means).

F.5 Repository map

Path	Content
`amber/`	the eight modules (Chapter 13)
`demo_experiments/`	four demos, their output and the solar trace
`docs/`, `mkdocs.yml`	API reference generated from docstrings
`textbook/`	this book: `main.tex`, `chapters/`, `figures/`, `code/`

Bibliography

[1] 3GPP. NR; ambient IoT device radio transmission and reception. Technical Specification TS 38.191, 3rd Generation Partnership Project, . Release 19.

[2] 3GPP. Study on solutions for ambient IoT (internet of things) in NR. Technical Report TR 38.769, 3rd Generation Partnership Project, . Release 19.

[3] 3GPP. Study on ambient IoT (internet of things) in RAN. Technical Report TR 38.848, 3rd Generation Partnership Project, . Release 18.

[4] 3GPP. Study on channel model for frequencies from 0.5 to 100 GHz. Technical Report TR 38.901, 3rd Generation Partnership Project, 2020. v16.1.0, Release 16.

[5] Norman Abramson. The ALOHA system: Another alternative for computer communications. In *Proceedings of the AFIPS Fall Joint Computer Conference*, pages 281–285, 1970.

[6] Jerry Banks, John S. Carson II, Barry L. Nelson, and David M. Nicol. *Discrete-Event System Simulation*. Pearson, 5th edition, 2010.

[7] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2nd edition, 2006.

[8] Daniel M. Dobkin. *The RF in RFID: UHF RFID in Practice*. Newnes, 2nd edition, 2012.

[9] EPCglobal. EPC radio-frequency identity protocols class-1 generation-2 UHF RFID protocol for communications at 860 mhz–960 mhz. Technical Report Version 1.2.0, EPCglobal Inc., 2008.

[10] Harald T. Friis. A note on a simple transmission formula. *Proceedings of the IRE*, 34(5): 254–256, 1946.

[11] Jakob Hoydis, Sebastian Cammerer, Fayçal Ait Aoudia, Avinash Vem, Nikolaus Binder, Guillermo Marcus, and Alexander Keller. Sionna: An open-source library for next-generation physical layer research. *arXiv preprint arXiv:2203.11854*, 2022.

[12] Nguyen Van Huynh, Dinh Thai Hoang, Xiao Lu, Dusit Niyato, Ping Wang, and Dong In Kim. Ambient backscatter communications: A contemporary survey. *IEEE Communications Surveys & Tutorials*, 20(4):2889–2922, 2018.

[13] Averill M. Law. *Simulation Modeling and Analysis*. McGraw-Hill, 5th edition, 2015.

[14] Vincent Liu, Aaron Parks, Vamsi Talla, Shyamnath Gollakota, David Wetherall, and Joshua R. Smith. Ambient backscatter: Wireless communication out of thin air. In *Proceedings of ACM SIGCOMM*, pages 39–50, 2013.

[15] O. López et al. Zero-energy devices for 6G: Technical enablers at a glance. *IEEE Internet of Things Magazine*, 8(3):14–22, 2025.

[16] Brandon Lucia, Vignesh Balaji, Alexei Colin, Kiwan Maeng, and Emily Ruppel. Intermittent computing: Challenges and opportunities. In *2nd Summit on Advances in Programming Languages (SNAPL)*, 2017.

[17] Mirana Manafova, Thiemo Voigt, and Luca Mottola. Demo abstract: Amber: 6G ambient IoT simulator. In *2026 ACM/IEEE International Conference on Embedded Artificial Intelligence and Sensing Systems – Posters and Demos (SenSys-Adjunct)*, pages 23–24, 2026. doi: 10.1109/SenSys-Adjunct71932.2026.00021.

[18] David M. Pozar. *Microwave Engineering*. Wiley, 4th edition, 2011.

[19] Lawrence G. Roberts. ALOHA packet system with and without slots and capture. *ACM SIGCOMM Computer Communication Review*, 5(2):28–42, 1975.

[20] Frits C. Schoute. Dynamic frame length ALOHA. *IEEE Transactions on Communications*, 31 (4):565–568, 1983.

[21] SimPy Team. SimPy: Discrete event simulation for Python. https://simpy.readthedocs.io. Accessed 2026.

[22] Harry Stockman. Communication by means of reflected power. *Proceedings of the IRE*, 36 (10):1196–1204, 1948.

[23] William Tärneberg, Andrei Fedorov, Gilles Callebaut, Liesbet Van der Perre, and Emma Fitzgerald. Towards practical cell-free 6G network deployments: An open-source end-to-end ray tracing simulator. In *2023 57th Asilomar Conference on Signals, Systems, and Computers*, pages 1000–1005, 2023.

[24] David Tse and Pramod Viswanath. *Fundamentals of Wireless Communication*. Cambridge University Press, 2005.

[25] Fengyuan Zhu, Luwei Feng, Meng Jin, Xiaohua Tian, Xinbing Wang, and Chenghu Zhou. Towards ultra-low power OFDMA downlink demodulation. In *Proceedings of the 20th ACM Conference on Embedded Networked Sensor Systems (SenSys)*, pages 725–739, 2022.